

KOMBINASI *STABLE DIFFUSION* DENGAN *CONTROLNET* BERBASIS CITRA
TEPI *CANNY* UNTUK *IMAGE-TO-IMAGE TRANSLATION* DALAM PEMBUATAN
BACKGROUND ART BERGAYA ANIMASI

TUGAS AKHIR



Disusun Oleh :
ROWANG PRAMUDITO
123200098

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026

**KOMBINASI *STABLE DIFFUSION* DENGAN *CONTROLNET* BERBASIS CITRA
TEPI *CANNY* UNTUK *IMAGE-TO-IMAGE TRANSLATION* DALAM PEMBUATAN
BACKGROUND ART BERGAYA ANIMASI**

TUGAS AKHIR

Tugas Akhir ini sebagai salah satu syarat untuk memperoleh gelar sarjana Informatika
Universitas Pembangunan Nasional “Veteran” Yogyakarta



Disusun Oleh :
ROWANG PRAMUDITO
123200098

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
YOGYAKARTA
2026

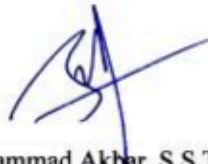
HALAMAN PENGESAHAN PEMBIMBING

**KOMBINASI *STABLE DIFFUSION* DENGAN *CONTROLNET* BERBASIS CITRA
TEPI CANNY UNTUK *IMAGE-TO-IMAGE TRANSLATION* DALAM
PEMBUATAN *BACKGROUND ART* BERGAYA ANIMASI**

Disusun Oleh:
Rowang Pramudito
123200098

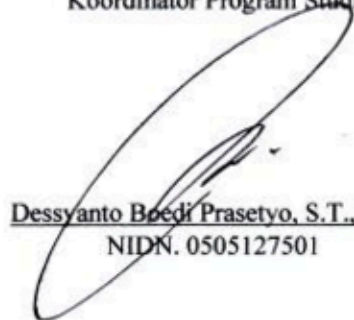
Telah diuji dan dinyatakan lulus oleh pembimbing
pada tanggal: 20 Februari 2026

Menyetujui,
Pembimbing



Bagus Muhammad Akbar, S.S.T., M.Kom.
NIDN. 0001088905

Mengetahui,
Koordinator Program Studi



Dessyanto Boedi Prasetyo, S.T., M.T.
NIDN. 0505127501

HALAMAN PENGESAHAN PENGUJI

KOMBINASI *STABLE DIFFUSION* DENGAN *CONTROLNET* BERBASIS CITRA TEPI CANNY UNTUK *IMAGE-TO-IMAGE TRANSLATION* DALAM PEMBUATAN *BACKGROUND ART* BERGAYA ANIMASI

Disusun Oleh:
Rowang Pramudito
123200098

Telah diuji dan dinyatakan lulus oleh penguji
pada tanggal: 20 Februari 2026

Menyetujui,

Penguji 1



Bagus Muhammad Akbar, S.S.T., M.Kom.
NIDN. 0001088905

Penguji 2



Bambang Yurwono, S.T., M.T.
NIDN. 0512027401

Penguji 3



Frans Richard K., S.T., M.Kom., Ph.D
NIDN. 0523026201

Penguji 4



Dhimas Arief Dharmawan, S.T., Ph.D
NIDN. 0004079305

**SURAT PERNYATAAN
KARYA ASLI TUGAS AKHIR**

Sebagai mahasiswa Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta, yang bertanda tangan di bawah ini, saya:

Nama : Rowang Pramudito

NIM : 123200098

Menyatakan bahwa karya ilmiah saya yang berjudul:

Kombinasi *Stable Diffusion* dengan *ControlNet* berbasis Citra Tepi *Canny* untuk *Image-to-Image Translation* dalam Pembuatan *Background Art* bergaya Animasi

Merupakan karya asli saya dan belum pernah dipublikasikan di manapun. Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, maka saya bersedia menerima konsekuensi apa pun yang diberikan oleh Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta kepada saya.

Demikian surat pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Yogyakarta

Pada Tanggal :

Yang Menyatakan

materai

Rowang Pramudito

123200098

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama : Rowang Pramudito
NIM : 123200098
Prodi : Informatika

Dengan ini menyatakan bahwa judul Tugas Akhir

Kombinasi *Stable Diffusion* dengan *ControlNet* berbasis Citra Tepi *Canny* untuk *Image-to-Image Translation* dalam Pembuatan *Background Art* bergaya Animasi

adalah hasil kerja saya dan benar bebas dari plagiasi kecuali cuplikan serta ringkasan yang terdapat di dalamnya telah saya jelaskan sumbernya (sitasi) dengan jelas. Hasil karya asli saya dan belum pernah dipublikasikan dimanapun. Apabila pernyataan ini terbukti tidak benar maka saya bersedia menerima sanksi sesuai peraturan Mendiknas RI No. 17 Tahun 2010 dan peraturan perundang-undangan yang berlaku.

Demikian surat pernyataan ini saya buat dengan penuh tanggung jawab.

Dibuat di : Yogyakarta
Pada Tanggal :

Yang Menyatakan

materai

Rowang Pramudito
123200098

ABSTRAK

Proses produksi animasi konvensional masih sangat bergantung pada pengerjaan manual, di mana sebagian besar visual terkait dibuat berdasarkan referensi pemandangan dunia nyata. Namun, transformasi adegan dunia nyata ke dalam gaya visual animasi merupakan proses yang kompleks dan memerlukan keahlian teknis tinggi. Oleh karena itu, diperlukan teknik yang secara otomatis mampu mentransformasi adegan dunia nyata menjadi gambar bergaya animasi. Salah satu solusi terhadap permasalahan ini adalah implementasi *Latent Diffusion Model* (LDM) melalui pendekatan *image-to-image translation*. Namun terdapat keterbatasan dalam model difusi, yaitu distorsi bentuk dan inkonsistensi struktur objek pada hasil generasi. Untuk mengatasi keterbatasan tersebut, diperkenalkan ControlNet yang memungkinkan model difusi untuk menambahkan pengondisian spasial untuk mempertahankan struktur utama dari citra referensi.

Dalam penelitian ini, *Latent Diffusion Model* (LDM), melalui model terlatih Stable Diffusion, dikombinasikan dengan ControlNet berbasis peta tepi Canny, digunakan untuk meningkatkan ketahanan struktur konten hasil *image-to-image translation* bergaya animasi. Metodologi penelitian dimulai dengan pengumpulan data latih, yang terdiri dari 270 gambar tangkapan layar film buatan Studio Ghibli diambil dari situs web Fanscap. Data kemudian diproses melalui tahap eliminasi duplikat dan kurasi secara manual. Selanjutnya, dilakukan pelatihan *Low-Rank Adaptation* (LoRA) agar model dapat mengerjakan tugas sesuai dengan objek penelitian yang diangkat. Pada tahap inferensi, model dasar Stable Diffusion digabungkan dengan ControlNet terlatih berbasis citra tepi Canny dan LoRA yang sudah dilatih sebelumnya.

Evaluasi model dilakukan terhadap 20 citra fotografis yang diambil dari situs web Unsplash sebagai citra referensi. Pengujian dilakukan menggunakan tiga metrik pengukuran, yaitu *Structural Similarity Index Measure* (SSIM) dan *Learned Perceptual Image Patch Similarity* (LPIPS) sebagai metrik pengukuran ketahanan struktural, dan penilaian oleh pakar sebagai metrik pengukuran kualitas tekstur. Hasil Inferensi tanpa ControlNet menghasilkan rata-rata nilai SSIM 0,37, LPIPS 0,588, dan skor penilaian pakar 8,5. Sementara itu, inferensi menggunakan ControlNet menghasilkan rata-rata nilai SSIM 0,461, LPIPS 0,475, dan skor penilaian pakar 6,7. Berdasarkan hasil pengujian tersebut, ditemukan bahwa terdapat peningkatan kualitas yang konsisten pada nilai SSIM dan LPIPS, yang mengindikasikan ketahanan struktur konten yang lebih baik pada citra output. Namun, dibalik peningkatan ketahanan struktural tersebut, terdapat trade-off yaitu turunnya kualitas tekstur dari citra output, yang tercermin dari penurunan rata-rata skor penilaian pakar.

Kata Kunci: image-to-image translation, stable diffusion, image generation, canny edge detection

ABSTRACT

The conventional animation production process remains heavily reliant upon manual execution, wherein the majority of related visuals are created based on references to real-world scenery. Nevertheless, the transformation of real-world scenes into animation visual styles constitutes a complex process requiring highly specialized technical expertise. Consequently, there exists a necessity for techniques capable of automatically transforming real-world scenes into animation-styled imagery. One solution to address this problem is the implementation of Latent Diffusion Model (LDM) through an image-to-image translation approach. However, there are limitations inherent in diffusion models, namely shape distortion and structural inconsistency of objects in the generated results. To overcome these limitations, ControlNet has been introduced, which enables diffusion models to add spatial conditioning to preserve the principal structure from the reference image.

In this research, the Latent Diffusion Model (LDM), using the pre-trained Stable Diffusion model, combined with ControlNet based on Canny edge maps, is employed to enhance the structural robustness of content in animation-styled image-to-image translation results. The research methodology commenced with the collection of training data, consisting of 270 screenshot images from films produced by Studio Ghibli, obtained from the Fanscap website. The data was subsequently processed through duplicate elimination and manual curation stages. Following this, Low-Rank Adaptation (LoRA) training was conducted to enable the model to perform tasks in accordance with the research objectives being addressed. During the inference stage, the base Stable Diffusion model was integrated with the pre-trained ControlNet based on Canny edge imagery and the previously trained LoRA.

Model evaluation was performed on 20 photographic images obtained from the Unsplash website serving as reference images. Testing was conducted utilizing three measurement metrics, namely Structural Similarity Index Measure (SSIM) and Learned Perceptual Image Patch Similarity (LPIPS) as structural robustness measurement metrics, and expert assessment as a texture quality measurement metric. Inference results without ControlNet yielded average values of SSIM 0.37, LPIPS 0.588, and an expert assessment score of 8.5. Meanwhile, inference utilizing ControlNet produced average values of SSIM 0.461, LPIPS 0.475, and an expert assessment score of 6.7. Based upon these testing results, it was discovered that there exists a consistent quality improvement in SSIM and LPIPS values, indicating superior content structural robustness in the output images. However, behind this structural robustness enhancement, there exists a trade-off, namely the degradation of texture quality in the output images, reflected in the decrease of average expert assessment scores.

Keywords: image-to-image translation, stable diffusion, image generation, canny edge detection

KATA PENGANTAR

Puji syukur kepada Tuhan Yang Maha Esa, atas berkatNya, penulis dapat menyelesaikan tugas akhir ini yang berjudul “KOMBINASI STABLE DIFFUSION DENGAN CONTROLNET BERBASIS CITRA TEPI CANNY UNTUK IMAGE-TO-IMAGE TRANSLATION DALAM PEMBUATAN BACKGROUND ART BERGAYA ANIMASI.”

Penulisan skripsi diajukan untuk memenuhi salah satu syarat kelulusan dalam jenjang perkuliahan Strata 1 Universitas Pembangunan Nasional “Veteran” Yogyakarta. Penyelesaian skripsi ini tidak lepas dari bimbingan, bantuan, dan saran berbagai pihak yang telah membantu penulis mengatasi berbagai tantangan.

Penulis menyadari bahwa skripsi ini masih memiliki kekurangan dan terbuka terhadap kritik serta saran yang membangun. Dalam penulisan skripsi ini, penulis menerima banyak bantuan dari berbagai pihak. Dalam kesempatan ini, penulis dengan tulus hati mengucapkan terima kasih kepada:

1. Ayahnya dan ibunya atas dukungan dan semangat,
2. Bapak Bagus Muhammad Akbar, S.ST., M. Kom, selaku dosen pembimbing,
3. Bapak Bambang Yuwono, S.T., M.T., Frans Richard K., S.T., M.Kom., Ph.D, dan Dhimas Arief Dharmawan, S.T., Ph.D selaku dosen penguji,
4. Bapak Simon Pulung Nugroho, S.Kom., M.Cs.selaku dosen wali,
5. Bapak dan Ibu dosen program studi Informatika Fakultas Teknik Industri yang telah membagikan ilmu dan pengalamannya,
6. Berbagai pihak yang telah memberi bantuan dalam penyusunan skripsi ini.

Penulis berharap skripsi ini dapat memberikan manfaat bagi perkembangan ilmu Informatika dan dapat dikembangkan lebih baik lagi.

Yogyakarta, 11 Februari 2026

Penulis

DAFTAR ISI

BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan Penelitian	2
1.5 Manfaat Penelitian	2
1.6 Tahapan Penelitian	3
1.6.1 Studi Pustaka	3
1.6.2 Pengambilan Data	3
1.6.3 Pengolahan Data	3
1.6.4 Pengembangan Sistem	3
1.6.5 Evaluasi Sistem	3
1.7 Sistematika Penulisan	3
BAB II TINJAUAN LITERATUR	5
2.1 Animasi (Film)	5
2.1.1 <i>Background Art</i>	5
2.2 <i>Image Processing</i>	6
2.3 <i>Computer Vision</i>	6
2.4 <i>Image-to-Image Translation</i>	7
2.5 Model Difusi	8
2.6 <i>Latent Diffusion Model</i>	8
2.7 <i>Autoencoder</i>	9
2.8 <i>Variational Autoencoder</i>	10
2.9 Arsitektur U-Net	11
2.10 <i>Convolutional Layer</i>	12
2.11 <i>Layer Normalization</i>	13
2.12 <i>Group Normalization</i>	14
2.13 <i>Sigmoid Linear Unit (SiLU)</i>	14
2.14 Transformer Block	15
2.15 ResNet Block	16
2.16 <i>Stable Diffusion</i>	17
2.17 ControlNet	18
2.17.1 ControlNet untuk Difusi <i>Text-to-Image</i>	19
2.18 Canny	20

2.19	LoRA dan LoCon	21
2.20	Metode Evaluasi	22
2.20.1	Structural Similarity Index (SSIM)	22
2.20.2	Learning Perceptual Image Patch Similarity (LPIPS).....	24
2.21	Diffusers	25
2.22	Studi Pustaka (<i>State-of-the-Art</i>)	26
BAB III METODOLOGI PENELITIAN		29
3.1	Metodologi Penelitian	29
3.2	Pengumpulan Data	30
3.3	<i>Pre-Processing</i> Data	31
3.4	<i>Fine-Tuning</i> Stable Diffusion	32
3.5	Inferensi <i>Stable Diffusion</i>	33
3.5.1	Citra Tepi Canny	34
3.5.2	<i>Variational Autoencoder</i>	39
3.5.3	Arsitektur U-Net	40
3.6	Pengujian	44
3.6.1	SSIM	44
3.6.2	LPIPS	47
3.6.3	Penilaian oleh Pakar	47
3.7	Analisis Kebutuhan Data	48
3.7.1	Jenis dan Sumber Data	48
3.7.2	Prosedur Pengumpulan Data	48
3.8	Analisis Kebutuhan Sistem	48
3.8.1	Kebutuhan Fungsional	48
3.8.2	Kebutuhan Non-Fungsional	48
3.9	Perancangan Sistem	49
3.9.1	Perancangan Antar Muka	49
3.9.2	Pengujian Sistem	50
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN		51
4.1	Implementasi	51
4.1.1	Pengumpulan Data	51
4.1.2	Pembuatan Model (<i>Fine-Tuning Stable Diffusion</i>)	52
4.1.3	Implementasi Sistem	58
4.2	Hasil	61
4.2.1	Hasil Pengujian Model	61
4.2.2	Hasil Pengujian Sistem	64

4.3 Pembahasan	65
BAB V PENUTUP	68
5.1 Kesimpulan.....	68
5.2 Saran.....	68
DAFTAR PUSTAKA	69
LAMPIRAN	72

DAFTAR TABEL

Tabel 2. 1 State-of-the-art.....	26
Tabel 3. 1 functional test case pengujian sistem.....	50
Tabel 4. 1 Tabel hasil pengujian SSIM.....	62
Tabel 4. 2 Tabel hasil pengujian LPIPS.....	63
Tabel 4. 3 Tabel hasil pengujian pakar.....	64
Tabel 4. 4 functional test case pengujian sistem.....	64
Tabel 4. 5 Rata-rata skor SSIM, LPIPS.....	67

DAFTAR GAMBAR

Gambar 2. 1 Beberapa contoh pengaplikasian Image-to-image Translation.....	7
Gambar 2. 2 Struktur Latent Diffusion Model (LDM).....	9
Gambar 2. 3 Struktur dasar autoencoder.....	10
Gambar 2. 4 Arsitektur U-Net.....	12
Gambar 2. 5 Contoh konvolusi dua dimensi dengan kernel 2 x 2.....	13
Gambar 2. 6 Grafik perbandingan SiLU dengan ReLU.....	15
Gambar 2. 7 Arsitektur Transformer (Vaswani et al. 2023).....	16
Gambar 2. 8 Pembelajaran residual (He et al. 2015).....	17
Gambar 2. 10 ControlNet dan blok neural network.....	18
Gambar 2. 11 Arsitektur U-Net Stable Diffusion yang dihubungkan dengan ControlNet	19
Gambar 2. 12 Reparameterisasi dalam LoRA.....	22
Gambar 2. 13 Diagram sistem pengukuran similaritas struktural.....	23
Gambar 2. 14 Diagram sistem pengukuran LPIPS.....	25
Gambar 3. 1 Tahapan Metodologi Penelitian.....	29
Gambar 3. 2 Tahapan image-to-image translation stable diffusion.....	30
Gambar 3. 3 Flowchart training bobot LoRA.....	32
Gambar 3. 4 Contoh citra input Canny.....	34
Gambar 3. 5 Intensitas gradien dan Non-Maximum Supression dalam Canny.....	37
Gambar 3. 6 Double threshold dan hasil akhir citra tepi Canny.....	39
Gambar 3. 7 Encoding citra menggunakan Variational Autoencoder (VAE).....	40
Gambar 3. 8 Flowchart proses di dalam U-Net (Hugging Face n.d.).....	41
Gambar 3. 9 Citra input setelah melalui tahap encoding dan penambahan noise.....	42
Gambar 3. 10 Laten di-decode untuk menghasilkan output beresolusi penuh.....	44
Gambar 3. 11 Perancangan antar muka sistem.....	49
Gambar 4. 1 Contoh proses hashing.....	51
Gambar 4. 2 Contoh gambar dominan background.....	52
Gambar 4. 3 Contoh gambar dominan subjek.....	52
Gambar 4. 4 Hasil average loss pada tiap epoch selama proses pelatihan.....	57
Gambar 4. 5 Tampilan utama aplikasi.....	58
Gambar 4. 6 Proses pembuatan citra tepi Canny.....	59
Gambar 4. 7 Contoh penerapan image-to-image translation.....	61
Gambar 4. 8 Contoh hasil pengujian pada salah satu data uji.....	62
Gambar 4. 9 Perbandingan skor SSIM di seluruh data uji.....	65
Gambar 4. 10 Perbandingan skor LPIPS di seluruh data uji.....	66
Gambar 4. 11 Perbandingan skor pakar di seluruh data uji.....	66

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Animasi merupakan bentuk seni yang umum dijumpai dalam kehidupan sehari-hari dan banyak digunakan di berbagai bidang, seperti periklanan dan perfilman. Saat ini, proses produksi animasi konvensional masih sangat bergantung pada pengerjaan manual, di mana sebagian besar karya visual dibuat berdasarkan pemandangan dunia nyata. Akan tetapi, mengubah adegan dunia nyata ke dalam gaya visual animasi merupakan proses yang kompleks, melelahkan, dan memerlukan keahlian tinggi (J. Chen, Liu, and Chen 2020). Oleh karena itu, dibutuhkan teknik otomatis yang mampu mentransformasi adegan dunia nyata menjadi gambar bergaya animasi. Pendekatan ini tidak hanya mempercepat proses produksi, tetapi juga memungkinkan para seniman untuk lebih fokus pada aspek kreatif lain yang tak kalah penting, seperti pengembangan plot, perancangan karakter, dan lain-lain.

Salah satu solusi untuk permasalahan ini adalah *neural style transfer*. *Style transfer*, dalam artian tradisional, merupakan proses yang melibatkan dua jenis citra masukan, citra konten dan citra gaya (*style*) dan mentransfer karakteristik visual citra gaya ke citra konten (Cai et al. 2023), atau definisi lebih luasnya yaitu mengubah karakteristik visual suatu gambar, tanpa mengubah struktur semantik dari citra konten tersebut. Metode *style transfer* untuk mengubah gambar dunia nyata ke gaya animasi pertama kali diperkenalkan melalui CartoonGAN (Y. Chen, Lai, and Liu 2018), yang dilatih menggunakan *unpaired data*. Model ini menggunakan *loss function* yang mengontrol konten semantik dan ketajaman tepi. Pendekatan berbasis GAN lainnya adalah AnimeGAN (J. Chen, Liu, and Chen 2020), yang meningkatkan kualitas output dengan tiga jenis *loss function*, yaitu *grayscale loss function*, *color reconstruction loss*, dan *grayscale adversarial loss*. Meskipun keberhasilannya untuk membuat gambar, metode berbasis GAN terbukti kalah unggul dibandingkan kualitas gambar yang dihasilkan oleh model difusi (DM) (Dhariwal and Nichol 2021).

Berbeda dengan pendekatan GAN, model difusi (DM) merupakan model probabilistik yang mempelajari distribusi data $p(x)$ dengan melakukan penghilangan *noise* (*denoising*) bertahap pada variabel terdistribusi normal. Dalam *image generation*, model ini bekerja sebagai rangkaian *denoising autoencoder* terbobot imbang $\epsilon_\theta(x_t, t); t = 1, \dots, T$, yang dilatih untuk memprediksi versi lebih bersih dari input x_t , di mana x_t adalah versi *noisy* dari input x . Dalam proses pembuatan gambar, model dipandu oleh *embedding* dari *text prompt* yang diinput oleh pengguna, yang dikenal sebagai proses difusi *text-to-image*. Meskipun mampu untuk menghasilkan gambar dengan kualitas terbaik, DM memakan beban komputasi yang sangat berat. Hal ini dikarenakan proses training dan evaluasi model DM memerlukan perulangan *evaluation function* (dan perhitungan gradien) dalam gambar RGB berdimensi tinggi. Untuk mengatasi masalah di atas, (Rombach et al. 2021a) memperkenalkan *Latent Diffusion Model* (LDM), yang bekerja bukan di ruang piksel berdimensi tinggi (x), melainkan di ruang laten ($\epsilon(x)$) yang lebih ringkas, mengurangi beban komputasi serta waktu inferensi secara signifikan dibandingkan model difusi konvensional.

Meskipun model difusi bukan merupakan model *style transfer* dalam pengertian tradisional, model difusi dapat melakukan tugas layaknya dalam *style transfer*, melalui proses *image-to-image translation*. Namun, terdapat keterbatasan besar dalam model difusi, yaitu distorsi bentuk dan inkonsistensi struktur objek, terutama ketika model kesulitan mempertahankan proporsi atau kontur yang ada pada citra asli (J. Zhang et al. 2024). Hal ini membuat hasil akhir terlihat tidak stabil atau bahkan berbeda jauh dari keluaran yang diinginkan. Untuk mengatasi permasalahan tersebut, dikembangkanlah ControlNet, yang memungkinkan model difusi untuk menambahkan pengondisian spasial (eg. Citra tepi Canny, estimasi pose, peta kedalaman, sketsa, etc.) untuk mempertahankan struktur utama dari citra referensi (L. Zhang, Rao, and Agrawala 2023a). Oleh karena itu, dalam penelitian ini, *Latent Diffusion Model* (LDM), melalui model terlatih *Stable Diffusion*, dikombinasikan dengan ControlNet berbasis peta tepi Canny, digunakan untuk meningkatkan ketahanan struktural konten hasil *image-to-image translation* bergaya animasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah disampaikan, dapat dirumuskan permasalahan yang diangkat dalam penelitian ini adalah mengetahui pengaruh kombinasi Stable Diffusion dengan ControlNet berbasis citra tepi Canny terhadap ketahanan struktur objek hasil *image-to-image translation* dalam pembuatan *background art* bergaya animasi.

1.3 Batasan Masalah

Batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Data gambar uji merupakan citra pemandangan, diambil dari halaman web *Unsplash*.
2. Resolusi maksimal citra keluaran adalah 768x768 piksel.
3. *Prompt* teks dibiarkan kosong dalam pengujian model.
4. *Fine-tuning* stable diffusion menggunakan LoCon (*LoRA for Convolutional Network*).
5. Model dasar yang digunakan adalah model berbasis *stable diffusion v1.5*.
6. Citra data latih diambil dari tangkapan layar film animasi yang dibuat yang dibuat oleh Studio Ghibli.
7. Ketahanan struktur objek dalam citra keluaran diukur berdasarkan metrik LPIPS (Ghazanfari et al. 2023), dan SSIM (R. Zhang et al. 2018a).
8. Kemiripan gaya visual citra keluaran terhadap keseluruhan gaya visual dalam data latih diukur berdasarkan penilaian oleh pakar.
9. Kemiripan gaya visual yang diukur hanya pada aspek pengaplikasian tekstur.

1.4 Tujuan Penelitian

Tujuan penelitian ini adalah mengombinasikan Stable Diffusion dengan ControlNet berbasis citra tepi Canny untuk *image-to-image translation* dalam pembuatan *background art* bergaya animasi.

1.5 Manfaat Penelitian

Luaran dari penelitian ini diharapkan dapat bermanfaat dalam dunia seni visual, khususnya pada bidang film animasi. Dengan menggunakan teknik difusi *image-to-image* untuk membuat *background art*, seniman dapat berfokus ke tugas-tugas lain yang lebih

penting, seperti pembuatan animasi, pengembangan plot, perancangan karakter, dan lain-lain.

1.6 Tahapan Penelitian

Dalam penyusunan laporan tugas akhir ini, digunakan metode penelitian kuantitatif dengan tahapan:

1.6.1 Studi Pustaka

Pada tahap ini dilakukan studi terkait penelitian, jurnal, dan buku yang membahas *image-to-image translation*, *style transfer*, dan *image generation*. Serta mempelajari hal-hal yang berhubungan dengan objek penelitian dan metode yang digunakan pada penelitian.

1.6.2 Pengambilan Data

Data gambar untuk *training* LoRA diperoleh melalui *scraping* dari website *Fanscap*, berupa tangkapan layar film animasi. Setelah itu, data dikurasi secara manual untuk memilih gambar-gambar adegan dominan *background* untuk ditambahkan ke dalam dataset.

1.6.3 Pengolahan Data

Data yang telah dikurasi kemudian dilakukan *downscaling* sesuai dengan resolusi maksimal yang dipilih untuk penelitian ini, yaitu 768x768 piksel.

1.6.4 Pengembangan Sistem

Sistem dibuat dengan menggunakan data yang telah diolah. Model dasar *Stable Diffusion* dilakukan *fine-tuning* menggunakan LoCon (*LoRA for Convolutional Network*), agar model dapat menjalankan tujuan yang spesifik dengan objek penelitian, yaitu membuat gambar *background art* bergaya serupa film animasi.

1.6.5 Evaluasi Sistem

Sistem yang sudah jadi kemudian dievaluasi kualitas visual citra keluarannya, berdasarkan ketahanan struktural kontennya dan kemiripan gaya visual terhadap data latih, diukur menggunakan metrik LPIPS (Ghazanfari et al. 2023), SSIM (R. Zhang et al. 2018a), dan penilaian oleh pakar.

1.7 Sistematika Penulisan

Sistematika penulisan laporan pada penelitian ini adalah sebagai berikut:

1. BAB I PENDAHULUAN

Bab ini berisi mengenai latar belakang masalah, perumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, tahapan penelitian, dan sistematika penulisan dari penelitian kombinasi *Stable Diffusion* dengan *ControlNet* berbasis citra tepi Canny untuk difusi *image-to-image* dalam pembuatan *background art* animasi.

2. BAB II TINJAUAN PUSTAKA

Bab ini berisi mengenai teori-teori yang berhubungan dengan pembahasan dari penelitian yang terkait dengan objek penelitian, seperti pengolahan citra, *computer vision*, *stable diffusion*, *image-to-image translation*, *style transfer*, dan lain-lain. Teori-teori tersebut diambil dari jurnal penelitian, artikel, buku referensi, dan sumber-sumber lainnya.

3. BAB III METODOLOGI PENELITIAN

Bab ini berisi tahapan metodologi penelitian dan pengembangan sistem yang digunakan dalam perancangan sistem.

4. BAB IV HASIL PENGUJIAN DAN PEMBAHASAN

Bab ini menguraikan tentang implementasi dari analisis dan perancangan sistem serta pengujian dan analisis terhadap hasil yang diperoleh pada tampilan implementasi sistem.

5. BAB V PENUTUP

Bab ini berisi kesimpulan, yang merangkum apa saja yang sudah dibahas, dan saran, berisi saran perluasan, pengembangan, dan pengkajian ulang.

BAB II TINJAUAN LITERATUR

2.1 Animasi (Film)

Film animasi adalah sebuah karya sinema di mana gerakan dan penampilan karakter dibuat dengan teknik *frame-by-frame*, yaitu setiap bingkai diproduksi secara individual melalui gambar tangan, komputer, animasi stop-motion, atau teknik sejenis lainnya. Melalui proses ini, objek mati atau bentuk dua dimensi diberikan ilusi kehidupan dan gerak, sehingga menceritakan kisah atau menyampaikan emosi dengan cara yang unik dibanding film live-action biasa (Kehr 2025).

Sejarahnya, animasi telah berkembang dari alat hiburan sederhana seperti zoetrope di abad ke-19 hingga menjadi medium pembuatan film fitur penuh seperti yang dikembangkan oleh studio-studio besar pada abad ke-20 (Kehr 2025). Film animasi memiliki fleksibilitas tinggi dalam gaya visual, teknik produksi, serta target audiens, dari produksi anak-anak hingga karya seni dewasa atau eksperimental, sehingga dapat berfungsi sebagai sarana narasi, ekspresi artistik, maupun hiburan populer.

2.1.1 *Background Art*

Background art adalah elemen visual dalam film animasi yang berfungsi sebagai latar tempat atau lingkungan di mana karakter dan aksi berlangsung. Latar ini bisa berupa pemandangan alam, kota, ruangan dalam bangunan, atau dunia fantasi, yang membantu membangun suasana dan kontekstualisasi cerita. Latar dalam animasi menentukan lokasi, waktu, suasana hati, dan gaya visual di mana karakter akan bergerak (Wicaksono and Arief 2018).

Hubungan *background art* dengan keseluruhan produksi film animasi sangat erat. Bukan hanya sekadar hiasan pasif, latar memainkan peran penting dalam mengatur suasana dan mengembangkan alur cerita dalam produksi animasi (Nelis n.d.). Dalam pembuatan *background* yang baik, terdapat beberapa komponen penting yang harus dipertimbangkan, antara lain:

1. Komposisi

Pengaturan dan penempatan elemen di dalam *background* memainkan peran penting dalam membangun suasana dan membimbing fokus penonton. *Background* yang disusun dengan baik memperkuat *storytelling* dengan mengarahkan perhatian pada elemen-elemen cerita yang penting.

2. *Color palette*

Pemilihan warna sangat penting dalam membangun mood, *tone*, dan atmosfer dari sebuah adegan. *Color palette* yang dipilih dengan cermat dapat membangkitkan emosi, menciptakan kedalaman, dan memperkuat narasi.

3. *Detailing*

Detail menambah *richness* dan tekstur ke dalam latar, menciptakan kesan kedalaman dan realisme. Baik itu pola, tekstur, maupun nuansa halus, perhatian terhadap detail meningkatkan dampak visual animasi secara keseluruhan.

4. Perspektif

Penggunaan perspektif sangat penting dalam menciptakan kesan kedalaman dan dimensi pada *background*. Gambar perspektif yang tepat menambah realisme pada adegan dan menarik penonton ke dalam dunia animasi.

2.2 *Image Processing*

Digital image processing, merupakan sebuah bidang penelitian yang mengacu pada pemrosesan citra digital menggunakan komputer. Tidak terdapat batasan yang jelas antara *image processing*, dengan bidang-bidang terkait seperti *image analysis*, dan *computer vision*. Terkadang, perbedaan dibuat dengan mendefinisikan *image processing* sebagai suatu disiplin ilmu di mana input dan output dari prosesnya adalah citra.

Pemrosesan dalam bidang-bidang terkait ini, terbagi menjadi tiga level, yaitu rendah, menengah, dan tinggi. Pemrosesan tingkat rendah meliputi operasi primitif seperti *preprocessing* citra untuk mengurangi noise, *contrast enhancement*, dan *image sharpening*. Pemrosesan tingkat menengah meliputi berbagai *task* seperti segmentasi and klasifikasi. Terakhir, pemrosesan tingkat tinggi “memahami” citra dari sekumpulan objek yang dikenali, dan melakukan fungsi kognitif yang biasanya terkait dengan penglihatan manusia.

2.3 *Computer Vision*

Computer vision merupakan sebuah bidang ilmu multidisiplin yang mempelajari bagaimana komputer dapat mengambil informasi yang bermakna dari citra, video atau bentuk input visual lainnya. Computer vision mencakup pengaplikasian yang sangat luas. Pengaplikasian computer vision berkisar dari mereplikasi kemampuan visual manusia, hingga membuat kategori kemampuan visual yang benar-benar baru (Gonzalez and Woods 2018).

Pengorganisasian sistem dalam computer vision berbeda tiap aplikasinya. Implementasi spesifik dari computer vision juga bergantung pada apakah fungsionalitasnya telah ditentukan sebelumnya atau apakah beberapa bagiannya dapat dipelajari atau dimodifikasi selama inferensi. Akan tetapi, pada umumnya, terdapat beberapa fungsionalitas yang ada pada computer vision, yaitu:

1. *Image Acquisition*

Dalam proses ini, dilakukan penangkapan informasi visual dan mengubah sinyal analog ke data digital yang kemudian diproses oleh komputer

2. *Image Pre-processing*

Sebelum computer vision dapat diaplikasikan ke gambar untuk mengekstrak informasi darinya, penting untuk memproses data tujuannya adalah untuk memastikan data gambar sesuai dengan aturan atau kebutuhan metode yang akan digunakan.

3. Ekstraksi Fitur

Dalam proses ini, fitur gambar dari berbagai tingkatan kompleksitas diekstrak dari gambar data (eg. Tekstur, bentuk, titik *interest points* seperti sudut, gumpalan atau titik, dan lain-lain).

4. Deteksi / Segmentasi

Pada proses ini, sistem mencoba untuk memahami dan menginterpretasikan gambar berdasarkan fitur-fitur yang telah diekstrak pada tahap sebelumnya. Tujuannya adalah untuk mendeteksi keberadaan objek, mengenali objek tersebut, atau bahkan memahami konteks dari gambar secara keseluruhan.

5. *Image Understanding*

Pada proses ini, sistem tidak hanya mengekstraksi informasi dari gambar tetapi juga menafsirkannya untuk memahami konteks atau makna yang lebih dalam. Ini melibatkan transformasi gambar visual menjadi deskripsi dunia yang masuk akal bagi proses berpikir dan dapat memicu tindakan yang sesuai. *Image Understanding* dapat dilihat sebagai penguraian informasi simbolis dari data gambar menggunakan model yang telah dibangun.

2.4 *Image-to-Image Translation*

Image-to-image Translation (I2I) merupakan salah satu area dalam *computer vision* dan *machine learning* yang bertujuan untuk mentransformasikan citra masukan dari satu domain (domain sumber) ke domain lain (domain target), dengan tetap mempertahankan konten dari domain sumber sekaligus mentransfer gaya dari domain target (Safayani et al. 2025). Beberapa aplikasi dari I2I antara lain adalah *style transfer*, *image restoration*, *semantic segmentation*, *domain adaptation*, aplikasi *virtual reality*, dan lain-lain.

Preservasi konten merupakan aspek yang krusial dalam *Image-to-image translation*, karena untuk memastikan bahwa fitur dan struktur esensial dari citra asli seperti objek, bentuk, serta hubungan antar elemen dalam citra tetap terjaga selama proses transformasi (Safayani et al. 2025). Jika konten mengalami perubahan yang signifikan, citra hasil transformasi dapat menimbulkan kebingungan bagi pengamat atau gagal untuk menyampaikan pesan yang ingin disampaikan, mengurangi tingkat kepercayaan terhadap output model. Preservasi konten sangat esensial dalam berbagai pengaplikasian praktikal. **Gambar 2. 1** merupakan beberapa contoh penerapan *image-to-image translation*.



Gambar 2. 1 Beberapa contoh pengaplikasian *Image-to-image Translation* (Safayani et al. 2025)

2.5 Model Difusi

Model Difusi (*Diffusion Model* / DM) merupakan model-model probabilistik didesain untuk mempelajari distribusi data $p(x)$ dengan secara bertahap menghilangkan *noise* pada sebuah variabel terdistribusi normal, yang sesuai dengan pembelajaran *reverse process* (*denoising*) dari *Markov's Chain* dengan panjang T (Rombach et al. 2021a). Proses penghilangan *noise*, atau *reverse process* bisa juga disebut sebagai *sampling*.

Secara khusus, proses *sampling* dimulai dari x_T dan secara gradual menghasilkan sampel dengan lebih sedikit *noise* $x_{T-1}, x_{T-2}, \dots$ hingga diperoleh hasil akhir x_0 . Setiap *timestep* t merepresentasikan suatu tingkat *noise* tertentu, dan x_t dapat dimaknai sebagai campuran antara sinyal x_0 dengan *noise* ϵ , di mana rasio sinyal terhadap *noise* (*signal-to-noise ratio*) ditentukan oleh *timestep* t .

Model difusi bisa dimaknai sebagai sebuah rangkaian *denoising autoencoder* terbobot imbang $\epsilon_0(x_t, t); t = 1, \dots, T$, yang dilatih untuk memprediksi komponen *noise* dari sampel *noisy* x_t . Objektif dari model difusi didefinisikan sebagai (Rombach et al. 2021a):

$$L_{DM} := \mathbb{E}_{x, \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_0(x_t, t)\|_2^2] \quad (2.1)$$

dengan t di-sampel dari $\{1, \dots, T\}$

2.6 Latent Diffusion Model

Latent Diffusion Model (LDM) adalah model generatif yang diusulkan oleh (Rombach et al. 2021a). Berbeda dengan model difusi konvensional yang beroperasi langsung pada ruang piksel berdimensi tinggi dari citra RGB, LDM beroperasi dalam ruang laten. Dibandingkan ruang piksel, ruang laten lebih cocok untuk model generatif, karena model dapat: (i) berfokus pada representasi semantik penting dari data, serta (ii) melakukan proses pelatihan dan inferensi dalam ruang berdimensi rendah dengan efisiensi komputasi yang tinggi. Secara formal, LDM didefinisikan dengan fungsi objektif sebagai (Rombach et al. 2021a):

$$L_{LDM} := \mathbb{E}_{\epsilon(x), \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_0(z_t, t)\|_2^2] \quad (2.2)$$

Mirip dengan model-model generatif lainnya, model difusi secara dasar mampu memodelkan distribusi pengondisian $p(z|y)$. Implementasinya dilakukan melalui *denoising autoencoder* $\epsilon_0(z_t, t, y)$, di mana proses *denoising* dikendalikan oleh variabel masukan y , yang dapat berupa teks, peta semantik, maupun citra pada tugas-tugas *image-to-image translation* lainnya.

Model difusi dapat diubah menjadi *conditional image generator* yang fleksibel dengan diperkuatnya *backbone* U-Net yang mendasarinya menggunakan mekanisme *cross-attention*. Mekanisme ini memungkinkan model untuk secara efektif mempelajari representasi berbasis perhatian dari berbagai modalitas input. Untuk melakukan *pre-processing* masukan y dari berbagai modalitas (seperti *prompt* teks), digunakan *domain specific encoder* τ_0 yang memproyeksikan y ke dalam representasi perantara $\tau_0(y) \in \mathbb{R}^{M \times d_r}$, yang kemudian dipetakan ke lapisan-lapisan perantara dalam U-Net melalui lapisan *cross-attention* mengimplementasikan (Rombach et al. 2021a):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V, \quad (2.3)$$

dengan

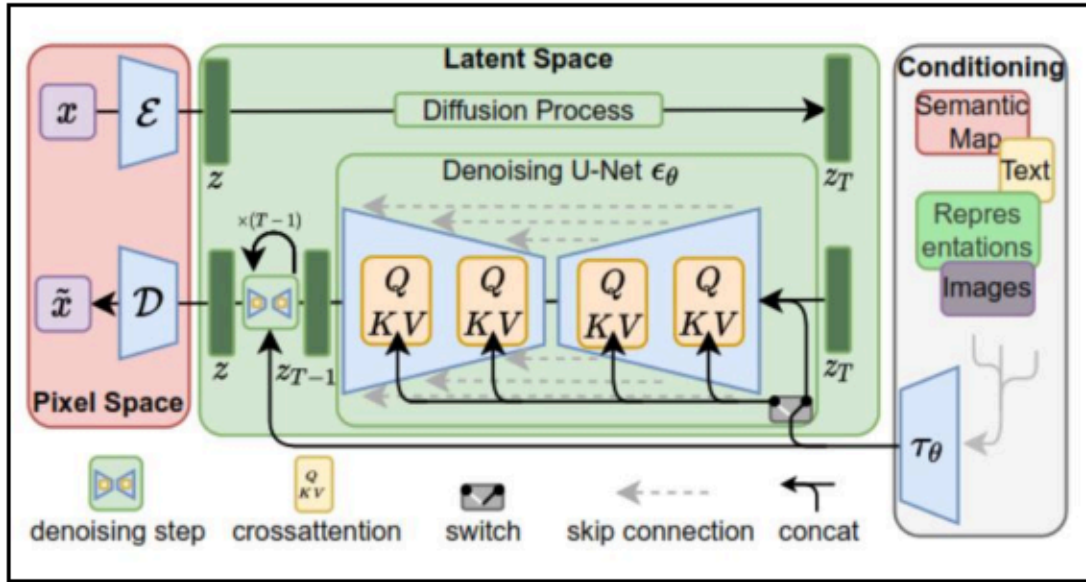
$$Q = W_Q^{(i)} \cdot \varphi_i(z_t), \quad K = W_K^{(i)} \cdot \tau_0(y), \quad V = W_V^{(i)} \cdot \tau_0(y)$$

Di sini, $\varphi_i(z_t)$ merepresentasikan keluaran perantara (yang sudah di-*flatten*) dari U-Net yang mengimplementasikan ϵ_θ dan W_V, W_Q , dan W_K adalah matriks proyeksi yang dapat dipelajari. Penggambaran visual bisa dilihat di **Gambar 2. 2 Struktur Latent Diffusion Model (LDM)**

(Rombach et al. 2021a). Berdasarkan mekanisme pengondisian di atas, *conditional LDM* dapat didefinisikan sebagai (Rombach et al. 2021a):

$$L_{LDM} := E_{\epsilon(x), y, \epsilon \sim N(0,1), t} \left[\left\| \epsilon - \epsilon_\theta(z_t, t, \tau_0(y)) \right\|_2^2 \right] \quad (2.4)$$

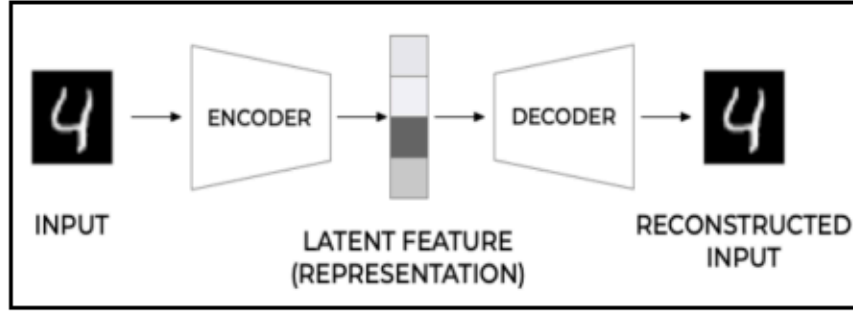
Persamaan di atas menunjukkan bahwa variabel pengondisi y , setelah diproyeksikan ke ruang representasi perantara $\tau_0(y)$, memandu proses *denoising* sehingga pembentukan citra tidak hanya konsisten dengan distribusi data laten, tetapi juga sesuai dengan informasi kontekstual yang diberikan melalui pengondisian.



Gambar 2. 2 Struktur *Latent Diffusion Model* (LDM)
(Rombach et al. 2021a)

2.7 Autoencoder

Autoencoder pertama kali dikenalkan oleh Rumelhart, Hinton, dan Williams pada tahun 1986, dengan tujuan untuk mempelajari rekonstruksi input observasi x_i dengan *error* sekecil mungkin. *Autoencoder* terbagi menjadi tiga komponen utama, yaitu: *encoder*, fitur representasi laten, dan *decoder*. *Encoder* dan *decoder* merupakan fungsi, sementara fitur representasi laten merupakan tensor berisi bilangan real (Michelucci 2022).



Gambar 2. 3 Struktur dasar autoencoder
(Vaswani et al. 2023)

Dalam arsitektur umumnya, *encoder* dan *decoder* merupakan neural network. Encoder bisa dirumuskan sebagai:

$$h_i = g(x_i) \quad (2.5)$$

di mana $h_i \in \mathbb{R}^q$ (fitur representasi laten) merupakan keluaran dari encoder saat dievaluasi dalam input x_i . Decoder kemudian bisa dirumuskan sebagai:

$$\hat{x}_i = f(h_i) = f(g(x_i)) \quad (2.6)$$

di mana $\hat{x}_i \in \mathbb{R}^n$ merupakan keluaran dari decoder. Melatih autoencoder berarti menemukan fungsi $g(\cdot)$ dan $f(\cdot)$ yang memenuhi:

$$\arg \min < \left[\Delta \left(x_i, f(g(x_i)) \right) \right] > \quad (2.7)$$

di mana Δ menyimbolkan pengukuran seberapa besar perbedaan antara masukan dan keluaran dari autoencoder, dan $<\cdot>$ menyimbolkan rata-rata dari seluruh data.

Autoencoder dapat diaplikasikan ke dalam berbagai macam tugas, seperti dimensionality reduction, classification, dan anomaly detection. Namun belakangan ini, autoencoder juga diaplikasikan dalam model generatif.

2.8 Variational Autoencoder

Variational Autoencoder (VAE) adalah model generatif berbasis probabilistik yang menggabungkan konsep autoencoder dan variational inference untuk mempelajari representasi laten dari data dan menghasilkan data baru yang mirip dengan data asli (Kingma and Welling 2019). Secara konseptual, VAE berupaya memodelkan proses generatif data, yaitu memahami bagaimana data observasi x dihasilkan dari variabel laten z . Dengan mempelajari hubungan antara data dan variabel laten ini, VAE dapat menangkap struktur tersembunyi dalam data dan menghasilkan sampel baru yang memiliki karakteristik serupa dengan data asli.

Struktur dasar dari VAE terdiri atas dua komponen utama, yaitu *encoder* dan *decoder*. *Encoder*, atau disebut juga *recognition model*, berfungsi untuk memetakan data ke dalam ruang laten dengan menghasilkan parameter distribusi, umumnya berupa mean (μ) atau simpangan baku (σ), dari distribusi *gaussian*. Hasil pemetaan ini berupa distribusi probabilistik $q_\phi(z|x)$, yang merepresentasikan kemungkinan nilai variabel laten z untuk setiap data x . Sementara itu, decoder, atau *generative model*, bertugas melakukan proses

sebaliknya, yakni merekonstruksi kembali data x dari representasi laten z melalui distribusi bersyarat $p_\theta(x|z)$.

Tujuan utama pelatihan VAE adalah untuk memaksimalkan kemungkinan (*likelihood*) data yang dihasilkan oleh model, atau secara matematis, memperkirakan distribusi $p_\theta(x)$. Namun, perhitungan langsung terhadap nilai tersebut tidak dimungkinkan karena integral yang melibatkan distribusi laten z bersifat *intractable*, artinya tidak memiliki solusi analitik yang dapat dihitung secara efisien. Untuk mengatasi ini, VAE menggunakan pendekatan *variational inference*, yakni menggantikan posterior sebenarnya $p_\theta(z|x)$ yang tidak dapat dihitung dengan approximate posterior $q_\phi(z|x)$ yang lebih mudah diestimasi.

Dari pendekatan ini, dikembangkan fungsi objektid yang dikenal sebagai *Evidence Lower Bound* (ELBO) yang berperan sebagai batas bawah dari log-likelihood data. ELBO dirumuskan sebagai (Vaswani et al. 2023):

$$\mathcal{L}(\theta, \phi, x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(z|x)] - D_{KL}(q_\phi(z|x) || p_\theta(z)) \quad (2.8)$$

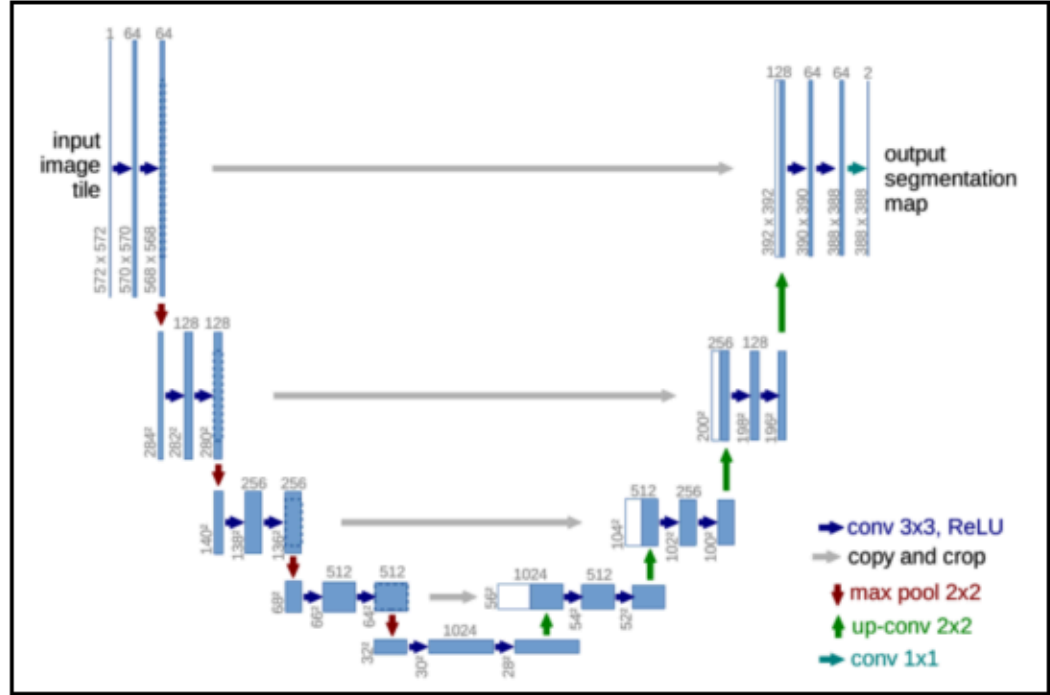
Persamaan ini memiliki dua komponen, pertama yaitu *expected reconstruction term* $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(z|x)]$, mengukur seberapa baik model dapat merekonstruksi data berdasarkan representasi laten. Komponen kedua yaitu *regularization term* berupa divergensi Kullback-Leibler $D_{KL}(q_\phi(z|x) || p_\theta(z))$, berfungsi untuk menjaga berfungsi untuk menjaga agar distribusi laten yang dipelajari tidak menyimpang terlalu jauh dari distribusi prior yang biasanya diasumsikan sebagai Gaussian standar $\mathcal{N}(\mathbf{0}, I)$.

2.9 Arsitektur U-Net

U-Net merupakan arsitektur jaringan yang dikembangkan berdasarkan *Fully Convolutional Network* (FCN) yang diperkenalkan oleh (Ronneberger, Fischer, and Brox 2015). Arsitektur ini awalnya dirancang untuk segmentasi dan lokalisasi pada citra biomedis.

Struktur U-Net terdiri atas dua komponen utama, yaitu *contracting path* (*downsampling*) dan *expanding path* (*upsampling*). *Contracting path* berfungsi menangkap konteks global citra dan mengikuti arsitektur konvolusional pada umumnya. Setiap langkah pada *contracting path* terdiri atas dua lapisan konvolusi 3x3, yang masing-masing diikuti oleh fungsi aktivasi *rectified linear unit* (ReLU) dan *max pooling* 2x2 dengan stride 2 untuk melakukan *downsampling*.

Sebaliknya, *expansive path* memungkinkan lokalisasi yang lebih presisi. Setiap langkah pada *expansive path* terdiri atas lapisan konvolusi 2x2 (*up-convolution*), penggabungan (*skip connection*) dengan peta fitur dari *contracting path* yang telah mengalami *cropping*, dan dua lapisan konvolusi 3x3, yang masing-masing diikuti oleh fungsi aktivasi ReLU. Pada lapisan terakhir, sebuah konvolusi 1x1 digunakan untuk memetakan setiap vektor fitur ke jumlah kelas yang diinginkan.



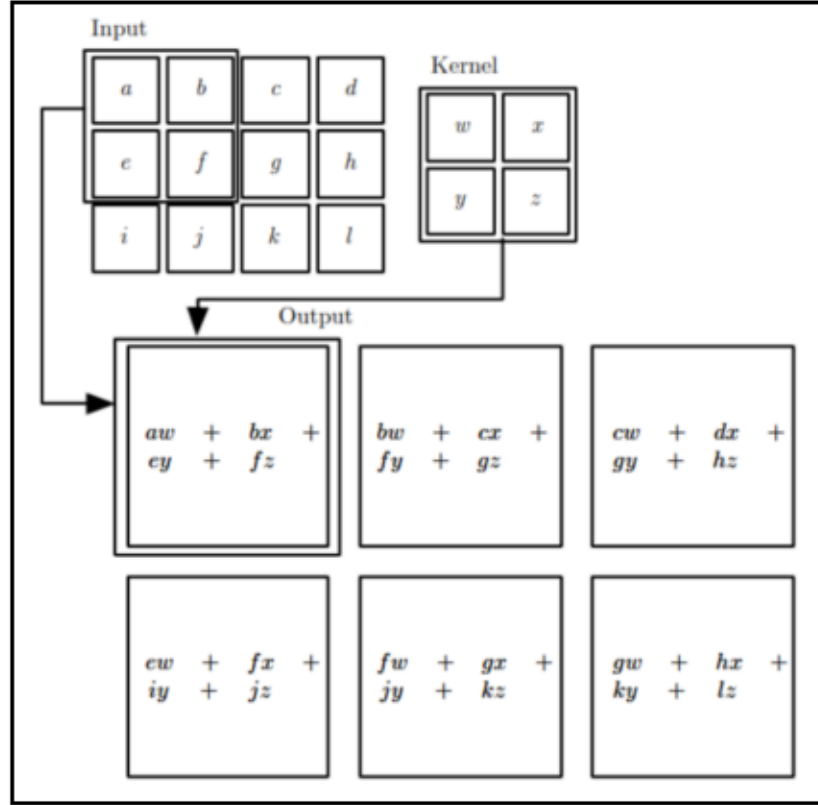
Gambar 2. 4 Arsitektur U-Net
(Ronneberger, Fischer, and Brox 2015)

2.10 Convolutional Layer

Lapisan konvolusi atau *convolutional layer* merupakan komponen utama dalam *Convolutional Neural Network* (CNN). Convolutional layer bertugas untuk mengekstrak fitur-fitur penting dari data melalui proses konvolusi (Ronneberger, Fischer, and Brox 2015). Dalam konvolusi, input biasanya berupa matriks dari data, dan kernel berupa matriks dari parameter, yang telah diadaptasi. Kernel kemudian digeser (dikonvolusi) melalui seluruh area input data untuk menghasilkan peta fitur atau *feature map*. Sebagai contoh, Apabila inputnya adalah sebuah gambar 2-dimensi I , operasi konvolusi dapat didefinisikan sebagai:

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(m, n) \cdot K(i - m, j - n) \quad (2.9)$$

Di mana $S_{(i,j)}$ adalah nilai pada posisi (i, j) dalam matriks feature map, I adalah citra input, K adalah filter konvolusi atau kernel, m dan n adalah posisi piksel pada citra input, dan i dan j adalah posisi pada output (*feature map*).



Gambar 2. 5 Contoh konvolusi dua dimensi dengan kernel 2 x 2
(Lecun, Bengio, and Hinton 2015)

2.11 Layer Normalization

Layer normalization (LN) adalah teknik normalisasi yang diperkenalkan untuk mempercepat dan menstabilkan proses pelatihan berbagai arsitektur *neural network*. *Layer normalization*, seperti *batch normalization*, bertujuan untuk mengurangi masalah *covariate shift* yang tidak diinginkan (Ba, Kiros, and Hinton 2016). Kondisi ini terjadi ketika perubahan pada keluaran dari sebuah lapisan menyebabkan perubahan yang berkorelasi kuat pada input lapisan berikutnya, sehingga memperlambat konvergensi.

Secara matematis, untuk lapisan l dengan H *hidden units*, mean (μ_l) dan simpangan baku (σ_l) dihitung dari *summed up* input (a_i^l) dalam lapisan tersebut:

$$\mu_l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad (2.10)$$

$$\sigma_l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu_l)^2} \quad (2.11)$$

Perbedaan utama *layer normalization* dan *batch normalization* terletak pada perhitungan statistik (mean dan varians-nya). BN menghitung *mean* dan *varians* dari input yang dijumlahkan ke neuron tertentu di seluruh kasus pelatihan dalam satu *mini-batch*. Sementara LN menghitung mean dan varians dari semua input yang dijumlahkan ke neuron dalam satu lapisan pada satu kasus pelatihan tunggal. Dalam LN, seluruh *hidden unit* pada

satu lapisan menggunakan nilai *mean* dan *varians* yang sama, namun setiap kasus pelatihan memiliki persyaratan normalisasi yang berbeda.

Tidak seperti *batch normalization*, *layer normalization* tidak memberlakukan batasan apa pun pada ukuran *mini-batch*. Ini membuat *layer normalization* lebih cocok untuk kasus-kasus di mana ukuran batch kecil atau bervariasi, seperti pada model *Recurrent Neural Networks* (RNNs) atau *Transformers*.

2.12 Group Normalization

Group normalization (GN) merupakan teknik normalisasi layer yang membagi *channel* ke beberapa grup dan melakukan komputasi normalisasi di dalam setiap grupnya (Wu and He 2018). GN menghitung mean dan varians untuk normalisasi di dalam setiap grup *channel*. Pendekatan ini memastikan komputasi normalisasi independen dari ukuran *batch*.

Dalam konteks fitur visual 2D, fitur diindeks menggunakan vektor 4D $i = (i_N, i_C, i_H, i_W)$, di mana N adalah poros *batch*, C adalah poros *channel*, dan (H, W) adalah poros spasial (tinggi dan lebar). GN, seperti BN, *layer normalization*, dan *instance normalization*, melakukan perhitungan normalisasi dasar dengan:

$$\hat{x}_i = \frac{1}{\sigma_i} (x_i - \mu_i) \quad y_i = \gamma \hat{x}_i + \beta \quad (2.12)$$

di mana μ_i dan σ_i adalah mean dan simpangan baku, dan γ serta β adalah parameter *scale* dan *shift* yang dapat dilatih per-*channel*. μ_i dan σ_i dihitung menggunakan:

$$\mu_i = \frac{1}{m} \sum_{k \in S_i} x_k \quad \sigma_i = \sqrt{\frac{1}{m} \sum_{k \in S_i} (x_k - \mu_i)^2 + \epsilon}, \quad (2.13)$$

dengan ϵ sebagai konstanta kecil. S_i adalah kumpulan (set) piksel di mana mean dan simpangan baku dihitung, dan m adalah ukuran set tersebut. Perbedaan mendasar antara berbagai jenis normalisasi fitur adalah bagaimana S_i -nya didefinisikan.

GN menghitung mean dan simpangan baku dalam set S_i yang didefinisikan sebagai:

$$S_i = \left\{ k \mid k_N = i_N, \left\lfloor \frac{k_C}{G} \right\rfloor = \left\lfloor \frac{i_C}{G} \right\rfloor \right\} \quad (2.14)$$

Di sini G adalah jumlah grup, yang merupakan *hyperparameter* yang sudah ditentukan sebelumnya (default $G = 32$). C/G adalah jumlah *channel* per grupnya. $\lfloor \cdot \rfloor$ adalah operasi *floor*, dan " $\left\lfloor \frac{k_C}{G} \right\rfloor = \left\lfloor \frac{i_C}{G} \right\rfloor$ " berarti indeks i dan k berada di dalam kelompok *channel* yang sama. GN menghitung mean dan simpangan baku sepanjang sumbu spasial (H, W) dan sepanjang sekelompok C/G *channel*.

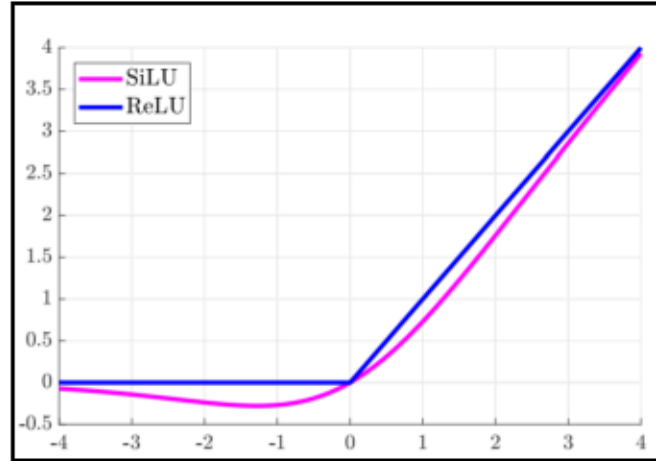
2.13 Sigmoid Linear Unit (SiLU)

Sigmoid Linear Unit (SiLU) merupakan salah satu fungsi aktivasi yang digunakan dalam *neural network*. SiLU merupakan hasil penggabungan dari fungsi sigmoid dan *Rectified Linear Unit* (ReLU) (Elfwing, Uchibe, and Doya 2017). Fungsi SiLU dirumuskan sebagai:

$$\text{SiLU}(x) = x\sigma(x) \quad (2.15)$$

Di mana x adalah fungsi aktivasi dan $\sigma(x)$ adalah fungsi sigmoid.

Fungsi SiLU bersifat *smooth*, non-linear, yang memungkinkan *neural network* untuk mempelajari hubungan yang kompleks dalam data. Dibandingkan dengan ReLU, untuk nilai x positif, SiLU berperilaku mirip dengan ReLU karena $\sigma(x)$ mendekati 1. Untuk nilai x yang negatif, SiLU tidak sepenuhnya nol seperti ReLU, akan tetapi mendekati nol karena $\sigma(x)$ mendekati 0 (Elfwing, Uchibe, and Doya 2017).



Gambar 2. 6 Grafik perbandingan SiLU dengan ReLU (Elfwing, Uchibe, and Doya 2017)

2.14 Transformer Block

Transformer Block adalah komponen dasar dalam arsitektur Transformer, yang diperkenalkan oleh (Vaswani et al. 2023). Transformer terdiri dari dua tahap utama, yaitu *Multi-Head Self-Attention* (MHSA) dan *Feed-Forward Network* (FFN), yang masing-masing diikuti oleh *Add & Layer Normalization* untuk menjaga stabilitas training.

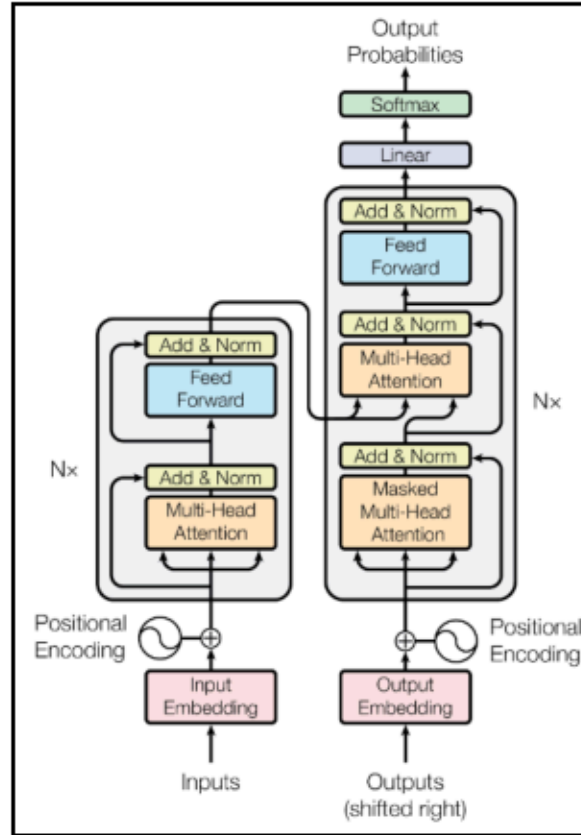
Tahap pertama, Multi-Head Self-Attention, memungkinkan model memahami hubungan antar elemen dalam input dengan menghitung skor perhatian (attention scores) antara setiap token. Proses ini dilakukan dengan menggunakan *query* (Q), *key* (K), dan *value* (V) (Vaswani et al. 2023), yang dihitung sebagai:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V \quad (2.16)$$

sehingga informasi dari seluruh sekuen dapat dipertimbangkan tanpa bergantung pada urutan tertentu. Setelah itu, hasilnya dijumlahkan dengan input awal melalui residual connection, lalu dinormalisasi menggunakan *layer normalization*.

Tahap kedua adalah Feed-Forward Network (FFN), yang merupakan lapisan MLP (multilayer perceptron) sederhana dengan aktivasi non-linear seperti ReLU atau GELU. FFN bertujuan untuk menangkap pola kompleks dalam data setelah perhatian dihitung. Seperti pada tahap sebelumnya, output FFN kembali dijumlahkan dengan inputnya melalui residual connection dan dinormalisasi lagi dengan Layer Normalization.

Transformer Block sangat efisien karena tidak bergantung pada urutan sekuens, berbeda dari model RNN yang bersifat sekuensial. Mekanisme *self-attention* yang digunakan juga memungkinkan model menangkap hubungan panjang dalam data, menjadikannya sangat fleksibel untuk berbagai tugas seperti Natural Language Processing (NLP) dalam GPT dan BERT, serta pemrosesan gambar dalam Vision Transformer (ViT) dan Stable Diffusion.



Gambar 2. 7 Arsitektur Transformer
(Vaswani et al. 2023)

2.15 ResNet Block

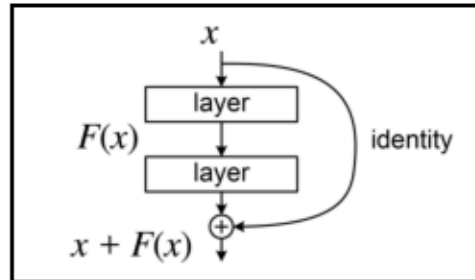
ResNet block, atau disebut juga sebagai *residual block* merupakan komponen utama dari ResNet (*Residual Network*), yang diperkenalkan oleh (He et al. 2015). Residual block merupakan blok skip-connection yang mempelajari fungsi residual dengan mengacu pada lapisan input, alih-alih mempelajari fungsi yang tidak direferensikan.

Dalam sebuah model neural network multi-lapisan, Terdapat subjaringan dengan sejumlah lapisan bertumpuk. Fungsi yang dilakukan oleh subjaringan ini disimbolkan sebagai $H(x)$, di mana x adalah input dari subjaringan ini. Pembelajaran residual mengubah parameter subjaringan ini dan membiarkan lapisan parameter merepresentasikan “fungsi residual” $F(X) = H(X) - x$. Output y dari subjaringan ini kemudian direpresentasikan sebagai:

$$y = F(X) + x \quad (2.17)$$

Operasi “ $+x$ ” diimplementasikan melalui “*skip-connection*” yang melakukan pemetaan identitas (*identity mapping*) untuk menghubungkan input dari subjaringan dengan

outputnya. Fungsi $F(X)$ sering direpresentasikan dengan perkalian matriks yang diselingi dengan fungsi aktivasi dan operasi normalisasi (eg. *Batch normalization*, *layer normalization*). Secara penuh subjaringan ini disebut sebagai *residual block*.



Gambar 2. 8 Pembelajaran residual
(He et al. 2015)

2.16 Stable Diffusion

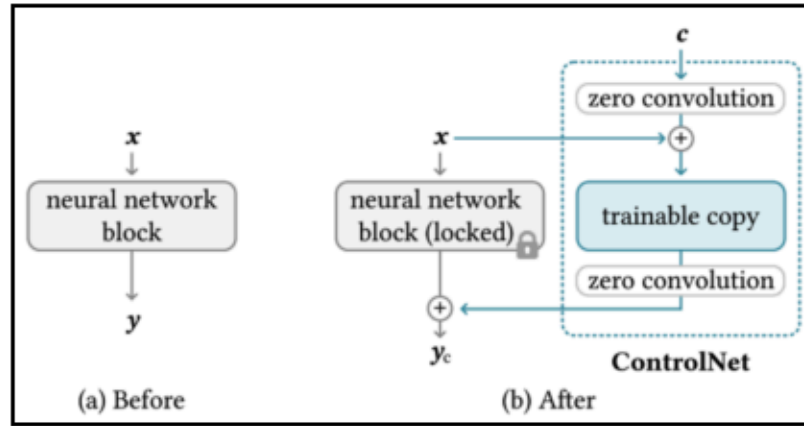
Stable diffusion merupakan sebuah model difusi terlatih berskala besar yang dikembangkan oleh Computer Vision and Learning research group at Ludwig Maximilian University of Munich (CompVis) dengan bantuan dari StabilityAI dan RunwayML (Rombach et al. 2021b). Mirip seperti Google Imagen, model ini menggunakan frozen CLIP ViT-L/14 *text encoder* untuk mengondisi model dengan *text prompt*. Dengan U-Net 860M dan *text encoder* 123M, model ini relatif ringan dan bisa berjalan dapat dijalankan pada GPU dengan VRAM 10GB.

Terdapat banyak model terlatih berbasis *stable diffusion* yang telah dikembangkan, namun hanya Stable Diffusion v1 yang merupakan model resmi yang dikembangkan oleh CompVis dan runwayML. Stable Diffusion v1 merujuk pada konfigurasi arsitektur spesifik yang memanfaatkan *autoencoder* dengan faktor downsampling 8, U-Net berukuran 860M, dan *text encoder* CLIP ViT-L/14. Model ini mula-mula dilatih menggunakan citra berukuran 256×256 piksel, kemudian disempurnakan untuk menghasilkan citra beresolusi 512×512 piksel. Saat ini terdapat 5 *checkpoint* dari *stable diffusion*, yaitu:

1. v1-1: 237k step pada resolusi 256x256 di dataset LAION2B-en. 194k step pada resolusi 512x512 di LAION-high-resolution.
2. v1-2: Dilanjutkan dari v1-1 - 515k *step* pada resolusi 512x512 pada laion-"aesthetics v2 5+".
3. v1-3: Dilanjutkan dari v1-2 - 195k step pada resolusi 512x512 pada "laion-aesthetics v2 5+" dan 10% dropping pada pengondisian teks untuk meningkatkan *classifier-free sampling*.
4. v1-4: Dilanjutkan dari v1-2 - 225k *step* pada resolusi 512x512 pada "laion-aesthetics v2 5+" dan 10% dropping pada pengondisian teks untuk meningkatkan *classifier-free sampling*.
5. v1-5: Dilanjutkan dari v1-2 - 595k *step* pada resolusi 512x512 pada "laion-aesthetics v2 5+" dan 10% dropping pada pengondisian teks untuk meningkatkan *classifier-free sampling*.

2.17 ControlNet

ControlNet adalah teknik dalam model difusi yang memungkinkan pengendalian lebih presisi terhadap hasil citra yang dihasilkan oleh *Stable Diffusion* dengan menggunakan pengondisian spasial tambahan, seperti sketsa, peta kedalaman, peta tepi Canny, atau estimasi pose. ControlNet memperluas kemampuan DM dengan menambahkan jaringan pengontrol yang dilatih untuk mempertahankan struktur input, sekaligus memungkinkan model tetap menghasilkan gambar dengan detail serta gaya visual yang diinginkan. ControlNet diperkenalkan oleh (L. Zhang, Rao, and Agrawala 2023b).



Gambar 2.9 ControlNet dan blok *neural network*
(L. Zhang, Rao, and Agrawala 2023a)

ControlNet menambahkan kondisi eksternal pada blok dalam *neural network*. Misal $\mathcal{F}(\cdot; \Theta)$ merupakan sebuah *neural block* terlatih, dengan parameter Θ , yang mentransformasikan sebuah input peta fitur (*feature map*) x , ke peta fitur output y didefinisikan sebagai (L. Zhang, Rao, and Agrawala 2023b):

$$y = \mathcal{F}(x; \Theta) \quad (2.18)$$

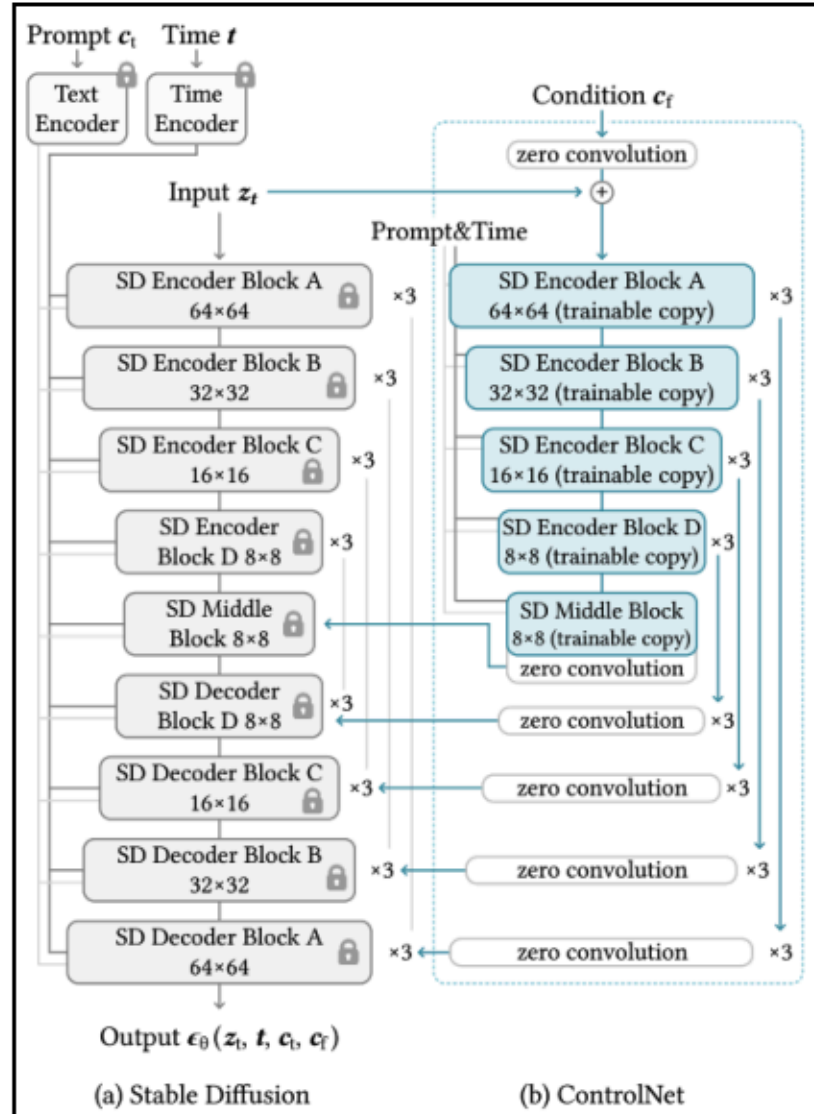
Di sini, x dan y umumnya berupa peta fitur dua dimensi, *i.e.*, $x \in \mathbb{R}^{h \times w \times c}$ dengan $\{h, w, c\}$ masing-masing menyatakan tinggi, lebar, dan jumlah *channel* pada peta. (lihat gambar Gambar 2.9a).

Untuk mengintegrasikan ControlNet ke dalam sebuah *neural block* terlatih, parameter Θ pada blok asli dikunci (dibekukan), kemudian blok tersebut disalin menjadi sebuah salinan yang dapat dilatih (*trainable copy*) dengan parameter Θ_c (lihat gambar Gambar 2.9b). Blok salinan ini menerima vektor pengondisian eksternal c sebagai input.

Selanjutnya, Blok salinan dihubungkan dengan blok asli yang terkunci melalui *zero convolutional layer*, dilambangkan dengan $Z(\cdot; \cdot)$. Untuk membangun ControlNet, digunakan dua instans *zero convolution* dengan parameter masing-masing Θ_{z1} dan Θ_{z2} . Dengan demikian, ControlNet didefinisikan sebagai:

$$y_c = y = \mathcal{F}(x; \Theta) + Z(\mathcal{F}(x + Z(c; \Theta_{z1}); \Theta_c); \Theta_{z2}) \quad (2.19)$$

di mana y_c adalah output dari blok ControlNet.



Gambar 2. 10 Arsitektur U-Net Stable Diffusion yang dihubungkan dengan ControlNet (L. Zhang, Rao, and Agrawala 2023a)

2.17.1 ControlNet untuk Difusi *Text-to-Image*

Stable Diffusion pada dasarnya adalah U-Net dengan *encoder*, *middle block*, dan *skip-connected decoder*. Bagian *Encoder* dan *decoder* masing-masing terdiri dari 12 blok, sehingga keseluruhan model mencakup 25 blok, termasuk *middle block*. Dari 25 blok tersebut, 8 di antaranya merupakan lapisan konvolusi untuk proses *down-sampling* maupun *up-sampling*, sementara 17 blok lainnya merupakan blok utama yang masing-masing terdiri atas 4 lapisan *ResNet* dan 2 *Vision Transformer* (ViTs). Setiap ViT mengimplementasikan mekanisme *cross-attention* dan *self-attention* (L. Zhang, Rao, and Agrawala 2023b).

Struktur ControlNet diaplikasikan pada setiap level *encoder* dari arsitektur U-Net (lihat Gambar 2. 10 (b)). Secara khusus, ControlNet digunakan untuk membuat salinan *trainable* dari 12 blok *encoder* serta 1 *middle block* pada stable diffusion. Kedua belas blok *encoder* terbagi ke dalam empat tingkat resolusi ($64 \times 64, 32 \times 32, 16 \times 16, 8 \times 8$) dengan

masing-masing blok direplikasi sebanyak tiga kali. Output dari blok-blok ini kemudian ditambahkan pada 12 *skip-connection* dan 1 *middle block* dari U-Net.

Untuk mengintegrasikan ControlNet ke Stable Diffusion, input berupa citra pengondisian (eg. Peta tepi, pose, peta kedalaman, sketsa, etc.) berukuran 512×512 terlebih dahulu diproyeksikan ke dalam vektor ruang fitur berukuran 64×64 yang sesuai dengan dimensi internal stable diffusion. Secara khusus, digunakan sebuah jaringan kecil $\varepsilon(\cdot)$ dari keempat lapisan konvolusi dengan kernel 4×4 dan stride 2×2 (diaktivasi dengan ReLU, menggunakan *channel* 12, 32, 64, 128 masing-masing, diinisiasi dengan bobot Gaussian dan dilatih bersamaan dengan model penuh) untuk memetakan sebuah pengondisian ruang-gambar c_i ke vektor pengondisian ruang fitur c_f , melalui:

$$c_f = \varepsilon(c_i) \quad (2.20)$$

Selanjutnya, vektor pengondisian c_f ini diberikan sebagai input ke dalam ControlNet.

2.18 Canny

Canny edge detection adalah algoritma deteksi tepi yang dikembangkan oleh (Canny 1986) dan banyak digunakan dalam computer vision serta pengolahan citra digital. Algoritma ini bertujuan untuk mendeteksi tepi dengan presisi tinggi, memisahkan objek dari latar belakang, serta mengurangi noise yang dapat mengganggu hasil deteksi. Proses deteksi tepi Canny dibagi menjadi 5 tahapan yaitu:

1. Gaussian Filter

Untuk menghaluskan gambar, kernel *Gaussian Filter* dikonvolusikan dengan citra. Langkah ini akan menghaluskan gambar untuk mengurangi efek noise yang jelas pada detektor tepi. Persamaan untuk kernel *Gaussian* berukuran $(2k + 1) \times (2k + 1)$ melalui:

$$H_{ij} = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(i - (k + 1))^2 + (j - (k + 1))^2}{2\sigma^2}\right); 1 \leq i, j \leq (2k + 1) \quad (21)$$

2. Menemukan intensitas gradien dari gambar

Canny menggunakan empat filter untuk mendeteksi tepi horizontal, vertikal, dan diagonal dalam gambar. Operator deteksi tepi (seperti Roberts, Prewitt, atau Sobel) mengembalikan nilai untuk turunan pertama dalam arah horizontal G_x dan arah vertikal G_y . Dari sini gradien tepi dan arah dapat ditentukan (OpenCV n.d.):

$$G = \sqrt{G_x^2 + G_y^2}$$

$$\Theta = \arctan2(G_y, G_x),$$

Di mana G dapat dihitung menggunakan *hypot function*, dan *atan2* adalah fungsi *arctangent* dengan dua argumen. Sudut arah tepi dibulatkan ke salah satu dari empat sudut yang mewakili vertikal, horizontal, dan dua diagonal ($0^\circ, 45^\circ, 90^\circ$ dan 135°).

3. Non-Maximum Supression

Langkah ini bertujuan untuk mengurangi piksel penggabungan duplikat di sepanjang tepian agar tidak rata. Untuk setiap piksel, temukan dua tetangga dalam arah gradien positif dan negatif, dengan asumsi bahwa setiap tetangga menempati sudut $\pi/4$, dan 0 adalah arah

lurus ke kanan. Jika besaran piksel saat ini lebih besar dari besaran tetangga, tidak ada yang berubah, jika tidak, besaran piksel saat ini ditetapkan ke nol.

4. *Double Thresholding*

Besaran gradien dibandingkan dengan dua nilai *threshold* yang ditentukan, yang pertama lebih rendah dari yang kedua. Gradien yang lebih kecil dari nilai *low threshold* ditekan, gradien yang lebih tinggi dari nilai *high threshold* ditandai sebagai yang kuat dan piksel yang sesuai disertakan dalam peta tepi akhir. Semua gradien lainnya ditandai sebagai yang lemah dan piksel yang sesuai dengan gradien ini dipertimbangkan pada langkah berikutnya.

5. Pelacakan tepi menggunakan Hysteresis

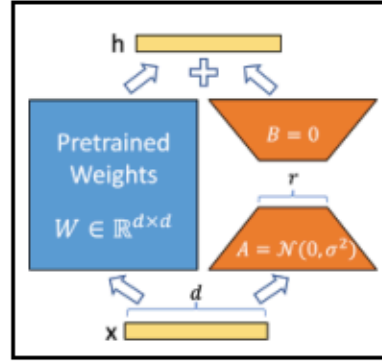
Karena piksel tepi lemah yang disebabkan oleh tepi sejati akan dihubungkan ke piksel tepi kuat, piksel W dengan gradien lemah ditandai sebagai tepi dan disertakan dalam peta tepi akhir jika dan hanya jika piksel tersebut terlibat dalam komponen terhubung yang sama dengan beberapa piksel S dengan gradien kuat. Dengan kata lain, harus ada rantai piksel lemah tetangga yang menghubungkan W dan S (tetangganya berjarak 8 piksel di sekitar piksel yang dipertimbangkan).

2.19 LoRA dan LoCon

Low-Rank Adaptation of Large Models (LoRA) merupakan teknik yang diperkenalkan oleh (Hu et al. 2021) untuk melakukan *fine-tuning* pada model berskala besar. Awalnya dikembangkan untuk model berbasis transformer (seperti GPT), metode ini kemudian diadaptasi secara luas, termasuk pada model generatif seperti *Stable Diffusion*. LoRA bekerja dengan cara membekukan bobot model terlatih, kemudian menambahkan matriks dekomposisi berperingkat rendah (*rank decomposition matrices*) yang dapat dilatih pada lapisan-lapisan terpilih. Ide dasarnya adalah bahwa matriks bobot terlatih $W_0 \in \mathbb{R}^{p \times q}$ memiliki “dimensi intrinsik” yang rendah, sehingga pembaruan bobot $\Delta W \in \mathbb{R}^{p \times q}$ dapat dipisahkan menjadi dua matriks berdimensi lebih kecil, $A \in \mathbb{R}^{p \times r}$ dan $B \in \mathbb{R}^{r \times q}$, di mana p, q adalah dimensi dari parameter model asli, r adalah dimensi dari matriks *low-rank*, dan $r \ll \min(d, k)$. Selama proses *fine-tuning*, bobot asli tetap dibekukan, sementara hanya matriks A dan B yang diperbarui. Sebagai hasilnya, *forward process* $h = W_0x + \Delta Wx$ diperbarui menjadi (Hu et al. 2021):

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (2.22)$$

di mana h adalah peta fitur output.



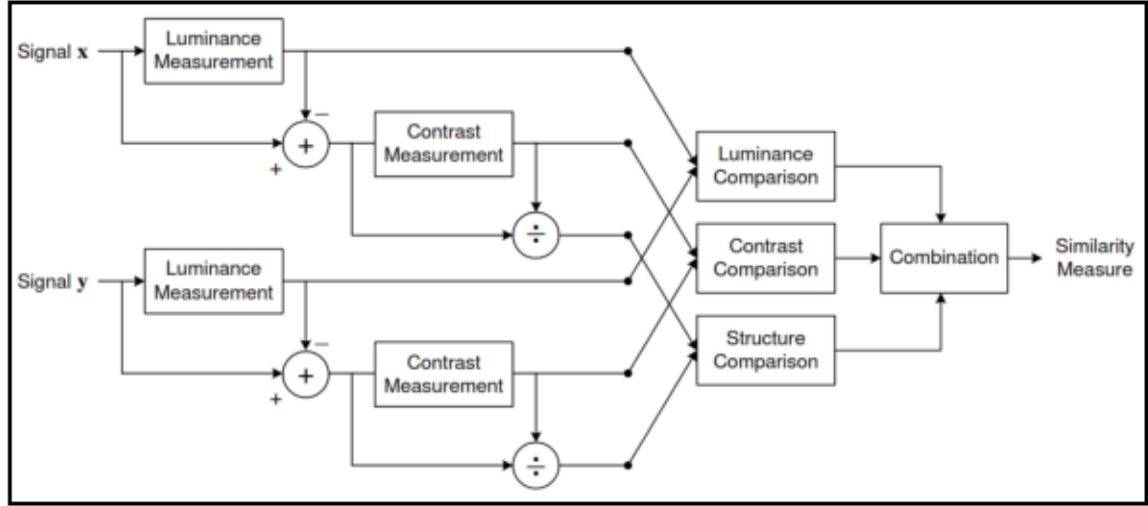
Gambar 2. 11 Reparameterisasi dalam LoRA
(Hu et al. 2021)

Pada awalnya, penggunaan LoRA difokuskan pada lapisan *attention* dalam *large language model*, atau *cross-attention* pada Stable Diffusion. (Yeh et al. 2023) kemudian memperkenalkan *LoRA for Convolutional Networks* (LoCon), yang memperluas LoRA ke lapisan konvolusi. Untuk sebuah lapisan konvolusi dengan pembaruan bobot $\Delta W \in \mathbb{R}^{c_{in} \times c_{out} \times k \times k}$, di mana k adalah ukuran kernel, serta c_{in} dan c_{out} masing-masing menyatakan jumlah *channel* input dan output. Pembaruan bobot dapat diaproksimasi dengan dua lapisan konvolusi berurutan; sebuah konvolusi berukuran $k \times k$ dengan bobot $A \in \mathbb{R}^{r \times c_{in} \times k \times k}$ dan, diikuti konvolusi 1×1 dengan bobot $B \in \mathbb{R}^{c_{out} \times r \times 1 \times 1}$, dengan $r \ll \min(c_{in}, c_{out})$.

2.20 Metode Evaluasi

2.20.1 Structural Similarity Index (SSIM)

Structural Similarity Index (SSIM), dikembangkan oleh (Wang et al. 2004), merupakan sebuah metode objektif untuk menilai kualitas citra berdasarkan kesamaan struktural antara citra referensi dan citra hasil distorsi. Berbeda dari pendekatan tradisional seperti *Mean Squared Error* (MSE) dan *Peak Signal-to-Noise Ratio* (PSNR) yang mengukur perbedaan nilai piksel secara langsung, SSIM menilai kesamaan dari segi struktur spasial. Pendekatan ini berlandaskan pada asumsi bahwa sistem penglihatan manusia (*Human Visual System* / HVS) sangat adaptif dalam mengekstraksi informasi struktural dari suatu pemandangan.



Gambar 2. 12 Diagram sistem pengukuran similaritas struktural (SSIM)
(R. Zhang et al. 2018b)

SSIM membandingkan dua citra non-negatif x (referensi) dan y (terdistorsi) dengan memisahkan analisis menjadi tiga bagian, yaitu luminansi, kontras, dan struktur. Pertama, luminansi dihitung sebagai rata-rata nilai intensitas piksel (Wang et al. 2004):

$$\mu_x = \frac{1}{N} \sum_{i=1}^N x_i \quad (2.23)$$

Fungsi perbandingan luminansi $l(x,y)$ merupakan fungsi dari μ_x dan μ_y . Fungsi perbandingan luminansi $l(x,y)$ dirumuskan sebagai (Wang et al. 2004):

$$l(x,y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1} \quad (2.24)$$

dengan

$$C_1 = (K_1L)^2,$$

di mana L adalah rentang dinamis nilai piksel (misal 255 untuk citra *grayscale* 8-bit), dan K_1 adalah konstanta kecil.

Kedua, rata-rata intensitas dihilangkan dari sinyal ($x - \mu_x$). Simpangan baku digunakan sebagai perkiraan dari sinyal kontras, dirumuskan sebagai (Wang et al. 2004):

$$\sigma_x = \left(\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu_x)^2 \right)^{1/2} \quad (2.25)$$

Perbandingan kontras $c(x,y)$ merupakan perbandingan antara σ_x dan σ_y . Fungsi perbandingan kontras $c(x,y)$ kemudian dirumuskan sebagai (Wang et al. 2004):

$$c(x,y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2} \quad (2.26)$$

Ketiga, sinyal kemudian dinormalisasi dengan simpangan bakunya sendiri. Perbandingan struktur $s(x,y)$ dilakukan pada sinyal-sinyal yang sudah dinormalisasi $(x - \mu_x)/\sigma_x$ dan $(y - \mu_y)/\sigma_y$. Fungsi perbandingan struktur $s(x,y)$ kemudian dirumuskan sebagai (Wang et al. 2004):

$$s(x, y) = \frac{\sigma_{xy} + C_3}{\sigma_x \sigma_y + C_3}, \quad \sigma_{xy} = \frac{1}{N-1} \sum_{i=1}^N (x_i - \mu_x)(y - \mu_y) \quad (2.27)$$

Terakhir, ketiga fungsi perbandingan tersebut dikombinasikan, menghasilkan indeks *structural similarity* (SSIM) antara sinyal x dan y :

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (2.28)$$

Untuk meniru karakteristik persepsi manusia yang fokus pada area lokal (foreation), SSIM dihitung secara lokal menggunakan jendela geser (*sliding window*) berukuran 11×11 berbobot *Gaussian* $w = \{w_i | i = 1, 2, \dots, N\}$ dengan simpangan baku 1.5, dan *unit sum* $\sum_{i=1}^N w_i x_i$. Estimasi lokal μ_x , σ_x , dan σ_{xy} kemudian diubah masing-masing menjadi:

$$\mu_x = \frac{1}{N} \sum_{i=1}^N w_i x_i$$

$$\sigma_x = \left(\frac{1}{N-1} \sum_{i=1}^N w_i (x_i - \mu_x)^2 \right)^{1/2}, \quad \sigma_{xy} = \sum_{i=1}^N w_i (x_i - \mu_x)(y - \mu_y)$$

Setiap posisi jendela menghasilkan nilai SSIM lokal, yang kemudian dirata-rata untuk memperoleh mean SSIM (MSSIM) untuk mengevaluasi kualitas citra secara keseluruhan (Wang et al. 2004):

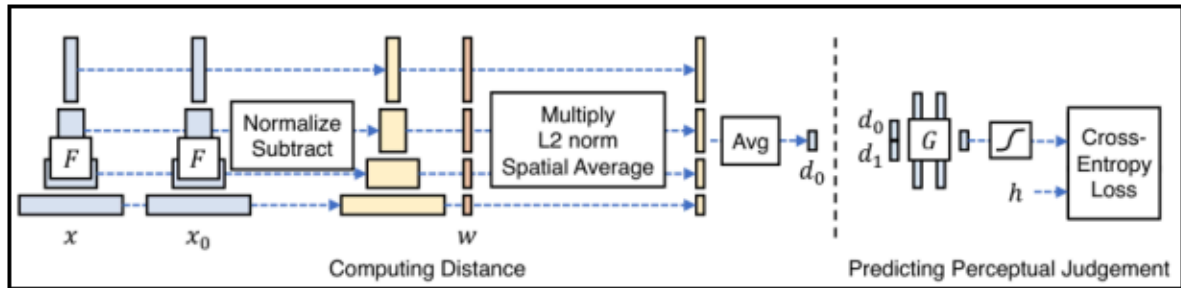
$$MSSIM(X, Y) = \frac{1}{M} \sum_{j=1}^M SSIM(x_j, y_j) \quad (2.29)$$

di mana X dan Y masing-masing merupakan citra referensi dan citra terdistorsi, x_j dan y_j merupakan konten dari citra pada jendela ke- j , dan M adalah jumlah jendela dalam citra.

2.20.2 Learning Perceptual Image Patch Similarity (LPIPS)

Learning Perceptual Image Patch Similarity (LPIPS), dikembangkan oleh (R. Zhang et al. 2018b), merupakan sebuah pendekatan evaluasi kualitas citra yang dirancang untuk menilai kesamaan perseptual antara dua gambar berdasarkan persepsi manusia. LPIPS muncul sebagai solusi keterbatasan metrik tradisional seperti *Peak Signal-to-Noise Ratio* (PSNR) dan *Structural Similarity Index* (SSIM), yang hanya membandingkan nilai intensitas piksel secara langsung tanpa mempertimbangkan struktur atau makna visual yang ditangkap oleh sistem penglihatan manusia.

Berbeda dari metrik berbasis piksel, LPIPS mengukur jarak perseptual di ruang fitur hasil ekstraksi dari jaringan saraf konvolusional (CNN). Dua citra, yakni citra referensi dan citra hasil distorsi, dimasukkan ke dalam jaringan CNN terlatih (eg. VGG, AlexNet, atau SqueezeNet). Setiap lapisan konvolusi menghasilkan peta fitur yang mewakili representasi visual pada tingkat tertentu.



Gambar 2. 13 Diagram sistem pengukuran LPIPS
(R. Zhang et al. 2018b)

Selanjutnya, aktivasi pada setiap lapisan dinormalisasi agar tidak dipengaruhi oleh perbedaan skala intensitas antar *channel*. Setelah itu, tiap *channel* diberi bobot tertentu menggunakan pembobot w_1 , yang berfungsi menyesuaikan sensitivitas tiap fitur terhadap persepsi manusia. Bobot ini dapat ditentukan secara tetap dari jaringan pra-latih atau dipelajari melalui proses kalibrasi menggunakan data penilaian manusia.

Setelah proses normalisasi dan pembobotan, LPIPS menghitung jarak antara dua peta fitur dari citra referensi dan citra hasil distorsi menggunakan jarak Euclidean (L2 distance). Perhitungan dilakukan pada setiap posisi spasial dan kemudian dirata-ratakan di seluruh dimensi spasial dan lapisan jaringan. Hasil akhirnya berupa satu nilai skalar yang menunjukkan tingkat perbedaan perseptual antara kedua citra. Semakin kecil nilai LPIPS, semakin mirip kedua citra tersebut menurut persepsi manusia.

2.21 Diffusers

Diffusers adalah *library open-source* dari *Hugging Face* yang menyediakan model difusi terlatih untuk berbagai kebutuhan AI generatif, seperti pembuatan gambar, audio, dan bahkan struktur molekul 3D. Pustaka ini dirancang untuk memudahkan pengguna dalam melakukan inferensi sederhana maupun melatih model difusi (Hugging Face n. d.).

Diffusers menawarkan tiga komponen inti: *pipeline*, *scheduler*, dan *model*. *Pipeline* menyediakan cara yang mudah dan terintegrasi untuk melakukan inferensi dengan berbagai model. *Scheduler* memungkinkan pengaturan kecepatan difusi dan kualitas keluaran yang diinginkan. *Model* memungkinkan akses terhadap model-model terlatih yang dapat digunakan sebagai blok untuk membangun sistem difusi *end-to-end*.

2.22 Studi Pustaka (*State-of-the-Art*)

Tabel 2. 1 *State-of-the-art*

No	Penelitian	Metode	Hasil
1.	High Resolution Image Synthesis with Latent Diffusion Models (Rombach et al. 2021a)	VAE, LDM	Penelitian ini menghasilkan <i>Latent Diffusion Model</i> (LDM). LDM membagi proses pembentukan citra menjadi dua tahap: tahap kompresi perseptual menggunakan <i>autoencoder</i> , dan tahap difusi yang beroperasi dalam ruang laten. Penelitian ini juga mengenalkan mekanisme <i>cross-attention</i> ke dalam arsitektur U-Net, yang memungkinkan pengondisian yang fleksibel dalam proses difusi.
2.	Adding Conditional Control to Text-to-Image Diffusion Models (L. Zhang, Rao, and Agrawala 2023b)	ControlNet, Difusi	ControlNet adalah arsitektur yang dirancang untuk menambahkan kontrol pengondisian spasial (eg. Peta kedalaman, peta tepi, segmentasi, pose, sketsa, etc.) dalam model difusi terlatih.
3.	LoRA: Low-Rank Adaptation of Large Language Models (Hu et al. 2021)	LoRA	<i>Low-Rank Adaptation</i> (LoRA) bekerja dengan membekukan bobot model terlatih untuk menyuntikkan matriks dekomposisi rank rendah (<i>low-rank decomposition matrix</i>) yang dapat dilatih ke dalam lapisan arsitektur Transformer. LoRA terbukti berkinerja setara atau bahkan lebih baik daripada fine-tuning penuh dalam kualitas model pada RoBERTa, DeBERTa, GPT-2, dan GPT-3, meskipun hanya menggunakan sebagian kecil parameter yang dapat dilatih.
4.	Navigating Text-to-Image Customization: From LyCORIS Fine-Tuning to Model Evaluation (Yeh et al. 2023)	LoRA, LoCon, Difusi	Penelitian ini memperkenalkan <i>library</i> LyCoRIS (<i>LoRA beyond Conventional methods, Other Rank adaptation Implementations for Stable diffusion</i>), yang mencakup berbagai metodologi adaptasi yang dikembangkan dari LoRA, seperti LoCon (<i>LoRA for Convolutional Network</i>), LoHA (<i>LoRA with Hadamard Product</i>), dan LoKr (<i>LoRA with Kronecker Product</i>).
5.	SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations (Meng et al. 2022)	Difusi	SDEdit (<i>Stochastic Differential Editing</i>) adalah metode untuk pembentukan dan pengeditan citra terpandu memanfaatkan <i>Stochastic Differential Equations</i> (SDE). Mekanisme kerjanya melibatkan penambahan noise <i>Gaussian</i> pada citra panduan input, diikuti dengan proses penghilangan noise secara iteratif melalui <i>reverse SDE</i> .
6.	CartoonGAN: Generative Adversarial Networks for	GAN	CartoonGAN, metode berbasis GAN untuk mentransformasi foto pemandangan dunia nyata menjadi gambar bergaya kartun.

Tabel 2. 1 *State-of-the-art* (lanjutan)

No	Penelitian	Metode	Hasil
	Photo Cartoonization (Y. Chen, Lai, and Liu 2018)		Penelitian ini juga mengenalkan dua <i>loss</i> baru: <i>semantic content loss</i> dan <i>edge-promoting adversarial loss</i> . CartoonGAN dilatih menggunakan gambar-gambar yang diambil dari film karya Makoto Shinkai, Mamoru Hosoda, Hayao Miyazaki, dan Satoshi Kon.
7.	AnimeGAN: A Novel Lightweight GAN for Photo Animation (J. Chen, Liu, and Chen 2020)	GAN	AnimeGAN memiliki fungsionalitas yang sama dengan penelitian sebelumnya (CartoonGAN). Penelitian ini memperkenalkan tiga <i>loss function</i> baru: <i>grayscale style loss</i> , <i>color reconstruction loss</i> , dan <i>grayscale adversarial loss</i> .
8.	Diffusion Models Beat GANs on Image Synthesis (Dhariwal and Nichol 2021)	Difusi	Penelitian ini menunjukkan bahwa model difusi (<i>Diffusion Model / DM</i>) dapat mencapai kualitas sampel citra yang lebih unggul dibandingkan model generatif <i>state-of-the-art</i> , khususnya model-model berbasis GAN. Ini dapat dicapai dengan: 1) perubahan arsitektur, meningkatkan jumlah <i>attention heads</i> , menerapkan <i>attention</i> pada resolusi multi-skala, serta mengadopsi <i>residual block</i> dari BigGAN dan <i>Adaptive Group Normalization</i> (AdaGN). 2) Pengenalan <i>Classifier Guidance</i> , yang memungkinkan proses difusi untuk dipandu ke arah label kelas tertentu.
9.	Denoising Diffusion Probabilistic Models (Ho, Jain, and Abbeel 2020)	Difusi	Model generatif berbasis difusi yang menghasilkan citra berkualitas tinggi melalui proses pembalikan dari penambahan <i>noise Gaussian</i> secara bertahap. DDPM mencapai skor FID 3.17 dan Inception Score 9.46 pada dataset CIFAR-10, melampaui banyak model generatif lain seperti GAN dan VAE. Meski efisien dalam pelatihan, model ini relatif lambat dalam proses sampling karena memerlukan ratusan hingga ribuan langkah untuk menghasilkan satu citra.
10.	Denoising Diffusion Implicit Models (Song, Meng, and Ermon 2020)	Difusi	Pengembangan dari DDPM yang berfokus pada efisiensi dalam proses <i>sampling</i> . DDIM memperkenalkan pendekatan <i>non-Markovian</i> dalam proses difusi yang tetap mempertahankan fungsi pelatihan DDPM, tetapi memungkinkan proses generatif bersifat deterministik dan jauh lebih cepat. DDIM dapat menghasilkan citra berkualitas tinggi dalam hanya 10–100 langkah, dengan hanya sedikit penurunan kualitas.

Penelitian terdahulu yang serupa dengan objek penelitian, yaitu style transfer atau translasi citra *image-to-image* bergaya animasi, hanya terdapat dua yaitu penelitian yang dilakukan (J. Chen, Liu, and Chen 2020) dan (Y. Chen, Lai, and Liu 2018). Namun, seperti dijelaskan pada (Dhariwal and Nichol 2021), metode berbasis GAN kalah dibandingkan metode berbasis difusi, terutama dalam menghasilkan gambar yang realistis.

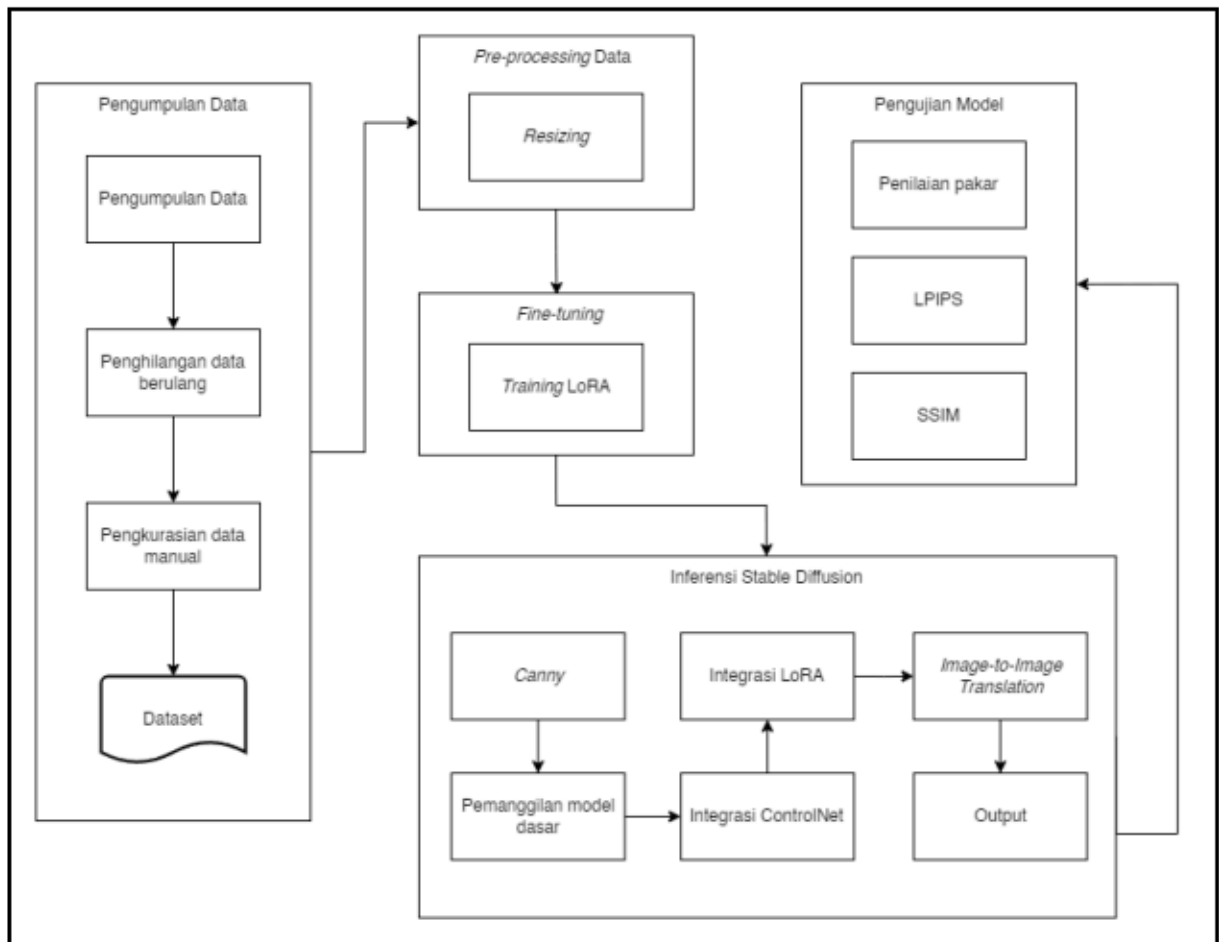
Dari penelitian-penelitian terdahulu di atas, terlihat bahwa, setelah tahun 2020, metode berbasis difusi mendominasi dalam tugas *image generation* dibandingkan metode berbasis GAN. Namun, seperti yang dijelaskan dalam (J. Zhang et al. 2024) dan (L. Zhang, Rao, and Agrawala 2023a), model berbasis difusi masih kesulitan dalam menjaga konsistensi struktur objek, juga mempertahankan proporsi atau kontur untuk konsep-konsep yang kurang dikenal, terutama pada tugas translasi *image-to-image*.

Oleh karena itu, berdasarkan gap penelitian di atas, yang membedakan penelitian ini dengan penelitian terdahulu adalah penelitian ini mengombinasikan Stable Diffusion (LDM) (Rombach et al. 2021a) dengan ControlNet (L. Zhang, Rao, and Agrawala 2023b) berbasis citra tepi Canny untuk membantu model mempertahankan struktur konten citra referensi dalam citra output. *Base model* Stable Diffusion juga akan dilakukan *fine-tuning* dengan *LoRA for Convolutional Network* (LoCon) untuk menyesuaikan model dengan gambar bergaya animasi.

BAB III METODOLOGI PENELITIAN

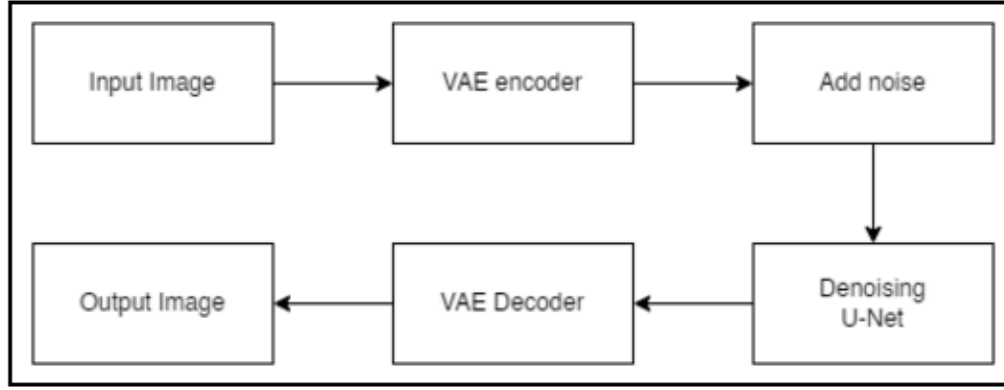
3.1 Metodologi Penelitian

Pada bab ini dibahas mengenai metodologi penelitian dan metodologi pengembangan sistem yang digunakan sebagai acuan dari penelitian kombinasi *stable diffusion* dengan controlnet berbasis citra tepi canny untuk translasi *image-to-image* dalam pembuatan *background art* bergaya animasi. Tahapan metodologi penelitian dijabarkan dalam **Gambar 3. 1**.



Gambar 3. 1 Tahapan Metodologi Penelitian

Dan adapun tahapan *input*, *process*, dan *output* dari *image-to-image translation* stable diffusion dijabarkan dalam **Gambar 3. 2**, dan dijelaskan lebih detil dalam **subbab 3.5**.



Gambar 3. 2 Tahapan *image-to-image translation* stable diffusion

3.2 Pengumpulan Data

Tahap pertama dalam penelitian ini yaitu pengumpulan data. Dalam penelitian ini, data primer digunakan untuk *fine-tuning* stable diffusion dan pengujian model. Data untuk *fine-tuning* dikumpulkan melalui *scrapping* dari situs *fanscap.com*. Fancaps merupakan sebuah situs yang menyediakan gambar tangkapan layar yang diambil dari berbagai film, serial TV, dan *anime* populer. Dari situs ini diambil gambar-gambar tangkapan layar dari film-film produksi *Studio Ghibli*, yang kemudian akan diproses dan digunakan untuk *fine-tuning* model.

Setelah dikumpulkan, data kemudian disaring untuk menghilangkan data berulang. Dalam proses ini, data gambar dalam satu film dibagi menjadi beberapa “adegan” berdasarkan nilai *perceptual similarity*-nya, kemudian gambar-gambar berulang dalam setiap adegan dihapus.

Langkah pertama yaitu *hashing*, atau lebih spesifiknya menemukan nilai *average hash* untuk setiap citra. Proses ini diawali dengan pengecilan ukuran citra menjadi berukuran 8×8 , atau 64 piksel. Citra 64-piksel tersebut kemudian diubah ke dalam bentuk *grayscale*, dan nilai rata-rata piksel-nya dihitung. Nilai rata-rata tersebut kemudian digunakan untuk mengubah citra menjadi bentuk *binary*, apabila sebuah piksel bernilai dibawah rata-rata, maka nilainya berubah menjadi 0, dan apabila piksel bernilai diatas rata-rata maka nilainya berubah menjadi 1. Proses ini menghasilkan output berupa *hash* 64 bit untuk setiap citranya.

Setelah didapat *hash* untuk setiap citra, dilakukan pembagian data menjadi beberapa adegan. Misal terdapat dua gambar x_1 dan x_2 dengan nilai *hash* H_1 dan H_2 , *Hamming distance* dari kedua *hash* ini dihitung menggunakan persamaan:

$$D(H_1, H_2) = \sum_{i=1}^{64} |H_{1i} - H_{2i}| \quad (3.30)$$

Hamming distance yang sudah dihitung kemudian dinormalisasi melalui pembagian dengan total jumlah bit-nya, dalam kasus ini, 64 bit. *Hamming distance* ternormalisasi ini kemudian digunakan untuk menghitung kemiripan antara kedua *hash*:

$$Similarity = 1 - \frac{D(H_1, H_2)}{64} \quad (3.31)$$

Nilai kemiripan kemudian dibandingkan dengan batas kemiripan (*similarity threshold*) dengan nilai konstan. Apabila nilai kemiripan antara dua gambar, x_1 dan x_2 , berada di atas batas, maka kedua gambar tersebut dianggap sebagai adegan yang sama, apabila nilai kemiripan di bawah batas, maka kedua gambar dianggap sebagai adegan **berbeda**, dan x_2 akan membentuk kelompok adegan baru. Terakhir, setelah data dikelompokkan menjadi sejumlah adegan, data berulang dalam setiap adegannya dihapus, menyisakan hanya data pertama dalam setiap adegannya. Contoh, apabila dalam satu adegan terdapat tiga gambar x_1, x_2 , dan x_3 , maka x_2 dan x_3 akan dihapus, menyisakan x_1 .

Setelah disaring, data kemudian dikurasi secara manual. Gambar yang akan dipilih sebagai data latih adalah gambar dominan *background*, atau lebih spesifiknya gambar yang memiliki elemen *background* yang lebih dominan daripada subjeknya.

3.3 Pre-Processing Data

Pre-processing merupakan tahap untuk menyiapkan data sebelum digunakan untuk pelatihan model. Dalam penelitian ini, data melalui proses *downscaling* terlebih dahulu untuk menyesuaikan dengan aproksimasi resolusi optimal dari *stable diffusion* v1.5, yaitu dari 512×512 hingga 1024×1024 piksel. Proses *downscaling* data latih ini dilakukan melalui mekanisme *bucketing*.

Bucketing merupakan sebuah sistem pengelompokan data latih yang diimplementasikan oleh *library fine-tuning* Kohya-SS. Sistem ini mengelompokkan citra berdasarkan rasio aspeknya ke dalam "*bucket*", memungkinkan pelatihan pada *batch* di mana semua citra, dalam tiap *batch*-nya, memiliki dimensi yang identik. Dengan pendekatan ini, gambar dapat di-*resize* dengan mempertahankan proporsi aslinya.

Dalam *bucketing*, terdapat empat parameter, yaitu resolusi maksimal (**max reso**), resolusi minimal *bucket* (**min size**) & maksimal *bucket* (**max size**), dan interval (*step*).

Untuk membuat daftar *bucket*, hal pertama yang harus dilakukan adalah menghitung luas maksimal, **max area** = **max width** $\times$ **max height**. Kemudian dilakukan proses *loop* dimulai dari **width** = **min size** hingga **max size** dengan interval = *step*, untuk menemukan nilai *height* dan menambah daftar resolusi *bucket*. Untuk setiap iterasinya, dihitung: **height** = **min**(**max reso**, **floor**(**max area** / **width**)).

Proses iterasi tersebut kemudian menghasilkan daftar yang berisi pasangan resolusi pengelompokan *bucket* yang memiliki luas piksel lebih kecil atau setara dengan **max reso**. Kemudian daftar tersebut diurutkan dari nilai **width** terkecil hingga terbesar.

Untuk mengelompokkan suatu citra ke dalam *bucket*, pertama dihitung rasio aspek, **aspect ratio** = **width** $\div$ **height**, dan luasnya, **area** = **width** $\times$ **height**. Apabila luas resolusi citra lebih besar dari resolusi maksimal, maka citra harus melalui proses *downscaling*.

Proses *downscaling* dimulai dari menghitung lebar dan tinggi gambar yang baru dengan tetap mempertahankan rasio aspeknya. Untuk lebar yang baru, dihitung melalui:

$$\text{resized width} = \sqrt{\text{max area} \times \text{aspect ratio}}$$

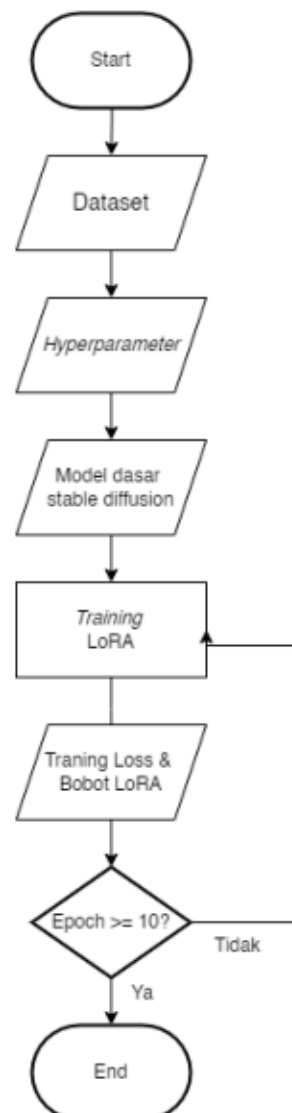
Setelah ditemukan nilai lebar yang baru, nilai tersebut selanjutnya digunakan untuk menemukan nilai tinggi yang baru:

$$resized\ height = \frac{max\ area}{resized\ width}$$

Setelah dihitung nilai lebar dan tinggi yang baru, kedua nilai tersebut kemudian dibulatkan ke kelompok *bucket* terdekat. Terakhir, gambar dilakukan *downscale* sesuai dengan ukuran *bucket* tersebut.

3.4 Fine-Tuning Stable Diffusion

Agar model dapat mengerjakan tugasnya sesuai dengan objek penelitian, dilakukan *fine-tuning* terhadap model dasar stable diffusion melalui pelatihan LoRA. Pelatihan LoRA dilakukan dengan menjalankan *script* dari *library* Kohya-SS, dengan menyertakan *hyperparameter* dan dataset yang sudah diproses sebelumnya. **Gambar 3.3** di bawah adalah *flowchart* proses pelatihan bobot LoRA.



Gambar 3.3 *Flowchart training bobot LoRA*

Tahap pertama yaitu load dataset yang berisi file citra tangkapan layar film *anime*. Setelah itu, dilakukan penentuan *hyperparameter* yang akan digunakan dalam pelatihan, mencakup:

1. *Resolution*: Menentukan resolusi maksimal citra dalam pelatihan. Apabila ukuran citra melebihi resolusi maksimal, maka akan dilakukan *downscaling* secara otomatis melalui mekanisme *bucketing*.
2. *Number of repeat*: Menentukan berapa kali tiap citra dalam dataset digunakan dalam satu *epoch*.
3. *Number of epoch*: Menentukan jumlah *epoch* dalam pelatihan.
4. *U-Net learning rate*: Menentukan *learning rate* arsitektur U-Net dari model dasar.
5. *Text encoder learning rate*: Menentukan *learning rate* untuk *text encoder*.
6. *Network dimension* (LoRA rank): Menentukan dimensi matriks LoRA. Semakin tinggi nilainya, semakin kompleks pola yang dapat ditangkap dalam pelatihan.
7. *Network alpha*: Parameter *scaling* untuk bobot LoRA. Digunakan mengontrol seberapa besar pengaruh adaptasi LoRA terhadap model asli. Nilai yang lebih tinggi membuat adaptasi lebih signifikan. Biasanya, *network alpha* diatur sebanding atau lebih rendah dari rank.
8. *Convolutional dimension*: Menentukan dimensi matriks LoRA untuk lapisan konvolusional. *Convolutional dimension* mirip dengan *network dimension*, tetapi khusus untuk lapisan konvolusional. Semakin tinggi nilainya, semakin tinggi kapasitas untuk menangkap fitur spasial dan tekstur.
9. *Convolutional alpha*: Parameter *scaling* untuk adaptasi LoRA pada lapisan konvolusional. Sama seperti *network alpha*, tetapi khusus untuk lapisan konvolusional.

Setelah semua *hyperparameter* diinput, model dasar stable diffusion v1.5 diinput untuk proses pelatihan bobot LoRA. Proses pelatihan akan berjalan sebanyak *epoch* yang telah ditentukan. Dalam setiap *epoch*-nya, bobot yang dihasilkan dari proses pelatihan disimpan dalam file berformat (.safetensors).

Proses pelatihan bobot LoRA dilakukan serupa seperti pelatihan penuh *stable diffusion*, yaitu dengan proses *forward diffusion*. Dalam proses ini, diambil *timestep* secara acak untuk setiap citra, dan ditambahkan *noise* ke laten sesuai dengan besarnya *noise* untuk *timestep* tersebut. U-Net kemudian dilatih untuk memprediksi komponen *noise* dalam laten dan dihitung nilai lossnya. Nilai loss kemudian dirata-rata setiap *epoch*-nya menghasilkan nilai *average loss*.

3.5 Inferensi Stable Diffusion

Pada penelitian ini, proses *image-to-image translation* dilakukan menggunakan model terlatih *stable diffusion*, yang terdiri dari *variational autoencoder* (VAE) untuk memproyeksikan citra ke ruang laten, dan U-Net sebagai inti proses *denoising*. Untuk menjaga konsistensi struktur visual pada hasil output, model ini dikombinasikan dengan ControlNet berbasis citra tepi Canny sebagai pengondisian spasial.

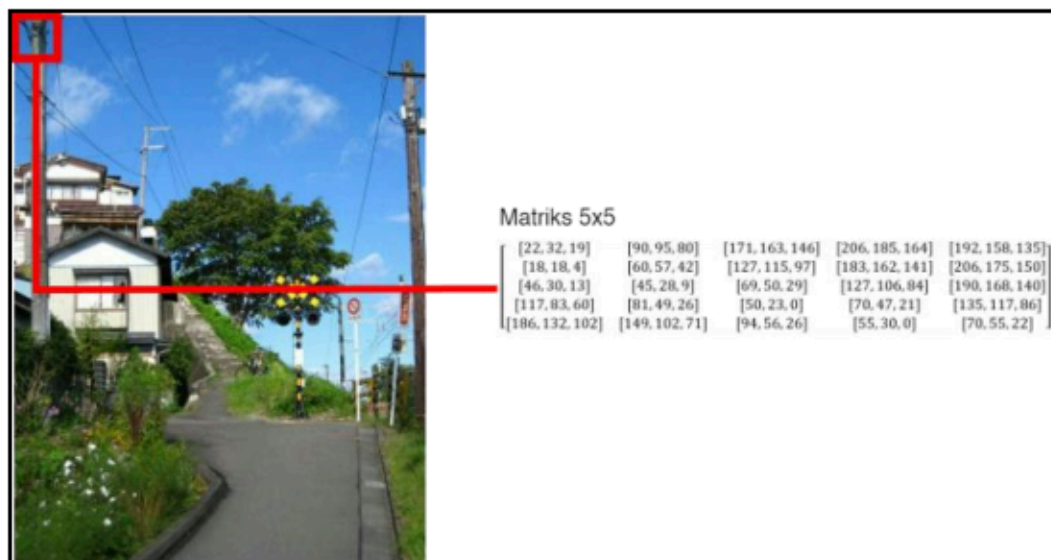
Dalam rancangan ini, terdapat dua jenis input, yaitu citra asli (referensi) dan citra tepi Canny yang dihasilkan dari citra asli tersebut. Citra asli diproyeksikan ke ruang laten melalui VAE (encoder) dan selanjutnya diproses oleh U-Net untuk menghasilkan citra bergaya animasi. Sementara itu, citra tepi Canny dimanfaatkan sebagai masukan pengondisian pada ControlNet, sehingga transformasi gaya animasi dapat dilakukan tanpa mengorbankan ketepatan struktur dari citra masukan.

Inferensi dilakukan menggunakan *Stable Diffusion Pipeline* yang disediakan dalam library *Diffusers*. Proses inferensi dimulai dengan memuat ControlNet terlatih berbasis citra tepi Canny. Selanjutnya, *pipeline Stable Diffusion ControlNet Img2Img* digunakan untuk memanggil model dasar stable diffusion dan menggabungkannya dengan ControlNet yang telah dimuat. Setelah *pipeline* siap, LoRA (Low-Rank Adaptation) yang telah dilatih dimuat ke dalam *pipeline* sistem.

Pipeline yang telah dibuat tersebut akan menerima beberapa argumen mencakup: *initial image* (citra referensi), *control image* (Canny), *strength*, *guidance scale*, dan *number of inference steps*. Selama proses ini, model secara bertahap menghilangkan *noise* dari citra dengan mencoba untuk menyeimbangkan dua aspek: konten citra referensi, struktur tepi dari ControlNet, dan gaya visual dari LoRA.

3.5.1 Citra Tepi Canny

Salah satu input dalam rancangan ini yaitu citra tepi Canny. Peta tepi tersebut diperoleh dari citra asli dengan memanfaatkan fungsi `Canny()` dari library *computer vision* OpenCV.



Gambar 3. 4 Contoh citra input Canny

Tahap pertama, citra RGB diinput, kemudian dikonversi ke dalam bentuk *grayscale*. Sebagai contoh, berikut adalah matriks 5x5 yang diambil dari **Gambar 3. 4**.

$$\begin{bmatrix} [22, 32, 19] & [90, 95, 80] & [171, 163, 146] & [206, 185, 164] & [192, 158, 135] \\ [18, 18, 4] & [60, 57, 42] & [127, 115, 97] & [183, 162, 141] & [206, 175, 150] \\ [46, 30, 13] & [45, 28, 9] & [69, 50, 29] & [127, 106, 84] & [190, 168, 140] \\ [117, 83, 60] & [81, 49, 26] & [50, 23, 0] & [70, 47, 21] & [135, 117, 86] \\ [186, 132, 102] & [149, 102, 71] & [94, 56, 26] & [55, 30, 0] & [70, 55, 22] \end{bmatrix}$$

Untuk mengubah sebuah piksel RGB ke *grayscale*, digunakan persamaan $grayscale = 0.299R + 0.587G + 0.114B$. Berikut adalah contoh perhitungan pada tiga kolom pertama dari kiri dalam baris pertama dari matriks di atas:

$$grayscale_{(1,1)} = 0.299 \cdot 22 + 0.587 \cdot 32 + 0.114 \cdot 19$$

$$grayscale_{(1,2)} = 0.299 \cdot 90 + 0.587 \cdot 95 + 0.114 \cdot 80$$

$$grayscale_{(1,3)} = 0.299 \cdot 171 + 0.587 \cdot 163 + 0.114 \cdot 146$$

...

Setelah semua piksel diubah menjadi *grayscale*, tampilan matriks berubah menjadi:

$$\begin{bmatrix} 27 & 90 & 159 & 181 & 155 \\ 14 & 53 & 111 & 158 & 171 \\ 27 & 24 & 46 & 102 & 162 \\ 80 & 46 & 19 & 42 & 110 \\ 129 & 98 & 51 & 24 & 47 \end{bmatrix}$$

Kemudian, setelah citra diubah ke dalam bentuk *grayscale*, *Gaussian blur* diaplikasikan ke citra dengan kernel 3×3 dan σ (sigma) dikalkulasi secara otomatis berdasarkan ukuran kernel, menggunakan persamaan $\sigma = 0.3 \cdot ((k - 1) \cdot 0.5 - 1) + 0.8$.

Untuk kernel dengan ukuran 3×3 , nilai sigma adalah:

$$\sigma = 0.3 \cdot ((3 - 1) \cdot 0.5 - 1) + 0.8$$

$$\sigma = 0.3 \cdot (2 \cdot 0.5 - 1) + 0.8$$

$$\sigma = 0.8$$

Setelah *gaussian blur* diaplikasikan, tampilan matriks berubah menjadi:

$$\begin{bmatrix} 39.9 & 51.7 & 133.4 & 160.7 & 155.4 \\ 32.1 & 60.5 & 103.3 & 135.8 & 155.6 \\ 39.2 & 44.8 & 65.2 & 102.2 & 140.8 \\ 71.5 & 56.8 & 47.9 & 65.1 & 105.1 \\ 108.6 & 88.4 & 61.6 & 50 & 66.7 \end{bmatrix}$$

Kemudian intensitas gradien dihitung dengan menggunakan sobel operator. Sobel operator terdiri dari dua kernel 3×3 , (G_x) untuk mendeteksi tepi horizontal, (G_y) mendeteksi tepi vertikal. Proses ini dilakukan dengan melakukan proses konvolusi kernel ke dalam citra. Berikut adalah contoh proses konvolusi pada piksel (2,2) dari matriks di atas:

Pertama, diambil tetangga 3×3 sekitar piksel (2,2):

$$\begin{bmatrix} 60.5 & 103.3 & 135.8 \\ 44.8 & 65.2 & 102.2 \\ 56.8 & 47.9 & 65.1 \end{bmatrix}$$

Kemudian kalikan kernel G_x secara bertahap:

$$\begin{bmatrix} 60.5 & 103.3 & 135.8 \\ 44.8 & 65.2 & 102.2 \\ 56.8 & 47.9 & 65.1 \end{bmatrix} \times \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} \text{gradient } x_{(2,2)} &= (60.5 \cdot (-1)) + (103.3 \cdot 0) + (135.8 \cdot 1) \\ &\quad + (44.8 \cdot (-2)) + (65.2 \cdot 0) + (102.2 \cdot 2) \\ &\quad + (56.8 \cdot (-1)) + (47.9 \cdot 0) + (65.1 \cdot 1) \end{aligned}$$

$$\text{gradient } x_{(2,2)} = -60.5 + 0 + 135.8 - 89.6 + 0 + 204.4 - 56.8 + 0 + 65.1$$

$$\text{gradient } x_{(2,2)} = 198.4$$

Proses yang sama kemudian diulang untuk kernel G_y :

$$\begin{bmatrix} 60.5 & 103.3 & 135.8 \\ 44.8 & 65.2 & 102.2 \\ 56.8 & 47.9 & 65.1 \end{bmatrix} \times \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$\text{gradient } y_{(2,2)} = -60.5 - 206.6 - 135.8 + 0 + 0 + 0 + 58.8 + 95.8 + 65.1$$

$$\text{gradient } y_{(2,2)} = -183.2$$

Setelah itu, nilai intensitas gradien dihitung, menggunakan formula Pythagoras, dalam proses ini gradien vertikal dan horizontal digabungkan untuk menghasilkan total kekuatan tepi.

$$\text{magnitude}_{(2,2)} = \sqrt{G_x^2 + G_y^2}$$

$$\text{magnitude}_{(2,2)} = \sqrt{(198.4)^2 + (-183.2)^2}$$

$$\text{magnitude}_{(2,2)} = \sqrt{38362.56 + 33562.24}$$

$$\text{magnitude}_{(2,2)} = \sqrt{71924.8}$$

$$\text{magnitude}_{(2,2)} = 268.2$$

Setelah intensitas gradien dihitung, dihitung juga arah dari gradien tersebut:

$$\theta = \arctan2(G_y, G_x)$$

$$\theta = \arctan2(-183.2, 198.4)$$

$$\theta = -0.75 \text{ radian}$$

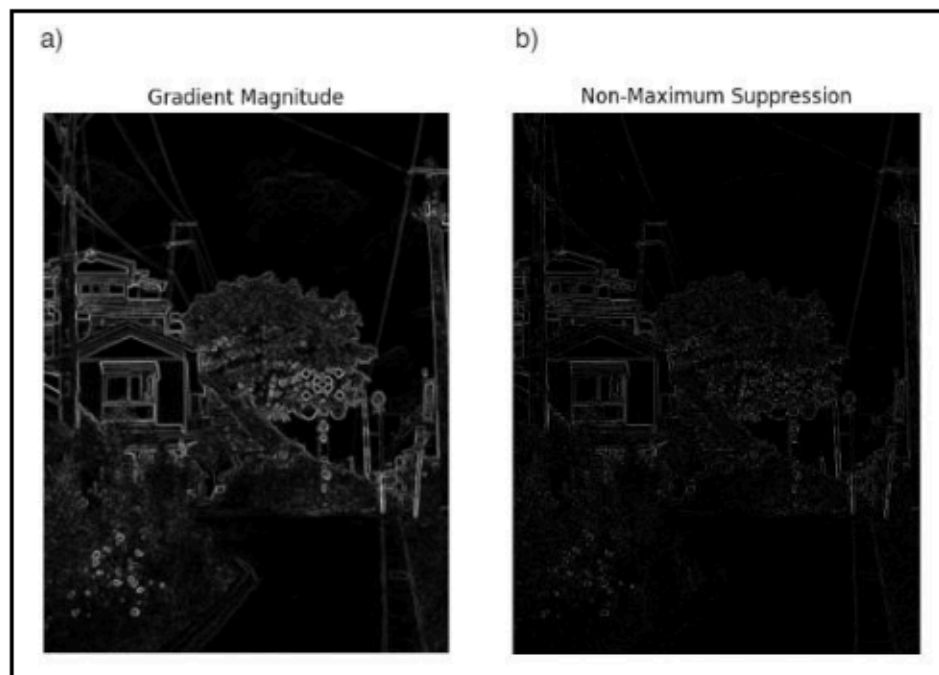
$$\theta = -42.71^\circ$$

Kedua proses ini (perhitungan intensitas dan arah gradien) kemudian diulang ke seluruh piksel dalam citra, mengubah tampilan matriks menjadi:

$$\begin{bmatrix} 160.3 & 360.7 & 332.2 & 138.4 & 64.5 \\ 111.4 & 298.3 & 375.1 & 284.3 & 83.8 \\ 117.4 & 102.1 & 268.2 & 364.7 & 271.3 \\ 255.5 & 167.6 & 39 & 266.6 & 365.6 \\ 233 & 263.4 & 186.7 & 30 & 256.8 \end{bmatrix}$$

Dengan arah gradien masing-masing pikselnya:

$$\begin{bmatrix} -2.83 & -2.24 & -3.13 & -2.89 & -3.04 \\ -0.35 & -0.49 & -0.66 & -0.78 & -0.9 \\ 1.35 & -0.23 & -0.75 & -0.77 & -0.74 \\ 1.74 & 1.99 & -0.41 & -0.75 & -0.77 \\ 1.86 & 2.19 & 2.25 & -0.35 & -0.79 \end{bmatrix}$$



Gambar 3.5 Intensitas gradien dan *Non-Maximum Supression* dalam Canny

Setelah intensitas dan arah tepi dihitung, Dapat dilihat dalam **Gambar 3. 5a** tepi yang ditemukan dalam gambar cenderung tebal dan kabur. Oleh karena itu, digunakan teknik *Non-Maximum Suppression* (NMS) untuk mengecilkan tepi menjadi lebar satu piksel, dan menghilangkan tepi lemah dan berulang.

Sebelum dilakukan *Non-maximum suppression*, arah tepi dinormalisasikan ke $0^\circ - 180^\circ$. Proses ini mengubah tampilan matriks arah menjadi:

$$\begin{bmatrix} 17.8 & 51.6 & 0.6 & 14.4 & 5.8 \\ 159.9 & 151.9 & 142.2 & 135.3 & 128.4 \\ 77.3 & 166.8 & 137 & 135.8 & 137.6 \\ 99.6 & 114 & 156.5 & 137 & 135.8 \\ 106.6 & 125.5 & 128.9 & 159.9 & 134.7 \end{bmatrix}$$

Kemudian arah yang telah dinormalisasi tersebut digunakan dalam penentuan tetangga dari sebuah piksel. Sebagai contoh, piksel (2,2) memiliki intensitas **268.2** dan arah **137°**.

$$\begin{bmatrix} 298.8 & 375.1 & 284.3 \\ 102.1 & 268.2 & 364.7 \\ 167.6 & 39 & 266.6 \end{bmatrix}$$

Karena pembulatan terdekat dari arah tepi tersebut adalah **135°**, maka tetangga yang akan dibandingkan adalah pada posisi kiri-atas dan kanan-bawah-nya, yaitu piksel (1,1) dan piksel (3,3).

Intensitas sebuah piksel kemudian dibandingkan dengan tetangganya. Apabila nilai piksel **lebih kecil** dari kedua tetangganya, nilai piksel tersebut **disupresi menjadi 0**, dan apabila nilai piksel **lebih besar** dari kedua tetangganya, maka **nilai piksel tersebut dipertahankan**. Karena intensitas piksel (2,2) tidak lebih besar dari kedua tetangganya, maka piksel tersebut disupresi menjadi 0.

Proses ini kemudian diulang untuk setiap piksel dalam citra, mengubah tampilan matriks menjadi:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 298.3 & 375.1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 365.6 \\ 0 & 263.4 & 0 & 0 & 0 \end{bmatrix}$$

Setelah *Non-maximum suppression*, dapat dilihat dalam **Gambar 3. 5b**, tepi dari citra sudah mulai terlihat. Tepi dalam citra kemudian diperkuat kembali menggunakan teknik *double thresholding*. Dalam tahap ini, piksel tepi yang telah ditemukan sebelumnya diklasifikasi menjadi tiga jenis: tepi kuat, tepi lemah, dan non-tepi. Dalam contoh ini, tepi kuat dan tepi lemah didefinisikan sebagai:

$$\text{strong edges} = 255$$

$$\text{weak edges} = 50$$

Dalam tahap ini terdapat dua parameter, yaitu *low threshold* dan *high threshold*. Sebagai contoh digunakan nilai *low threshold* = **50**, dan *high threshold* = **150**.

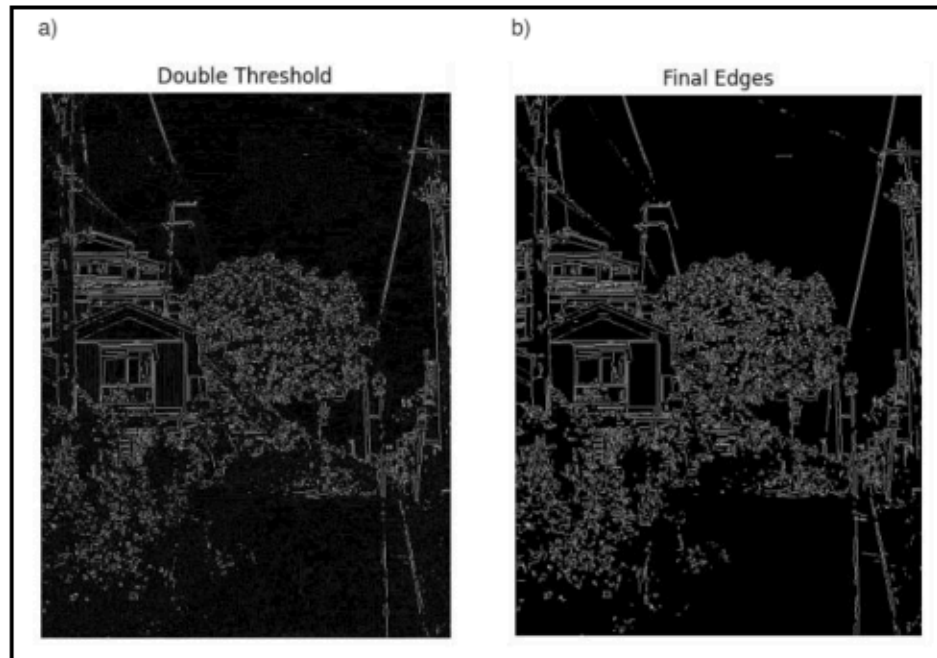
Tiap piksel dalam citra kemudian dibandingkan dengan kedua parameter *threshold* di atas, mengklasifikasikannya menggunakan kriteria sebagai berikut:

1. Piksel < **50**: Non-tepi
2. **50** ≤ Piksel < **150**: Tepi lemah
3. Piksel ≥ **150**: Tepi kuat

Setelah seluruh piksel dalam citra diklasifikasi, nilainya masing-masing dikonversi menjadi 0, 50, atau 255. Proses ini mengubah tampilan matriks menjadi:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 255 & 255 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 255 \\ 0 & 255 & 0 & 0 & 0 \end{bmatrix}$$

Tahap terakhir, tepi yang telah ditemukan diperkuat kembali melalui proses *edge tracking*. Dalam tahap ini, tiap piksel tepi lemah diperiksa apakah piksel tersebut bertetangga dengan tepi kuat. Apabila piksel tepi lemah tersebut bertetangga dengan piksel kuat, maka nilainya akan diubah menjadi nilai piksel tepi kuat = 250. Proses ini membuat seluruh piksel dalam citra memiliki nilai intensitas 0 atau 255. Dapat dilihat di **Gambar 3. 6b**, piksel-piksel samar yang sebelumnya ada dari hasil *double thresholding* (**Gambar 3. 6a**) hilang, membuat elemen tepi dalam citra terlihat lebih jelas.

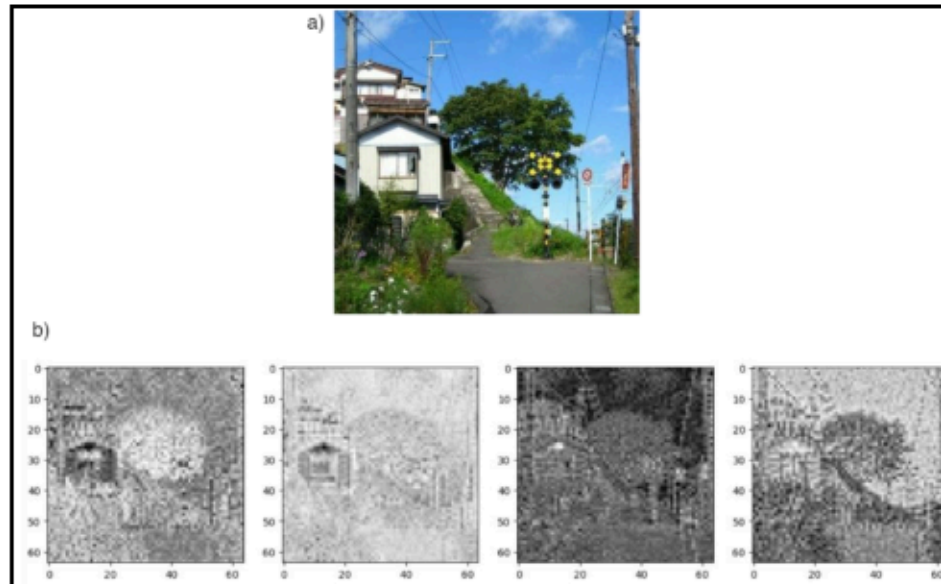


Gambar 3. 6 *Double threshold* dan hasil akhir citra tepi Canny

3.5.2 Variational Autoencoder

Sebelum memasuki tahap difusi, citra masukan terlebih dahulu diencode ke dalam representasi laten menggunakan *variational autoencoder* (VAE). VAE terdiri dari dua komponen utama, yaitu *encoder* yang mentransformasikan citra berdimensi tinggi ke ruang laten yang lebih ringkas, dan *decoder* yang merekonstruksi kembali representasi laten tersebut menjadi citra akhir.

Sebagai ilustrasi, sebuah citra asli (**Gambar 3. 7a**) berdimensi 512x512 dengan 3 channel setelah melalui encoder VAE akan diproyeksikan menjadi representasi laten berukuran 64x64 dengan 4 *channel*.



Gambar 3. 7 *Encoding citra menggunakan Variational Autoencoder (VAE)*

3.5.3 Arsitektur U-Net

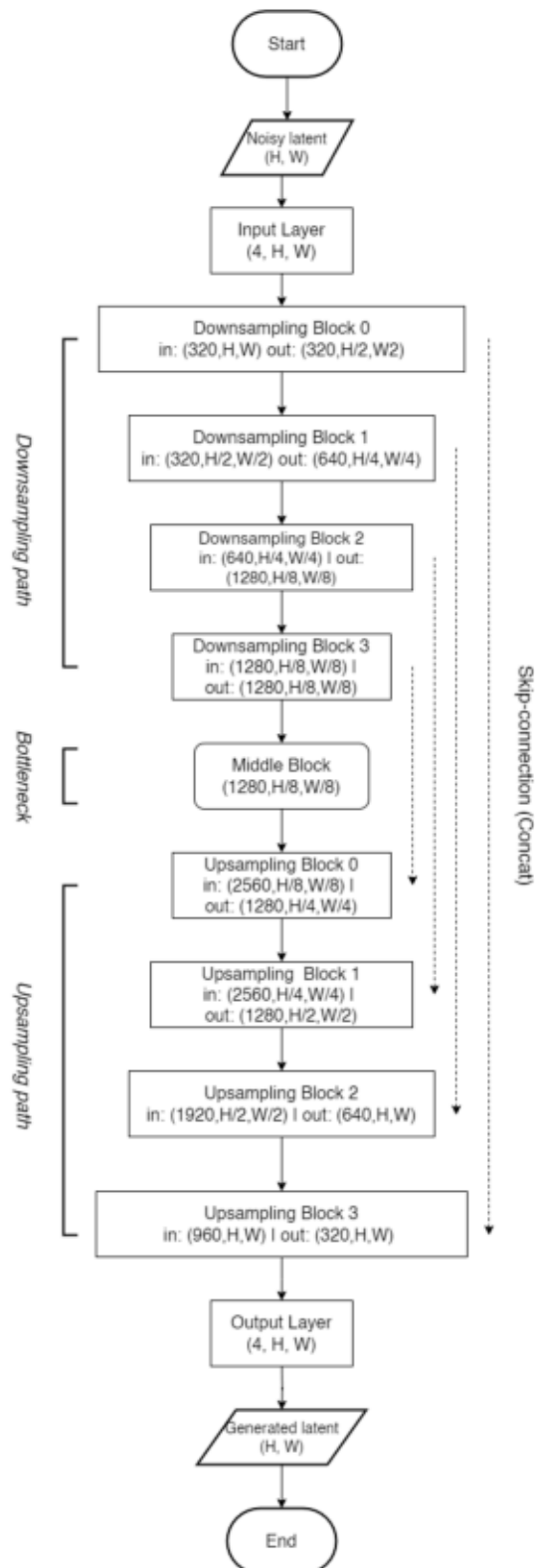
Arsitektur U-Net merupakan komponen utama dari *image generation* karena dalam arsitektur ini lah proses *denoising* berlangsung. *Denoising* berfungsi untuk secara bertahap menghilangkan *noise* dari citra awal yang sepenuhnya acak (*pure noise*), sampai terbentuknya struktur dan detail visual dalam citra yang sesuai dengan pengondisian yang diberikan. Proses *denoising* ini dilakukan secara bertahap sebanyak T langkah (*timestep*) untuk mengubah *pure noise* menjadi citra yang bermakna. Dalam penelitian ini, jumlah *timestep* yang digunakan adalah $T = 50$.

Pada tugas *image-to-image translation* dalam *Stable Diffusion*, laten input terlebih dahulu diberi *noise* sebelum memasuki tahap *denoising*. Jumlah *noise* yang ditambahkan dikendalikan oleh parameter *strength* dengan rentang nilai antara 0 hingga 1, di mana nilai 1 merepresentasikan kondisi *pure noise* dan nilai 0 menunjukkan tidak adanya penambahan *noise*. Parameter *strength* ini mencerminkan posisi awal citra dalam rentang keseluruhan *timestep*. Semakin besar nilai *strength*, semakin banyak *step* yang dilalui dan semakin dekat dengan nilai T , sehingga proses *denoising* dimulai dari tahap lebih awal dengan tingkat *noise* yang tinggi. Sebaliknya, nilai *strength* yang lebih kecil menyebabkan proses *denoising* berlangsung pada tahap lebih akhir dengan tingkat *noise* yang lebih sedikit, sehingga citra hasilnya mempertahankan lebih banyak detail dari gambar asli.

Sebagai contoh, digunakan nilai *strength* sebesar 0.7. Dengan jumlah *step* $T = 50$, maka jumlah *timestep* total yang baru menjadi:

$$T = 50 \times 0.7 = 35 \text{ step}$$

Secara umum, arsitektur U-Net pada *stable diffusion* memiliki struktur yang sama dengan U-Net konvensional, yaitu terdiri atas tiga bagian utama: *downsampling*, *middle block (bottleneck)*, dan *upsampling*. Berikut adalah *flowchart* arsitektur dalam U-Net *stable diffusion*:



Gambar 3. 8 Flowchart proses di dalam U-Net (Hugging Face n.d.)



Gambar 3. 9 Citra input setelah melalui tahap *encoding* dan penambahan *noise*

Dalam input layer, input laten *noisy* 4 *channel* akan menghasilkan peta fitur 320 channel melalui proses konvolusi 3x3 dengan stride dan padding 1. Berikut adalah contoh perhitungannya pada matriks 5x5 diambil dari salah satu *channel* dari laten *noisy* dari Gambar 3. 9b:

a. Input *noise*

$$\begin{bmatrix} 6 & 3 & 7 & 4 & 6 \\ 9 & 2 & 6 & 7 & 4 \\ 3 & 7 & 7 & 2 & 5 \\ 4 & 1 & 7 & 5 & 1 \\ 4 & 0 & 9 & 5 & 8 \end{bmatrix}$$

b. Padding

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & \dots \\ 0 & 6 & 3 & 7 & 4 & 6 & \dots \\ 0 & 9 & 2 & 6 & 7 & 4 & \dots \\ 0 & 3 & 7 & 7 & 2 & 5 & \dots \\ 0 & 4 & 1 & 7 & 5 & 1 & \dots \\ 0 & 4 & 0 & 9 & 5 & 8 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

c. Konvolusi

Proses filtering bekerja dengan stride berpindah 1 piksel ke arah kanan dan ke bawah.

Proses filtering dilakukan menggunakan persamaan:

$$S(i, j) = \sum_m \sum_n I(m, n) \cdot K(i - m, j - n)$$

$$\text{Matriks input padding} \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 3 & 7 & 4 & 6 & 0 \\ 0 & 9 & 2 & 6 & 7 & 4 & 0 \\ 0 & 3 & 7 & 7 & 2 & 5 & 0 \\ 0 & 4 & 1 & 7 & 5 & 1 & 0 \\ 0 & 4 & 0 & 9 & 5 & 8 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \text{filter:} \begin{bmatrix} 0.01 & -0.04 & 0.02 \\ 0.03 & 0.06 & -0.05 \\ -0.07 & 0.08 & 0.04 \end{bmatrix}$$

$$\text{Stride (1)} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 6 & 3 \\ 0 & 9 & 2 \end{bmatrix} \times \begin{bmatrix} 0.01 & -0.04 & 0.02 \\ 0.03 & 0.06 & -0.05 \\ -0.07 & 0.08 & 0.04 \end{bmatrix}$$

$$\text{Stride (2)} \begin{bmatrix} 0 & 0 & 0 \\ 6 & 3 & 7 \\ 9 & 2 & 6 \end{bmatrix} \times \begin{bmatrix} 0.01 & -0.04 & 0.02 \\ 0.03 & 0.06 & -0.05 \\ -0.07 & 0.08 & 0.04 \end{bmatrix}$$

$$\text{Stride (3)} \begin{bmatrix} 0 & 0 & 0 \\ 3 & 7 & 4 \\ 2 & 6 & 7 \end{bmatrix} \times \begin{bmatrix} 0.01 & -0.04 & 0.02 \\ 0.03 & 0.06 & -0.05 \\ -0.07 & 0.08 & 0.04 \end{bmatrix}$$

...

$$S(0,0) = (0.01 \times 0) + (-0.04 \times 0) + (0.02 \times 0) + (0.03 \times 0) + (0.06 \times 6) + (-0.05 \times 3) \\ + (-0.07 \times 0) + (0.08 \times 9) + (0.04 \times 2) = 1.01$$

$$S(1,0) = (0.01 \times 0) + (-0.04 \times 0) + (0.02 \times 0) + (0.03 \times 6) + (0.06 \times 3) + (-0.05 \times 7) \\ + (-0.07 \times 9) + (0.08 \times 2) + (0.04 \times 6) = -0.22$$

...

Berikutnya perhitungan dilanjutkan hingga stride (25), sehingga perhitungan menghasilkan matriks 5x5 baru:

$$\begin{bmatrix} 1.01 & -0.22 & 0.93 & 0.45 & 0.31 \\ 0.78 & 0.8 & 0.05 & 0.3 & 0.51 \\ -0.13 & 0.37 & 1.14 & -0.11 & 0 \\ 0.53 & -0.20 & 0.95 & 0.64 & 0.32 \\ 0.1 & -0.19 & 0.12 & 0.06 & 0.64 \end{bmatrix}$$

Setelah diproses oleh lapisan input, peta fitur akan memasuki tahap *downsampling*. Tahap *downsampling* terdiri dari *downsampling* block dengan resolusi masing-masing 64×64 , 32×32 , 16×16 , dan 8×8 . Dalam setiap *downsampling* block terdapat Visual Transformer (ViT) untuk mekanisme *self-attention* dan *cross-attention*, ResNet block, dan lapisan *downsampling*. Di akhir tahap *downsampling* ini, dimensi peta fitur berkurang 8 kali lipat (8×8), dengan jumlah channel 1280.

Setelah melalui bagian *downsampling*, peta fitur memasuki *middle block*, di mana fitur akan diproses pada resolusi terendahnya. Dalam *middle block* ini terdapat Visual Transformer (ViT) dan ResNet Block.

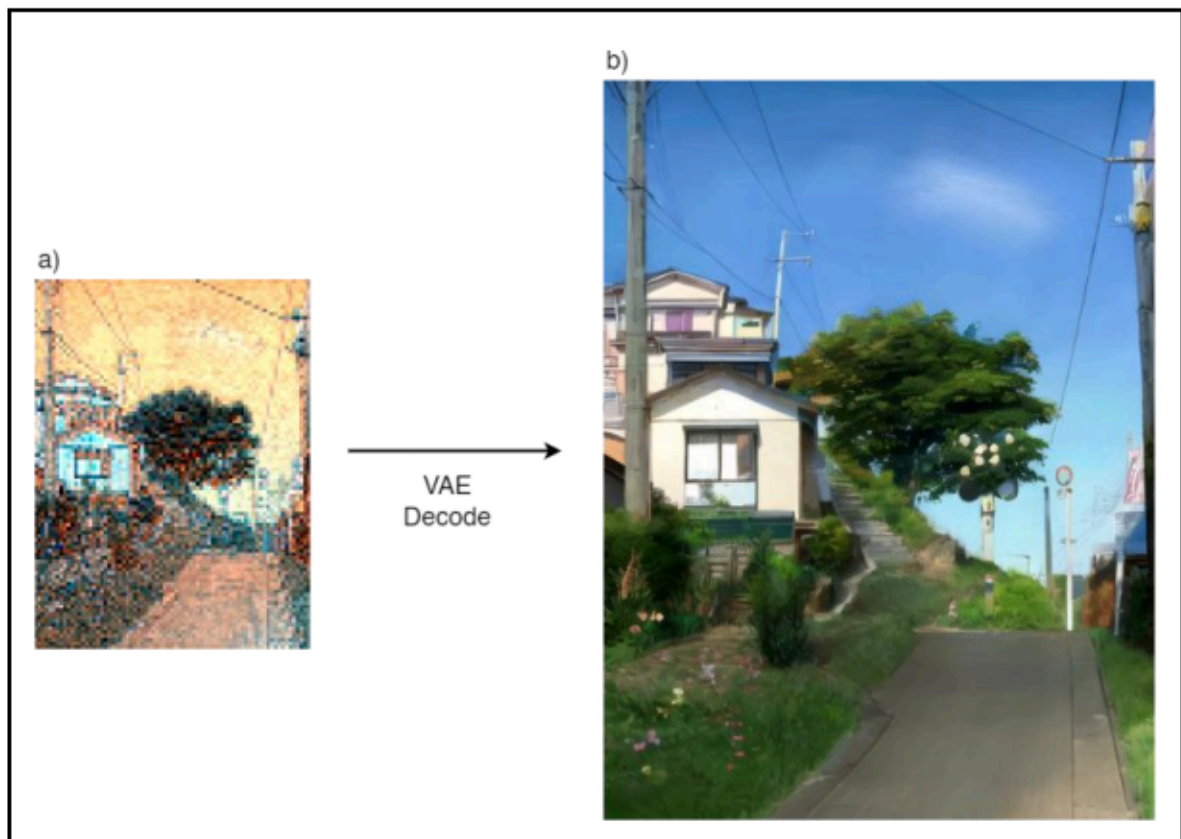
Setelah melalui bagian *middle block*, peta fitur memasuki bagian *upsampling*. Tahap *upsampling* terdiri dari sejumlah *upsampling* block yang terdiri dari 4 resolusi seperti pada tahap *downsampling*, namun urutannya yang terbalik. Tiap blok dalam tahap ini mengambil 2 input, yaitu input peta fitur dari tahap sebelumnya dan *skip-connection* peta fitur dari bagian *downsampling*, yang kemudian keduanya digabungkan (*concat*) sepanjang dimensi *channel*. Sebagai contoh, dapat dilihat pada gambar *Upsampling Block 1*, mengambil 2 input, yaitu fitur dari tahap sebelumnya, *Upsampling block 0* dengan bentuk output

(1280,16,16), dan *skip connection* dari *Downsampling block 2* dengan bentuk (640,16,16), kedua input ini kemudian digabungkan (*concat*) mengubah bentuk input menjadi (1920,16,16). Dalam setiap *upsampling block* terdapat Visual Transformer (ViT), ResNet block, dan lapisan *upsampling*. Di akhir tahap *upsampling* peta fitur kembali ke resolusi yang sama pada saat diinput ke U-Net, yaitu (320,64,64).

Tahap terakhir dalam U-Net yaitu output layer, di mana fitur 320 channel diproses kembali menjadi fitur 4 channel dengan resolusi 64x64, melalui *Group Normalization*, aktivasi SiLU, dan lapisan konvolusi 3x3 dengan *stride* dan *padding* 1.

Pada setiap *timestep t*, U-Net memprediksi komponen *noise* yang terkandung dalam laten *noisy* z_t . Laten yang lebih bersih, z_{t-1} , diperoleh dengan mengurangi *noise* yang sudah diprediksi tersebut dari z_t .

Laten yang dihasilkan dari proses *denoising* tersebut kemudian diproses kembali melalui VAE untuk di-*decode* menjadi gambar utuh, menghasilkan tekstur yang telah mengalami transformasi dari bentuk aslinya.



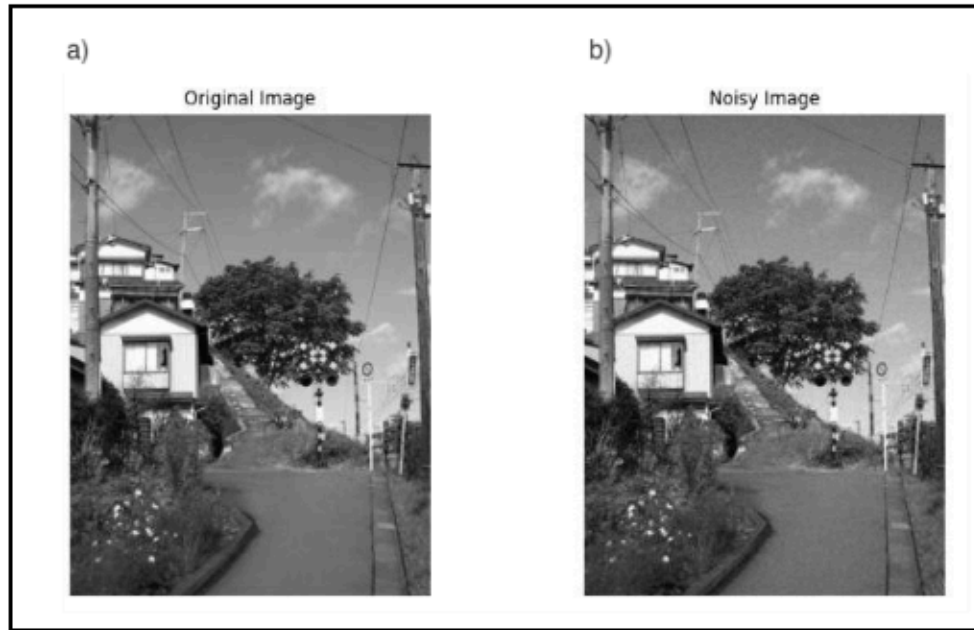
Gambar 3. 10 Laten di-*decode* untuk menghasilkan output beresolusi penuh

3.6 Pengujian

3.6.1 SSIM

Structural Similarity Index (SSIM) adalah metode objektif untuk menilai kualitas citra berdasarkan kesamaan struktural antara citra referensi dan citra hasil distorsi. SSIM

membandingkan dua citra non-negatif x (referensi) dan y (terdistorsi) dengan memisahkan analisis menjadi tiga bagian, yaitu luminansi, kontras, dan struktur.



Gambar 3. 11 Contoh citra referensi dan terdistorsi

Dalam perbandingan luminansi, tahap pertama, rata-rata luminansi lokal dari kedua citra (referensi dan terdistorsi) dihitung. Sebagai contoh, berikut adalah matriks 5x5 yang diambil citra referensi dari **Gambar 3. 11a**.

$$\begin{bmatrix} 27 & 90 & 159 & 181 & 155 \\ 14 & 53 & 111 & 158 & 171 \\ 27 & 24 & 46 & 102 & 162 \\ 80 & 46 & 19 & 42 & 110 \\ 129 & 98 & 51 & 24 & 47 \end{bmatrix}$$

Perhitungan SSIM ini menggunakan kernel 3×3 , nilai $K_1 = 0.01$, $K_2 = 0.03$, dan $L = 255$. Sebelum perhitungan dilakukan, konstanta C_1 dan C_2 ditentukan terlebih dahulu menggunakan:

$$C_1 = (K_1 \times L)^2 = (0.01 \times 255)^2 = 6.51$$

$$C_2 = (K_2 \times L)^2 = (0.03 \times 255)^2 = 58.52$$

Citra kemudian dilakukan padding menggunakan *reflect padding*, mengubah tampilan matriks menjadi:

$$\begin{bmatrix} 53 & 14 & 53 & 111 & 158 & 171 \\ 90 & 27 & 90 & 159 & 181 & 155 \\ 53 & 14 & 53 & 111 & 158 & 171 \\ 24 & 27 & 24 & 46 & 102 & 162 \\ 46 & 80 & 46 & 19 & 42 & 110 \\ 98 & 129 & 98 & 51 & 24 & 47 \end{bmatrix}$$

Kemudian rata-rata luminansi lokal dihitung melalui perkalian dengan kernel. Berikut adalah contoh perhitungannya pada piksel (0,0):

$$\begin{aligned}\mu_{x(0,0)} &= \begin{bmatrix} 53 & 14 & 53 \\ 90 & 27 & 90 \\ 53 & 14 & 53 \end{bmatrix} \times \begin{bmatrix} 0.111 & 0.111 & 0.111 \\ 0.111 & 0.111 & 0.111 \\ 0.111 & 0.111 & 0.111 \end{bmatrix} \\ \mu_{x(0,0)} &= \begin{bmatrix} 5.888 & 1.555 & 5.888 \\ 10 & 3 & 10 \\ 5.888 & 1.555 & 5.888 \end{bmatrix} \\ \mu_{x(0,0)} &= \frac{5.888 + 1.555 + 5.888 + \dots + 5.888}{9} \\ \mu_{x(0,0)} &= 49.67\end{aligned}$$

Proses ini kemudian diulang ke seluruh piksel dalam kedua citra, menghasilkan peta luminansi lokal dari citra referensi dan citra terdistorsi. Proses ini mengubah tampilan matriks citra referensi menjadi:

$$\begin{bmatrix} 49.6 & 70.2 & 119.3 & 152.7 & 165.6 \\ 44.6 & 61.2 & 102.6 & 138.3 & 152.2 \\ 40.7 & 46.6 & 66.7 & 102.3 & 116.3 \\ 63.5 & 57.7 & 50.2 & 67 & 72.7 \\ 74.3 & 63.1 & 43 & 51.5 & 53.6 \end{bmatrix}$$

dan citra terdistorsi menjadi:

$$\begin{bmatrix} 55.1 & 74.8 & 124.4 & 155.9 & 167.6 \\ 46.2 & 63.3 & 104.2 & 137.7 & 149.4 \\ 39.8 & 46.7 & 65.2 & 98.2 & 108.2 \\ 60.7 & 57.2 & 45.4 & 60.6 & 61.7 \\ 60.6 & 62.3 & 38.7 & 46.3 & 44.9 \end{bmatrix}$$

Setelah itu, perhitungan varians lokal dihitung dengan mengkuadratkan nilai tiap piksel dan dikalikan dengan kernel. Proses ini mengubah tampilan matriks citra referensi menjadi:

$$\begin{bmatrix} 706.2 & 2264.6 & 2018.4 & 561.1 & 97.3 \\ 740.8 & 2124.8 & 2718.6 & 1681.3 & 802.6 \\ 373.5 & 891.5 & 1921.9 & 2807 & 2227.5 \\ 1334.2 & 1233.2 & 824.2 & 1997.1 & 2041.3 \\ 820.4 & 1194.7 & 513.5 & 1101.6 & 966.2 \end{bmatrix}$$

dan citra terdistorsi menjadi:

$$\begin{bmatrix} 778.2 & 2413.1 & 1818.6 & 581.4 & 261.3 \\ 851.4 & 2343.1 & 2936.1 & 1877.8 & 1381.5 \\ 490.8 & 1033.8 & 1887.4 & 2724.2 & 2461.9 \\ 1650 & 1456.6 & 753 & 1652.4 & 1859.8 \\ 1172.6 & 1364.5 & 530.5 & 907.1 & 921.5 \end{bmatrix}$$

Setelah nilai rata-rata dan varians lokal piksel ditemukan, perhitungan SSIM lokal dapat dimulai. Berikut adalah contoh perhitungan SSIM lokal pada piksel (0,0):

Diketahui:

$$\begin{aligned}\mu_{x(0,0)} &= 49.67 & \mu_{x(0,0)}\mu_{y(0,0)} &= 2734.7 \\ \mu_{y(0,0)} &= 55.06 & \sigma_{x(0,0)}^2 &= 706.22 \\ \mu_{x(0,0)}^2 &= 2466.78 & \sigma_{y(0,0)}^2 &= 778.15 \\ \mu_{y(0,0)}^2 &= 3031.73 & \sigma_{xy(0,0)} &= 719.86\end{aligned}$$

$$\begin{aligned}SSIM &= \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \\ SSIM &= \frac{(2 \cdot 2734.7 + 6.51) \cdot (2 \cdot 719.86 + 58.52)}{(2466.78 + 3031.73 + 6.51) \cdot (706.2 + 778.15 + 58.52)} \\ SSIM &= \frac{8204268.90}{8493632.93} \\ SSIM &= 0.965\end{aligned}$$

Proses ini kemudian diulang ke seluruh piksel dalam kedua citra, menghasilkan peta SSIM lokal pada tiap pikselnya. Nilai-nilai SSIM per piksel tersebut kemudian dirata-rata untuk menemukan nilai *mean SSIM* (MSSIM), untuk mendapat nilai similaritas keseluruhan antara dua citra.

3.6.2 LPIPS

Learned Perceptual Image Patch Similarity (LPIPS) merupakan pengembangan dari metrik kesamaan citra tradisional, seperti PNSR dan SSIM, yang dirancang untuk menyerupai penilaian perseptual manusia.

LPIPS bekerja dengan memanfaatkan fitur internal dari *deep neural network* terlatih. LPIS bekerja dengan menghitung jarak antara dua citra, citra referensi x dan citra terdistorsi x' dalam ruang fitur mendalam (*deep feature space*) dari jaringan saraf konvolusional (CNN) terlatih.

LPIPS menggunakan fitur yang diekstrak dari lapisan-lapisan tertentu dari CNN terlatih, seperti arsitektur AlexNet dan VGG. Untuk setiap lapisan l dari jaringan, fitur yang diekstrak akan dinormalisasi pada dimensi *channel*. Selanjutnya aktivasi ini diskalakan per *channel* menggunakan vektor bobot w_1 .

Jarak antara fitur citra referensi dan citra terdistorsi dihitung menggunakan jarak L_2 (*Euclidean distance*). Jarak L_2 ini kemudian dirata-rata di seluruh dimensi spasial (tinggi dan lebar) dan dijumlahkan di seluruh lapisan yang digunakan.

3.6.3 Penilaian oleh Pakar

Untuk kemampuan menguji sistem dalam mentransfer gaya, terutama *painterly textures* dari data latih, dilakukan penilaian oleh pakar. Tiap gambar keluaran akan diberi penilaian dari 1 hingga 10 berdasarkan satu kriteria, yaitu seberapa mirip gaya dalam citra output dengan data latih. Semakin tinggi skornya, semakin baik gaya diaplikasikan ke dalam citra output.

3.7 Analisis Kebutuhan Data

3.7.1 Jenis dan Sumber Data

Jenis sumber data pada penelitian ini adalah data kuantitatif. Data kuantitatif adalah data yang dapat diukur secara numerik. Sumber data yang akan digunakan dalam penelitian adalah data primer, karena data yang akan digunakan dalam penelitian adalah tangkapan layar film yang diperoleh dan dikompilasi secara mandiri oleh peneliti.

3.7.2 Prosedur Pengumpulan Data

Proses pengumpulan data, untuk training, dilakukan dengan *scrapping* halaman web *Fancaps* untuk memperoleh gambar tangkapan layar dari film tertentu. Data kemudian dipilah agar hanya gambar dominan *background* digunakan untuk pelatihan.

3.8 Analisis Kebutuhan Sistem

3.8.1 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan apa yang harus dikerjakan oleh sistem. Kebutuhan fungsional yang akan dimiliki oleh sistem adalah:

1. Sistem dapat menampilkan gambar background art bergaya animasi yang dihasilkan
2. Sistem dapat melakukan proses difusi untuk menghasilkan gambar
3. Sistem dapat menggunakan citra tepi *Canny* untuk pengondisian proses difusi

3.8.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mendefinisikan bagaimana sistem bekerja. Kebutuhan non-fungsional dalam sistem ini adalah:

1. Kebutuhan perangkat keras

Penelitian ini dilakukan dengan dukungan perangkat keras, yaitu satu buah unit laptop Asus Zephyrus G14 GA401IU dengan spesifikasi:

- a. AMD Ryzen 7 4800HS (16 CPUs) ~ 2.9 GHz
- b. Nvidia GeForce GTX 1660 Ti with Max-Q Design
- c. *Random Access Memory* (RAM) 16 GB DDR4
- d. *Solid State Disk* 512 GB NVME
- e. Resolusi layar (1920x1080)

Penelitian ini juga didukung arsitektur *cloud* yang dijalankan melalui platform RunPod dengan GPU Nvidia RTX A4500, 20 GB VRAM, dan 62 GB RAM.

2. Kebutuhan perangkat lunak

Perangkat lunak yang digunakan dalam penelitian ini adalah sebagai berikut:

- a. Windows sebagai *operating system*
- b. Google Chrome sebagai *web browser*
- c. VSCode sebagai *code editor*
- d. Diffusers sebagai library untuk menjalankan *stable diffusion*
- e. Kohya-SS sebagai library untuk menjalankan training LoRA
- f. Python sebagai bahasa pemrograman

3.9 Perancangan Sistem

3.9.1 Perancangan Antar Muka

Antarmuka sistem pada penelitian ini terdiri atas satu halaman utama. Pada halaman tersebut, pengguna dapat mengunggah gambar referensi, memasukkan *text prompt*, serta menentukan nilai-nilai parameter yang diperlukan, seperti *strength*, jumlah *timestep*, *canny threshold*, dan *guidance scale*.

Bagian pertama, terdapat sebuah file uploader, di mana pengguna dapat mengunggah gambar referensi ke dalam aplikasi. Selanjutnya adalah bagian terkait Canny, yang terdiri atas dua kolom. Kolom kiri mencakup dua *slider* untuk menentukan nilai *low threshold* dan *high threshold* Canny, dan tombol “**Generate Canny**” yang berfungsi untuk menghasilkan peta tepi Canny dari gambar referensi yang sudah diinput. Hasil citra tepi kemudian ditampilkan di kolom bagian kanan, agar pengguna dapat meninjaunya.

Setelah itu, pengguna dapat menekan tombol “**Generate Image**” untuk memulai proses translasi gambar bergaya animasi. Gambar hasil keluaran akan ditampilkan secara penuh di bagian paling bawah halaman, dan pengguna dapat menyimpan gambar tersebut setelah proses selesai.

The image shows a web-based user interface for a system. At the top, there is a header bar. Below it, the main content area contains several interactive elements: an 'Open file' button next to a text input field; two sliders labeled 'Low threshold' and 'High threshold' with circular handles; a 'Generate Edge' button; a 'Prompt' label above a large text input field; three more sliders labeled 'Guidance scale', 'Strength', and 'Num inference steps', each with a circular handle; and finally, a 'Generate Image' button at the bottom. To the right of the 'Low threshold' and 'High threshold' sliders is a square area with a large 'X' inside, likely a placeholder for a generated image or Canny edge map.

Gambar 3. 12 Perancangan antar muka sistem

3.9.2 Pengujian Sistem

Proses pengujian sistem dalam penelitian ini dilakukan dengan menggunakan metode *black box testing*. *Black box testing* adalah metode pengujian perangkat lunak yang berfokus pada fungsionalitas sistem tanpa memeriksa struktur internal kode atau logika program. Berikut adalah tabel *functional test case* yang digunakan dalam penelitian ini:

Tabel 3. 1 *functional test case* pengujian sistem

No	Test case	Input	Expected Output	Status
1.	Mengunggah gambar input pertama kali	File citra RGB berformat .jpg, .jpeg, .png	Gambar berhasil dimuat	
2.	Mengunggah gambar lain di sesi yang sama	File citra RGB berformat .jpg, .jpeg, .png	Gambar berhasil dimuat, dan, apabila citra tepi Canny sudah dihasilkan di sesi sebelumnya, akan di-reset.	
3.	Melakukan transformasi ke citra tepi Canny	File citra RGB berformat .jpg, .jpeg, .png	Citra tepi Canny dari gambar yang sudah diunggah.	
4.	Melakukan transformasi gambar ke gaya animasi	Gambar input, text prompt, dan parameter denoising strength	Gambar hasil transformasi	

BAB IV

HASIL PENGUJIAN DAN PEMBAHASAN

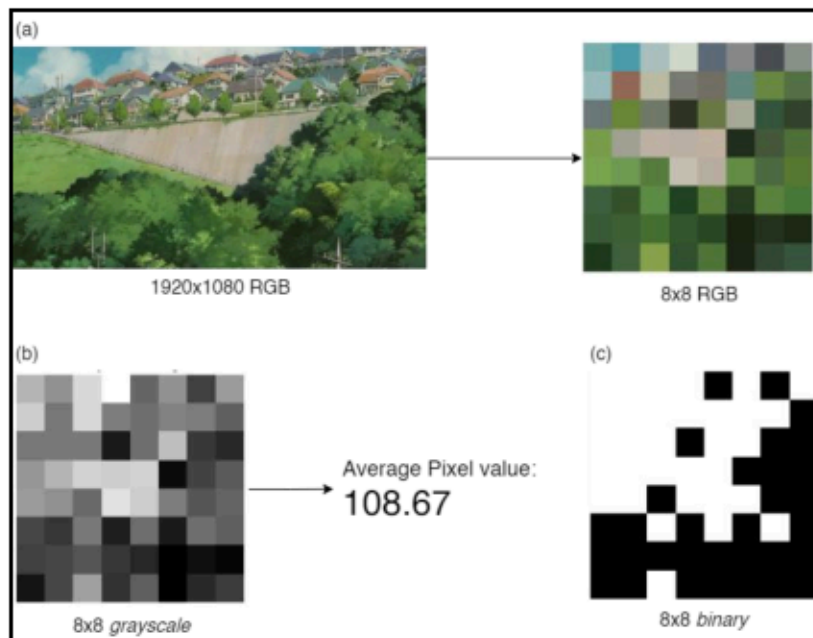
4.1 Implementasi

Bagian ini menjabarkan pelaksanaan pengembangan model dan sistem. Implementasi mencakup perwujudan dari rancangan yang telah diuraikan sebelumnya, serta tahapan-tahapan yang ditempuh dalam mengembangkan model dan sistem.

4.1.1 Pengumpulan Data

Data untuk fine-tuning stable diffusion diambil dari halaman web fanscap.com. Data diambil dari delapan film yang dibuat oleh Studio Ghibli, yaitu *Howl's Moving Castle*, *Kiki's Delivery Service*, *My Neighbor Totoro*, *Ocean Waves*, *Only Yesterday*, *Pom Poko*, *Spirited Away*, dan *Whisper of the Heart*. Setelah dikumpulkan, data berulang dihilangkan dengan membagi citra menjadi beberapa adegan, dan menyisakan hanya satu citra pada setiap adegannya.

Sebelum dilakukan pembagian adegan, citra diubah menjadi *hash* 64-bit terlebih dahulu, seperti yang dipaparkan dalam **Gambar 4. 1**. Proses ini kemudian diulang untuk semua citra seluruh dataset.



Gambar 4. 1 Contoh proses *hashing*

Untuk membagi data menjadi beberapa adegan, digunakan perbandingan *perceptual similarity* antara dua hash 64 piksel yang telah dihasilkan sebelumnya. Dalam penelitian ini, digunakan *similarity threshold* sebesar 0.75. Setelah citra dibagi menjadi adegan-adegan, seluruh citra dalam tiap adegannya dihapus hanya menyisakan satu citra tiap adegannya.

Setelah data berulang dihilangkan, dilakukan penguraian secara manual. Gambar yang akan dipilih sebagai data latih adalah gambar dominan *background*, atau lebih

spesifiknya gambar yang memiliki elemen *background* yang lebih dominan daripada subjeknya. **Gambar 4. 2** merupakan contoh gambar-gambar dominan *background* yang dipilih untuk data latih. **Gambar 4. 3** merupakan contoh gambar-gambar yang tidak dipilih untuk data latih.



Gambar 4. 2 Contoh gambar dominan background



Gambar 4. 3 Contoh gambar dominan subjek

4.1.2 Pembuatan Model (*Fine-Tuning Stable Diffusion*)

Dalam pembuatan model, hal pertama yang harus dilakukan adalah penentuan nama proyek, model dasar yang akan digunakan, dan nama dataset. Untuk menentukan model dasar yang akan digunakan dalam pelatihan, URL hugging face, atau CivitAI, file *checkpoint* dari

model dasar tersebut harus diinput ke dalam *script* pelatihan. Dalam penelitian ini, *stable diffusion v1.5* digunakan sebagai model dasar dalam pelatihan LoRA.

Setelah nama proyek, dataset, dan URL model dasar diinput, dibuat direktori lain yang akan digunakan untuk menyimpan data latih, *repository trainer*, *log*, *config file*, dan *output* model yang dihasilkan, dijabarkan dalam **Modul 4.1**. Data latih kemudian ditempatkan dalam satu folder.

Modul 4.1 *Source code* penentuan direktori

```
root_dir = "/kaggle/working"
deps_dir = os.path.join(root_dir, "deps")
repo_dir = os.path.join(root_dir, "kohya-trainer")

main_dir      = os.path.join(root_dir, "loras")
log_folder    = os.path.join(main_dir, "_logs")
config_folder = os.path.join(main_dir, project_name)
images_folder = os.path.join("/kaggle", "input", dataset_name)
output_folder = os.path.join(main_dir, project_name, "output")

config_file = os.path.join(config_folder, "training_config.toml")
dataset_config_file = os.path.join(config_folder, "dataset_config.toml")
accelerate_config_file = os.path.join(repo_dir,
"accelerate_config/config.yaml")
. . .
```

Setelah itu, ditentukan *hyperparameter* yang akan digunakan dalam pelatihan LoRA, dijabarkan dalam **Modul 4.2**.

Modul 4.2 *Source code* penentuan *hyperparameter* konfigurasi

```
resolution = 768
flip_aug = False
color_aug = False
random_crop = False

caption_extention = ".txt"
shuffle_tags = True
shuffle_caption = True
activation_tags = 1

num_repeats = 2
preferred_units = "epochs" # ["epoch", "steps" : type:String]
how_many = 10
. . .
save_every_n_epochs = 1
keep_only_last_n_epochs = 10
. . .
UNET_lr = 5e-4
text_encoder_lr = 0
lr_scheduler = "cosine_with_restarts"
lr_scheduler_number = 3
lr_scheduler_num_cycles = lr_scheduler_number if lr_scheduler.lower() ==
"cosine_with_restarts" else 0
```

Modul 4.2 *Source code* penentuan *hyperparameter* konfigurasi

```
lr_warmup_ratio = 0.05
lr_warmup_steps = 0
...
min_snr_gamma_value = 6.0

lora_type = "LoCon"

network_dim = 32
network_alpha = 32
conv_dim = 16
conv_alpha = 8

network_module = "networks.lora"
...
```

Hyperparameter tersebut kemudian dikompilasi menjadi dalam file konfigurasi (*config file*) berformat (toml). Fungsi `create_config()` bertanggung jawab dalam menyusun dan membuat dan menyusun file konfigurasi ini. Dalam pelatihan, terdapat dua konfigurasi yang digunakan, yaitu konfigurasi pelatihan dan konfigurasi dataset.

Modul 4.3 *Source code* pembuatan file konfigurasi; pengaturan *training* LoRA dan optimasinya

```
def create_config():
    global dataset_config_file, config_file, model_file

    ...
    config_dict = {
        "additional_network_arguments": {
            "unet_lr": unet_lr,
            "text_encoder_lr": text_encoder_lr,
            "network_dim": network_dim,
            "network_alpha": network_alpha,
            "network_module": network_module,
            "network_args": network_args,
            "network_train_unet_only": True if text_encoder_lr == 0 else None,
            "network_weights": continue_from_lora if continue_from_lora else None
        },
        "optimizer_arguments": {
            "learning_rate": unet_lr,
            "lr_scheduler": lr_scheduler,
            "lr_scheduler_num_cycles": lr_scheduler_num_cycles if lr_scheduler ==
"cosine_with_restarts" else None,
            "lr_scheduler_power": lr_scheduler_power if lr_scheduler ==
"polynomial" else None,
            "lr_warmup_steps": lr_warmup_steps if lr_scheduler != "constant" else
None,
            "optimizer_type": optimizer,
            "optimizer_args": optimizer_args if optimizer_args else None,
        },
        ...
    }
```

Dalam konfigurasi pelatihan, pertama terdapat komponen `additional_network_arguments` yang mengatur bagaimana adaptor LoRA dilatih

dari model dasar. Di sini, *UNet learning rate* dan *text encoder learning rate* memisahkan laju pembelajaran (*learning rate*) untuk U-Net dan *text encoder*, sehingga sensitivitas pembaruan keduanya dapat diatur secara terpisah. Dalam penelitian ini digunakan nilai *learning rate* masing-masing **0.0001** dan **0**. *Network dimension* dan *network alpha* mengatur dimensi dan intensitas adaptasi matriks LoRA, dengan nilai masing-masing **16** dan **8**. Karena dalam penelitian ini jenis LoRA yang digunakan adalah *LoRA for convolutional networks* (LoCon), maka terdapat dua *hyperparameter* tambahan: *convolutional dimension* dan *convolutional alpha*, dengan nilai masing-masing **16** dan **8**. Kedua parameter tersebut bekerja serupa dengan *network dimension* dan *alpha*, namun diterapkan pada lapisan konvolusional, yang memungkinkan detail visual seperti tekstur dan pencahayaan ditangkap dalam pelatihan.

Kemudian terdapat pengaturan *optimizer_arguments*, yang mengendalikan mekanisme pembaruan bobot. *Learning rate scheduler* menentukan kurva perubahan *learning rate* sepanjang pelatihan. Dalam penelitian ini digunakan *scheduler Cosine with Restart*, dengan nilai *cycles* 3. *Cycles* mengatur siklus atau berapa kali *learning rate* akan turun ke titik minimum lalu meningkat kembali selama keseluruhan proses pelatihan.

Modul 4. 4 *Source code* pembuatan file konfigurasi; pengaturan logistik pelatihan

```
def create_config():
    ...
    "training_arguments": {
        "max_train_steps": max_train_steps,
        "max_train_epochs": max_train_epochs,
        "save_every_n_epochs": save_every_n_epochs,
        "save_last_n_epochs": keep_only_last_n_epochs,
        "train_batch_size": train_batch_size,
        "noise_offset": None,
        "clip_skip": 2,
        "min_snr_gamma": min_snr_gamma_value,
        "weighted_captions": weighted_captions,
        "seed": 42,
        "max_token_length": 225,
        "xformers": XFORMERS,
        "lowram": LOW_RAM,
        "max_data_loader_n_workers": 8,
        "persistent_data_loader_workers": True,
        "save_precision": "fp16",
        "mixed_precision": "fp16",
        "output_dir": output_folder,
        "logging_dir": log_folder,
        "output_name": project_name,
        "log_prefix": project_name,
    },
    ...
)
```

Terakhir dalam konfigurasi pelatihan yaitu pengaturan *training_arguments*, yang merangkum merangkum logistik pelatihan. *Maximum training steps* dan *maximum*

training epochs mengatur batas durasi pelatihan, dalam penelitian ini digunakan nilai *maximum training epoch* 10. *Save every n-epoch* mengatur frekuensi *checkpointing*, dan *save last n-epoch* mengatur berapa epoch terakhir yang *checkpoint*-nya akan disimpan. *Train batch size* mengontrol ukuran *batch* dalam *training*; seberapa banyak gambar yang diproses secara bersamaan dalam satu langkah pembaruan parameter. *xformers* dan *lowram* diaktifkan untuk mengaktifkan operator memori-efisien serta strategi pemakaian memori konservatif.

Modul 4.5 Source code pembuatan file konfigurasi; pengelolaan data, preprocessing data (*bucketing*), dan augmentasi data

```
def create_config():
    """
    dataset_config_dict = {
        'general': {
            'resolution': resolution,
            'shuffle_caption': shuffle_caption,
            'keep_tokens': keep_tokens,
            'flip_aug': flip_aug,
            'color_aug': color_aug,
            'random_crop': random_crop,
            'caption_extension': caption_extension,
            'enable_bucket': True,
            'bucket_reso_steps': 64,
            'bucket_no_upscale': False,
            'min_bucket_reso': 320 if resolution > 640 else 256,
            'max_bucket_reso': 1280 if resolution > 640 else 1024,
        },
        'datasets': toml.loads(custom_dataset)['datasets'] if custom_dataset else
    [
        {
            'subsets': [
                {
                    'num_repeats': num_repeats,
                    'image_dir': images_folder,
                    'class_tokens': None if caption_extension else project_name
                }
            ]
        }
    ]
    """
```

Dalam konfigurasi dataset, terdapat komponen *general* yang merangkum mekanisme *preprocessing* dataset. **Resolution** mengatur resolusi maksimal citra dalam pelatihan dalam penelitian ini digunakan resolusi 768×768 . *cache_latents=True* menginstruksikan untuk terlebih dahulu mengekstrak laten VAE dari semua citra dan menyimpannya sementara di dalam *disk*, trainer dapat langsung menggunakannya alih-alih melakukan *encode* ulang tiap iterasinya. Untuk *preprocessing* data, mekanisme *bucketing* diaktifkan. Dalam mekanisme *bucketing*, terdapat tiga pengaturan, yaitu *bucket resolution steps*, *minimum bucket resolution*, dan *maximum bucket resolution*. *Bucket resolution steps* mengatur interval resolusi dalam pembagian *bucket*, sedangkan *minimum bucket resolution*, dan *maximum bucket resolution* membatasi rentang resolusi minimum dan maksimum *bucket*. Nilai minimum dan maksimum *bucket* ditentukan masing-masing 320 dan 1280

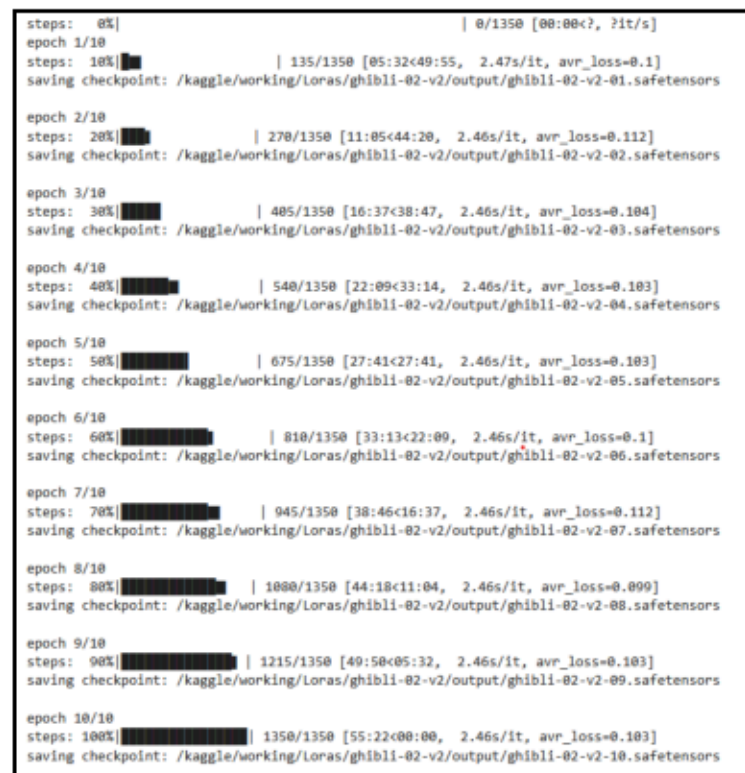
dengan interval **64**. Mekanisme *bucketing* ini memungkinkan proses *downscaling* dengan tetap mempertahankan rasio aspek citra, apabila resolusi citra melebihi resolusi maksimum.

Fungsi kemudian melakukan pembersihan sebelum membuat file konfigurasi. Semua entri bernilai `None` dibuang agar konfigurasi ringkas dan bebas parameter tak valid. Hasilnya adalah dua file konfigurasi, yaitu konfigurasi pelatihan (parameter model, optimisasi, logistik, penyimpanan) dan konfigurasi dataset (resolusi, augmentasi, *bucketing*, dan daftar dataset).

Modul 4.6 Memulai menjalankan proses pelatihan

```
'accelerate launch --config_file=(accelerate_config_file) --  
num_cpu_threads_per_process=1 train_network_wrapper.py --  
dataset_config=(dataset_config_file) --config_file=(config_file)
```

Terakhir, setelah dua file konfigurasi dibuat, proses pelatihan dimulai dengan menjalankan *script trainer* `train_network.py` dari *library kohya-ss*, dengan file `dataset_config_file`, dan `config_file` sebagai konfigurasi dataset dan *hyperparameter* pelatihannya.



Gambar 4.4 Hasil *average loss* pada tiap *epoch* selama proses pelatihan

Selama proses pelatihan, setiap *epoch* menghasilkan nilai *average loss*, (dapat dilihat di **Gambar 4.4**). Meskipun nilai tersebut merefleksikan kemampuan model dalam mempelajari pola *noise* pada citra, nilai tersebut tidak selalu berbanding lurus dengan

kualitas gambar yang dihasilkan saat inferensi. Oleh karena itu, evaluasi kualitas keluaran model tetap perlu dilakukan secara manual pada tahap inferensi.

4.1.3 Implementasi Sistem

Implementasi model dibuat ke dalam bentuk aplikasi *image-to-image translation* gambar bergaya animasi menggunakan Stable Diffusion dan pengondisian tambahan ControlNet berbasis citra tepi Canny. Tampilan halaman dalam aplikasi ini seperti yang telah dijabarkan pada bab 3.



Gambar 4.5 Tampilan utama aplikasi

Tampilan UI aplikasi ini dibuat menggunakan library *streamlit*. Pada bagian pertama, terdapat sebuah *file uploader* untuk mengunggah citra yang akan ditransformasi. File citra yang diinput oleh pengguna, apabila memiliki resolusi lebih besar dari 768×768 , akan di-*downscale* dengan tetap mempertahankan rasio aspeknya, melalui fungsi pada Modul 4. 7.

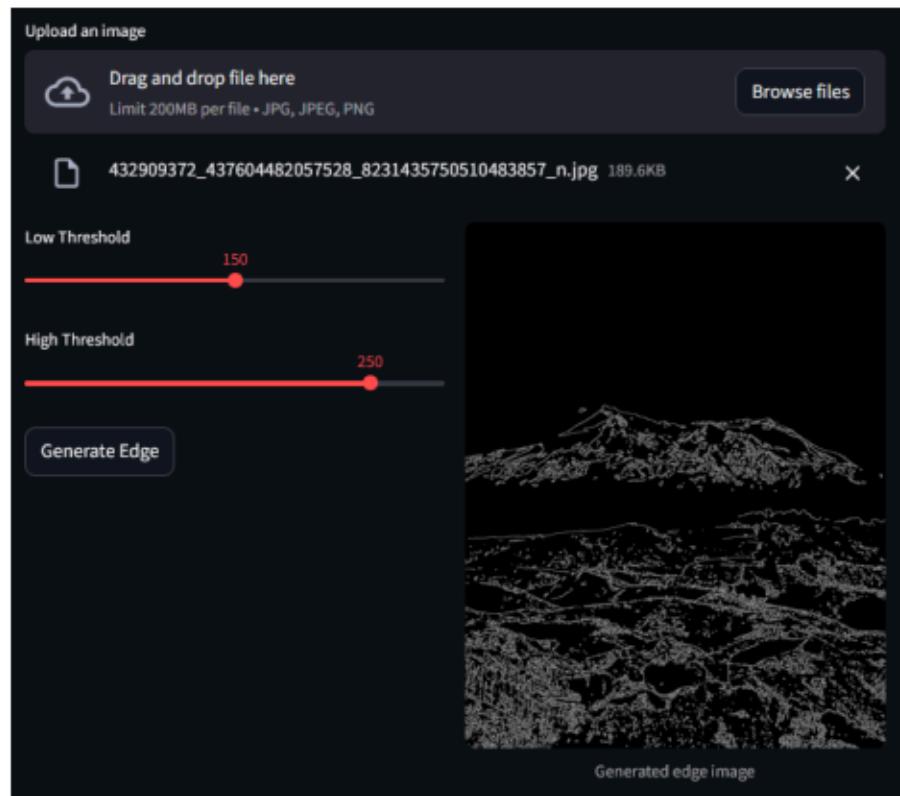
Modul 4. 7 Source code mekanisme *resize* citra input

```
def resize(image, width, height):  
    aspect_ratio = width / height  
    resized_width = int(math.sqrt(max_area * aspect_ratio))  
    resized_height = int(max_area / resized_width)
```

Modul 4.7 *Source code* mekanisme *resize* citra input

```
resized_image = init_image.resize((resized_width, resized_height))  
return resized_image
```

Selanjutnya, pada bagian bawah *file uploader* terdapat menu dua kolom yang mengatur pembuatan citra tepi Canny. Pada kolom bagian kiri, terdapat dua *slider* untuk mengatur nilai *low threshold* dan *high threshold*, dan tombol “**Generate Edge**” untuk memulai proses transformasi citra tepi Canny.



Gambar 4.6 Proses pembuatan citra tepi Canny

Pembuatan citra tepi Canny dilakukan menggunakan fungsi *Canny* dari *library* OpenCV. Pembuatan citra tepi Canny ini menggunakan tiga parameter, yaitu *low threshold*, *high threshold*, dan kernel menggunakan nilai *default* 3×3 .

Modul 4.8 *Source code* pembuatan citra tepi Canny

```
def generate_edge(image_in, low_threshold, high_threshold):  
    edge_image = np.array(image_in)  
    edge_image = cv2.Canny(edge_image, low_threshold, high_threshold)  
    edge_image = edge_image[:, :, None]  
    edge_image = np.concatenate([edge_image, edge_image, edge_image], axis=2)  
    edge_image = Image.fromarray(edge_image)  
    return edge_image
```

Setelah itu, terdapat menu pengaturan terkait parameter *denoising*, yaitu *text prompt*, dan tiga slider mencakup *guidance scale*, *strength*, dan jumlah *step*. Kemudian pada bagian paling bawah terdapat tombol “**Generate Image**.” Tombol ini memanggil model ControlNet *pipeline* Stable Diffusion, dan memulai proses pembuatan gambar. Model dasar *stable diffusion* yang digunakan dalam inferensi adalah AnyLoRA.

Modul 4.9 Source code proses pembuatan gambar

```
controlnet = ControlNetModel.from_pretrained("lillyasviel/sd-controlnet-
canny", torch_dtype=torch.float16)
pipeline = AutoPipelineForImage2Image.from_pretrained("Lykon/AnyLoRA",
controlnet=controlnet, torch_dtype=torch.float16).to("cuda")
pipeline.load_lora_weights(".", weight_name="ghibli01-07.safetensors",
adapter_name="anime")
pipeline.set_adapters("anime", adapter_weights=1.0)

# Generate image
image_out = pipeline(
    prompt,
    image=st.session_state.init_image,
    control_image=st.session_state.edge_image,
    strength=strength,
    guidance_scale=guidance_scale,
    num_inference_steps=num_inference_steps,
).images[0]
st.image(image_out, caption="Generated Image")
```

Fungsi di atas kemudian memulai proses *image-to-image translation*, yang membuat citra fotografis menjadi bergaya animasi. Contoh proses *image-to-image translation* dapat dilihat dalam **Gambar 4.7**.



Gambar 4. 7 Contoh penerapan *image-to-image translation*

4.2 Hasil

Bagian ini menampilkan hasil pengimplementasian model dan sistem yang telah dibangun, mencakup hasil pengujian model dan hasil perancangan sistem.

4.2.1 Hasil Pengujian Model

Pengujian performa model dilakukan dengan menggunakan tiga metrik pengukuran, yaitu: SSIM dan LPIPS untuk mengukur kemiripan struktural citra keluaran dengan citra referensi, juga skor penilaian pakar untuk mengukur kemiripan citra output dengan data latih. Pengujian dilakukan terhadap 20 citra fotografis yang diambil dari situs web *Unsplash*.

Dalam proses inferensi, digunakan nilai-nilai parameter yang sama untuk seluruh data uji, yaitu $\text{strength} = 0.7$, $\text{guidance scale} = 2.0$, $\text{timesteps} = 50$, dan $\text{LoRA.adapter weight} = 1.0$.

Khusus untuk inferensi menggunakan ControlNet, citra tepi Canny dibuat dengan nilai parameter $\text{low threshold} = 150$, dan $\text{high threshold} = 250$. Dalam ControlNet, juga terdapat parameter $\text{conditioning scale}$ yang mengukur seberapa besar pengaruh ControlNet dalam inferensi. Dalam penelitian ini digunakan nilai $\text{default conditioning scale} = 1.0$.



Gambar 4. 8 Contoh hasil pengujian pada salah satu data uji

Hasil pengujian terhadap tiga metrik penilaian tersebut dijabarkan dalam tiga tabel di bawah. Tiap tabelnya terdapat tiga kolom, yaitu kolom 'nomor' untuk tiap data uji citra referensi, 'ControlNet' untuk hasil inferensi menggunakan ControlNet, dan 'No ControlNet' untuk hasil inferensi tanpa ControlNet.

Tabel 4. 1 merupakan hasil pengujian terhadap metrik SSIM. Semakin tinggi skor SSIM, maka semakin tinggi tingkat kemiripan struktural antara dua citra.

Tabel 4. 1 Tabel hasil pengujian SSIM

SSIM ↑ (lebih tinggi lebih baik)		
No.	ControlNet	No ControlNet
1	0.513	0.538 (+ 0.025)
2	0.573 (+ 0.169)	0.404
3	0.489 (+ 0.131)	0.358
4	0.477 (+ 0.133)	0.344
5	0.399 (+ 0.132)	0.267
6	0.351 (+ 0.096)	0.255

Tabel 4. 1 Tabel hasil pengujian SSIM

SSIM ↑ (lebih tinggi lebih baik)		
No.	ControlNet	No ControlNet
7	0.397 (+ 0.083)	0.314
8	0.33 (+ 0.117)	0.213
9	0.447 (+ 0.121)	0.326
10	0.49 (+ 0.092)	0.398
11	0.527 (+ 0.014)	0.513
12	0.522 (+ 0.1)	0.422
13	0.343 (+ 0.118)	0.225
14	0.551 (+ 0.173)	0.378
15	0.325 (+ 0.06)	0.265
16	0.271 (+ 0.075)	0.196
17	0.683 (+ 0.082)	0.601
18	0.81 (+ 0.0583)	0.752
19	0.401 (+ 0.026)	0.375
20	0.331 (+ 0.054)	0.277

Tabel 4. 2 merupakan hasil pengujian terhadap metrik LPIPS. Semakin rendah skor LPIPS, maka semakin tinggi tingkat kemiripan struktural antara dua citra.

Tabel 4. 2 Tabel hasil pengujian LPIPS

LPIPS ↓ (lebih rendah lebih baik)		
No.	ControlNet	No ControlNet
1	0.484 (- 0.048)	0.532
2	0.412 (- 0.172)	0.584
3	0.484 (- 0.139)	0.623
4	0.542 (- 0.077)	0.619
5	0.556 (- 0.104)	0.66
6	0.506 (- 0.097)	0.603
7	0.507 (- 0.116)	0.623
8	0.429 (- 0.13)	0.559
9	0.449 (- 0.157)	0.606
10	0.453 (- 0.118)	0.571
11	0.459 (- 0.084)	0.543
12	0.471 (- 0.113)	0.584
13	0.474 (- 0.183)	0.657
14	0.467 (- 0.15)	0.617
15	0.476 (- 0.128)	0.604
16	0.524 (- 0.096)	0.62
17	0.419 (- 0.106)	0.525
18	0.401 (- 0.047)	0.448
19	0.475 (- 0.078)	0.553

Tabel 4. 2 Tabel hasil pengujian LPIPS

LPIPS ↓ (lebih rendah lebih baik)		
No.	ControlNet	No ControlNet
20	0.519 (-0.112)	0.631

Tabel 4. 3 merupakan hasil pengujian oleh pakar. Skor penilaian mencerminkan kemiripan gaya citra output dengan gaya pada data latih. Semakin tinggi skor, maka semakin tinggi kualitas tekstur citra yang dihasilkan.

Tabel 4. 3 Tabel hasil pengujian pakar

Skor penilaian pakar ↑ (lebih tinggi lebih baik)		
No.	ControlNet	No ControlNet
1	3	7 (+4)
2	3	7 (+4)
3	4	9 (+5)
4	4	8 (+4)
5	5	9 (+4)
6	8	9 (+1)
7	8	9 (+1)
8	5	9 (+4)
9	9	9
10	9	9
11	8	9 (+1)
12	7	9 (+2)
13	7	9 (+2)
14	8	9 (+1)
15	6	9 (+3)
16	9	9
17	9	9
18	7	7
19	8	9 (+4)
20	7 (+4)	6

4.2.2 Hasil Pengujian Sistem

Hasil pengujian sistem dilakukan menggunakan metode *black box*. Pengujian ini bertujuan untuk mengevaluasi kinerja fitur-fitur di dalam sistem.

Tabel 4. 4 *functional test case* pengujian sistem

No	Test case	Input	Expected Output	Status
1.	Mengunggah citra referensi	File citra RGB berformat .jpg, .jpeg, .png	Gambar berhasil dimuat	Lulus

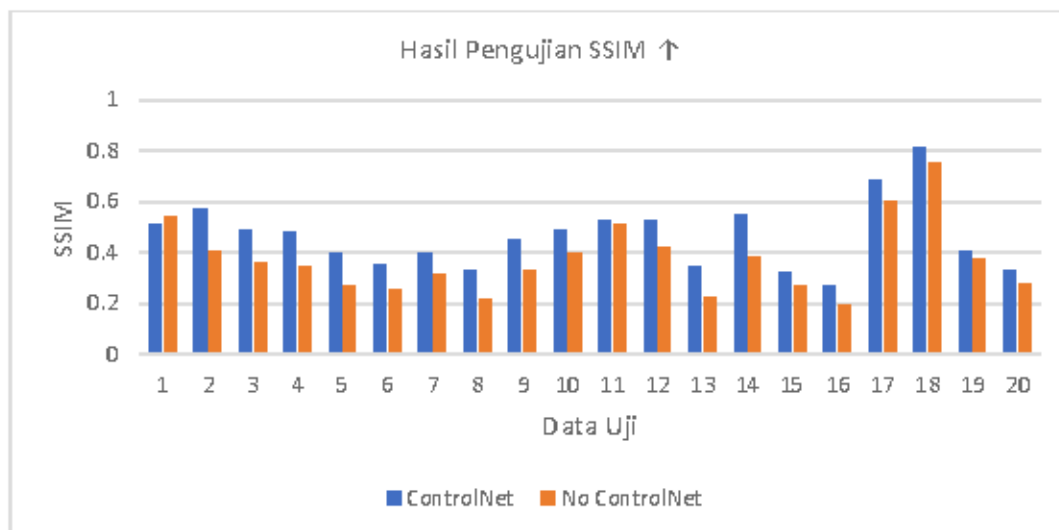
Tabel 4. 4 *functional test case* pengujian sistem

No	Test case	Input	Expected Output	Status
2.	Mengganti citra referensi	File citra RGB berformat .jpg, .jpeg, .png	Gambar berhasil dimuat, dan citra tepi Canny dari sesi sebelumnya di-reset.	Lulus
3.	Transformasi citra tepi Canny	File citra RGB berformat .jpg, .jpeg, .png	Citra tepi Canny dari citra referensi yang telah diunggah	Lulus
4.	Transformasi citra menggunakan stable diffusion	File citra RGB, citra tepi Canny, text prompt, parameter <i>guidance scale</i> , <i>strength</i> , dan <i>timestep</i>	Citra hasil transformasi	Lulus

4.3 Pembahasan

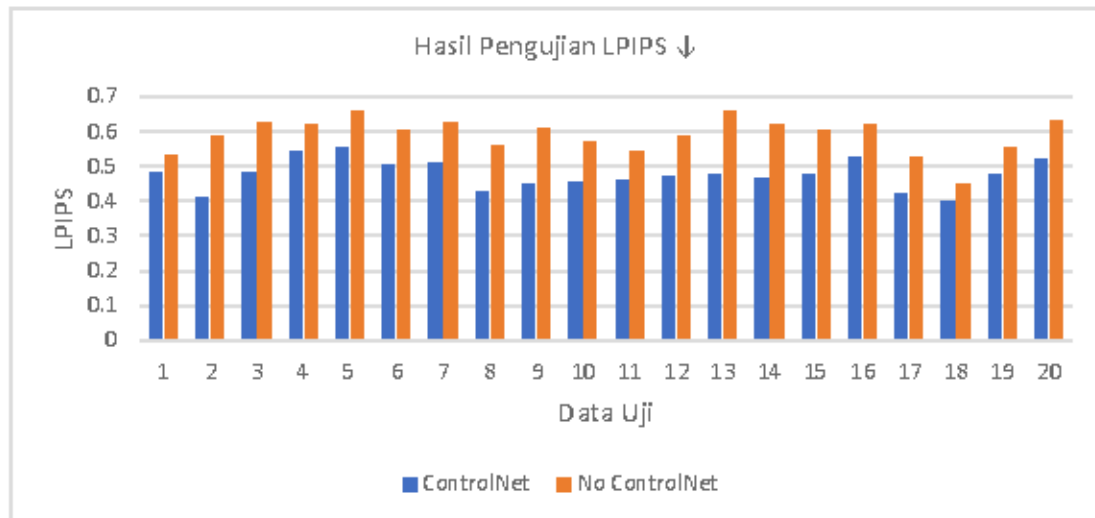
Bagian ini menjelaskan hasil penelitian yang diperoleh untuk menjawab rumusan masalah yang diangkat, yaitu bagaimana pengaruh kombinasi Stable Diffusion dengan ControlNet berbasis citra tepi Canny terhadap ketahanan struktural konten hasil *image-to-image translation* dalam pembuatan *background art* bergaya animasi.

Dalam metrik pengujian SSIM, inferensi menggunakan ControlNet dengan konsisten menghasilkan skor yang lebih tinggi. Terlihat dalam **Gambar 4. 9**, inferensi dengan ControlNet menghasilkan skor SSIM yang lebih tinggi pada hampir semua data uji, kecuali pada data uji no 1. Berdasarkan skor yang didapat dari pengujian ini, penggunaan ControlNet dalam inferensi meningkatkan rata-rata SSIM sebesar **0,0904**.



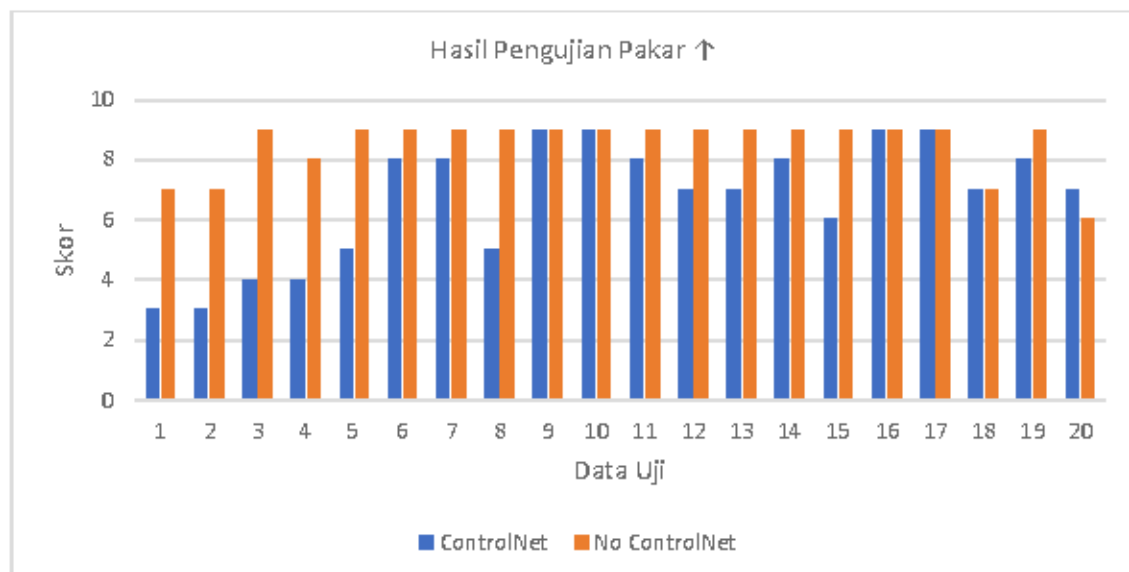
Gambar 4. 9 Perbandingan skor SSIM di seluruh data uji

Dalam metrik pengujian LPIPS, terdapat peningkatan skor yang lebih konsisten. Terlihat dalam **Gambar 4. 10**, inferensi dengan ControlNet menghasilkan skor LPIPS yang lebih rendah pada semua dataset, dengan rata-rata perubahan skor **0,112**.



Gambar 4. 10 Perbandingan skor LPIPS di seluruh data uji

Dalam pengujian oleh pakar, terdapat fenomena yang berkebalikan dari dua metrik pengujian sebelumnya, inferensi tanpa ControlNet menunjukkan kualitas yang lebih baik. Terlihat dalam **Gambar 4. 11**, meskipun inferensi menggunakan ControlNet mampu mengimbangi untuk beberapa data uji, inferensi tanpa ControlNet menghasilkan skor yang lebih tinggi untuk mayoritas data uji. Berdasarkan skor yang didapat dari pengujian ini, penggunaan ControlNet dalam inferensi mengurangi rata-rata skor sebesar **1,8**.



Gambar 4. 11 Perbandingan skor pakar di seluruh data uji

Hasil pengujian dari output model menghasilkan nilai rata-rata SSIM untuk output menggunakan ControlNet $SSIM = 0,461$ dan tanpa ControlNet $SSIM = 0,37$. Sementara untuk nilai rata-rata LPIPS untuk output menggunakan ControlNet $LPIPS = 0,475$ dan tanpa ControlNet $LPIPS = 0,588$. Dari kedua metrik penilaian tersebut, terlihat bahwa pengombinasian model dengan ControlNet berbasis Canny meningkatkan nilai rata-rata skor secara konsisten, menandakan terjadinya dalam ketahanan struktural konten dalam citra output.

Dalam pengujian terhadap kualitas tekstur melalui penilaian oleh pakar, citra output tanpa ControlNet menghasilkan nilai rata-rata yang lebih tinggi yaitu **8,5**. Sementara untuk citra output menghasilkan nilai rata-rata lebih rendah yaitu **6,7**, dengan range yang kurang konsisten, dari **3** hingga **9**.

Tabel 4. 5 Rata-rata skor SSIM, LPIPS, dan pakar untuk inferensi ControlNet dan tanpa ControlNet

Inferensi	SSIM	LPIPS	Pakar
ControlNet	<u>0,461</u>	<u>0,475</u>	6,7
No ControlNet	0,371	0,588	<u>8,5</u>

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil evaluasi dari 20 data uji, pengombinasian Stable Diffusion dengan ControlNet berbasis citra tepi Canny memberi pengaruh terhadap ketahanan struktur objek hasil *image-to-image translation* dalam pembuatan *background art* bergaya animasi. Pengombinasian Stable Diffusion dengan ControlNet berbasis citra tepi Canny menghasilkan peningkatan konsisten dalam ketahanan struktur objek dalam citra output, tampak dalam peningkatan rata-rata skor SSIM dan pengurangan skor LPIPS. Dalam metrik **SSIM**, inferensi menggunakan ControlNet menghasilkan peningkatan skor rata-rata **0,0904**, sementara dalam metrik **LPIPS** menghasilkan pengurangan skor rata-rata **1,112**.

5.2 Saran

Pada penelitian ini masih terdapat kekurangan dan dapat dikembangkan lebih lanjut. Untuk meningkatkan kemampuan dan mengembangkan penelitian mengenai implementasi *stable diffusion*, atau model difusi lainnya, terdapat beberapa saran dan ide yang dapat dieksplorasi untuk penelitian kedepannya, yaitu:

1. Dalam pengumpulan data, data tidak hanya dikumpulkan dari film animasi saja, melainkan dari karya-karya lain dari berbagai medium, baik digital maupun tradisional. Dengan ini, data latih yang dikumpulkan dapat merangkum range objek dan gaya yang lebih luas.
2. Pengujian lebih lanjut terhadap variasi kombinasi parameter inferensi untuk menghasilkan *trade-off* yang paling optimal agar menghasilkan output yang terbaik.
3. Penggabungan antara pengondisian tekstual dan spasial, seperti dengan mengimplementasikan BLIP untuk melakukan *prompting* secara otomatis.
4. Implementasi jenis controlnet lain, seperti controlnet berbasis *depth map*.
5. Implementasi *multi-controlnet* untuk menggabungkan berbagai jenis pengondisian spasial lain, seperti *depth map*, *sketch*, dan *human pose* dan lain-lain.

DAFTAR PUSTAKA

- Ba, Jimmy Lei, Jamie Ryan Kiros and Geoffrey E. Hinton. 2016. "Layer Normalization", July. <<http://arxiv.org/abs/1607.06450>>.
- Cai, Qiang, Mengxu Ma, Chen Wang and Haisheng Li. 2023. "Image Neural Style Transfer: A Review". *Computers and Electrical Engineering* 108. <<https://doi.org/10.1016/j.compeleceng.2023.108723>>.
- Canny, John. 1986. "A Computational Approach to Edge Detection". *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-8: 679–698. <<https://doi.org/10.1109/TPAMI.1986.4767851>>.
- Chen, Jie, Gang Liu and Xin Chen. 2020. "AnimeGAN: A Novel Lightweight GAN for Photo Animation". In: . *Communications in Computer and Information Science*. Springer. 1205 CCIS: 242–256. <https://doi.org/10.1007/978-981-15-5577-0_18>.
- Chen, Yang, Yu-Kun Lai and Yong-Jin Liu. 2018. "CartoonGAN: Generative Adversarial Networks for Photo Cartoonization". *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Dhariwal, Prafulla and Alex Nichol. 2021. "Diffusion Models Beat GANs on Image Synthesis", May. <<http://arxiv.org/abs/2105.05233>>.
- Elfwing, Stefan, Eiji Uchibe and Kenji Doya. 2017. "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", February. <<http://arxiv.org/abs/1702.03118>>.
- Ghazanfari, Sara, Siddharth Garg, Prashanth Krishnamurthy, Farshad Khorrami and Alexandre Araujo. 2023. "R-LPIPS: An Adversarially Robust Perceptual Similarity Metric", July. <<http://arxiv.org/abs/2307.15157>>.
- Gonzalez, Rafael C. and Richard E. Woods. 2018. *Digital Image Processing*. Pearson.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren and Jian Sun. 2015. "Deep Residual Learning for Image Recognition", December. <<http://arxiv.org/abs/1512.03385>>.
- Ho, Jonathan, Ajay Jain and Pieter Abbeel. 2020. "Denoising Diffusion Probabilistic Models", June. <<http://arxiv.org/abs/2006.11239>>.
- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang and Weizhu Chen. 2021. "LoRA: Low-Rank Adaptation of Large Language Models", June. <<http://arxiv.org/abs/2106.09685>>.
- Hugging Face. n.d. "Diffusers Documentation". accessed February 12, 2026a <<https://huggingface.co/docs/diffusers/index>> [accessed 12 February 2026].
- Hugging Face. n.d. "StableDiffusionControlNet Pipeline". accessed December 15, 2025b <<https://huggingface.co/docs/diffusers/v0.36.0/en/api/pipelines/controlnet#diffusers.StableDiffusionControlNetPipeline>> [accessed 15 December 2025].

Kehr, Dave. 2025. "Animation". In: . *Britannica*.

<<https://www.britannica.com/art/animation>> [accessed 21 October 2025].

Kingma, Diederik P. and Max Welling. 2019. "An Introduction to Variational Autoencoders", June. <<https://doi.org/10.1561/22000000056>>.

Lecun, Yann, Yoshua Bengio and Geoffrey Hinton. 2015. "Deep Learning". *Nature*. Nature Publishing Group. <<https://doi.org/10.1038/nature14539>>.

Meng, Chenlin, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu and Stefano Ermon. 2022. "SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations", January. <<http://arxiv.org/abs/2108.01073>>.

Michelucci, Umberto. 2022. "An Introduction to Autoencoders", January. <<http://arxiv.org/abs/2201.03898>>.

Nelis, Phelim. n.d. "Crafting Captivating Narratives: The Power of Animation Art Background". Squirt Gun Studios. accessed October 21, 2025 <<https://squirtgunstudios.com/crafting-captivating-narratives-the-power-of-animation-art-background/>> [accessed 21 October 2025].

OpenCV. n.d. "Canny Edge Detection". accessed February 12, 2026 <https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html> [accessed 12 February 2026].

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser and Björn Ommer. 2021a. "High-Resolution Image Synthesis with Latent Diffusion Models", December. <<http://arxiv.org/abs/2112.10752>>.

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser and Björn Ommer. 2021b. "High-Resolution Image Synthesis with Latent Diffusion Models", December. <<http://arxiv.org/abs/2112.10752>>.

Ronneberger, Olaf, Philipp Fischer and Thomas Brox. 2015. "U-Net: Convolutional Networks for Biomedical Image Segmentation", May. <<http://arxiv.org/abs/1505.04597>>.

Safayani, Mehran, Behnaz Mirzapour, Hanieh aghaebrahimiyan, Nasrin Salehi and Harnid Ravvae. 2025. "Unpaired Image-to-Image Translation with Content Preserving Perspective: A Review", February. <<http://arxiv.org/abs/2502.08667>>.

Song, Jiaming, Chenlin Meng and Stefano Ermon. 2020. "Denoising Diffusion Implicit Models", October. <<http://arxiv.org/abs/2010.02502>>.

Vaswani, Ashish, Google Brain, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser and Illia Polosukhin. 2023. "Attention Is All You Need".

- Wang, Zhou, Alan C Bovik, Hamid R Sheikh and Eero P Simoncelli. 2004. "Image Quality Assessment: From Error Visibility to Structural Similarity". *IEEE TRANSACTIONS ON IMAGE PROCESSING*. Vol. 13. <<http://www.cns.nyu.edu/~lcv/ssim/>>.
- Wicaksono and Dimas Arief. 2018. "Perancangan Environment Dalam Animasi Pendek 'Concrete Jungle' ". Universitas Multimedia Nusantara. <<https://kc.umn.ac.id/id/eprint/2848/>> [accessed 21 October 2025].
- Wu, Yuxin and Kaiming He. 2018. "Group Normalization", March. <<http://arxiv.org/abs/1803.08494>>.
- Yeh, Shih-Ying, Yu-Guan Hsieh, Zhidong Gao, Bernard B W Yang, Giyeong Oh and Yanmin Gong. 2023. "Navigating Text-To-Image Customization: From LyCORIS Fine-Tuning to Model Evaluation", September. <<http://arxiv.org/abs/2309.14859>>.
- Zhang, Jeffrey, Shao-Yu Chang, Kedan Li and David Forsyth. 2024. "Preserving Image Properties Through Initializations in Diffusion Models", January. <<http://arxiv.org/abs/2401.02097>>.
- Zhang, Lvmin, Anyi Rao and Maneesh Agrawala. 2023a. "Adding Conditional Control to Text-to-Image Diffusion Models", February. <<http://arxiv.org/abs/2302.05543>>.
- Zhang, Richard, Phillip Isola, Alexei A. Efros, Eli Shechtman and Oliver Wang. 2018a. "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric", April. <<http://arxiv.org/abs/1801.03924>>.

LAMPIRAN

PROFIL PAKAR



Susilo Budi Purwanto

Tempat / Tanggal Lahir: Magelang, 26 Juli 1966

Alamat: Mejing Wetan, Gamping, Sleman, Yogyakarta
55294

Pendidikan: ISI Yogyakarta

Email: susilobp@gmail.com

- | | | | |
|-------------|--|-------------|--|
| 2017 | <ul style="list-style-type: none">• “Mitoni”, Omah Petruk Karang Klethak, Yogyakarta• “Imago Mundi”, Bentara Budaya Yogyakarta• Pameran Drawing “Purba”, Miracle Print, Yogyakarta• “Representasi”, Pendhapa Art Space, Yogyakarta• Pameran “Suka Pari Suka” ke 4, Plataran Joko Pekik, Yogyakarta• “Ilange Semar”, Bentara Budaya Yogyakarta | 2020 | <p>“Sugih Ora Nyimpen”, Penghormatan Untuk Jakob Oetama, Bentara Budaya Yogyakarta</p> |
| 2019 | <ul style="list-style-type: none">• Pameran “Kosen”, Bentara Budaya Yogyakarta• Group Exhibition “Exploration”, Titian Art Space, Bali• “Merayakan Optimisme”, Taman Budaya Yogyakarta• “Representasi #3”, Pendhapa Art Space, Yogyakarta• Pameran bertema “Pers”, Karang Klethak, Yogyakarta• Pameran “Mahardika”, Titian Art Space, Bali | 2021 | <ul style="list-style-type: none">• Pameran Tunggal “SUKROSONO”, Bentara Budaya Yogyakarta• Pameran Bersama “Menawar Isyarat”, Sangkring Art Space, Yogyakarta |
| | | 2022 | <ul style="list-style-type: none">• Pameran Bersama “60 tahun Hadiprana Gallery”, Jakarta• Pameran Bersama “Ajur Ajer”, Bentara Budaya Yogyakarta |
| | | 2023 | <ul style="list-style-type: none">• Pameran Bersama “Mikul Duwur Mendem Jero”, Bentara Budaya Yogyakarta• Pameran Bersama “Jateng Gayeng”, TBJT, Solo• Pameran Bersama “Pertemuan”, Bentara Budaya Yogyakarta• Pameran Tunggal “REFLEKSI”, Balai Budaya Jakarta |
| | | 2024 | <ul style="list-style-type: none">• Pameran Bersama “Tanik Tambang”, Bentara Budaya Yogyakarta• “Merefleksi 100 Tahun Surrealisme Andre Breton”, OHD Museum, Magelang |